Website Design with AI

A visual course you can follow yourself

By Paul Victor

Beloved Technology

Start with the mouse and keyboard. Learn how the internet works. Build with HTML, CSS, JavaScript, PHP and SQL, then use AI to improve work you understand.



25 focused practicals • 5 complete website projects

Notepad → VS Code → Codex and Qoder

Self paced edition • October 2026

No previous computer experience required.

Choose your next step

Click any title to jump there. In Word, hold Ctrl while clicking if needed. The full lesson index follows this page.

Part 01 Computer and internet
Open this part →

Part 02 Your first webpage
Open this part →

Part 03 HTML gives content a structure
Open this part →

Part 04 CSS gives the page its appearance
Open this part →

Part 05 JavaScript makes a page respond
Open this part →

Part 06 PHP works on the server
Open this part →

Part 07 SQL works with stored data
Open this part →

Part 08 Put the five subjects together
Open this part →

Part 09 Build with AI and explore more tools
Open this part →

Get the practice files **Meet Paul Victor**

Clickable lesson index

Welcome to your first website 11
Meet your course author 12
Download your course files 13
Find your way around the course 14
Part 01 Computer and internet 15
Computer lesson 1 Meet the mouse and keyboard 16
Computer lesson 2 Select copy paste and undo 17
Computer lesson 3 Make a folder and save your work 18
Internet lesson 1 What the internet actually is 19
Internet lesson 2 Open a browser and visit a website 20
Internet lesson 3 Names addresses and places to store a site 21
Internet lesson 4 Follow a webpage from request to screen 22
Internet lesson 5 Watch real requests in the browser 23
Internet lesson 6 Data travels in pieces 24
Internet lesson 7 Online local upload and download 25
Internet lesson 8 Meet the five subjects we will practise 26
Part 02 Your first webpage 27
Setup lesson 1 Make your first webpage in Notepad 28
Setup lesson 2 Open your page and change it 29
Setup lesson 3 Understand the few tags you used 30
Setup lesson 4 Make a safe copy before changing more 31
Setup lesson 5 Install Visual Studio Code 32
Setup lesson 6 Open your folder in VS Code 33
Setup lesson 7 Add Live Server to VS Code 34

Clickable lesson index

Get the practice files ready 35
Part 03 HTML gives content a structure 36
HTML help Save and reopen the right file 37
HTML foundation Build the document shell 38
HTML foundation Read tags and attributes 39
HTML practical 1 A personal introduction page 40
HTML practical 1 Output and code explanation 41
HTML practical 2 A restaurant menu page 42
HTML practical 2 Output and code explanation 43
HTML practical 3 An event information page 44
HTML practical 3 Output and code explanation 45
HTML practical 4 A workshop registration form 46
HTML practical 4 Output and code explanation 47
HTML practical 5 A school club home page 48
HTML practical 5 A school club home page code continued 2 49
HTML practical 5 Output and code explanation 50
Part 04 CSS gives the page its appearance 51
CSS help See which rule changes the page 52
CSS foundation Select and style the page 53
CSS practical 1 A clear business introduction 54
CSS practical 1 Output and code explanation 55
CSS practical 2 A product card with breathing room 56
CSS practical 2 Output and code explanation 57
CSS practical 3 A flexible navigation bar 58

Clickable lesson index

CSS practical 3 Output and code explanation 59
CSS practical 4 A responsive course card grid 60
CSS practical 4 Output and code explanation 61
CSS practical 5 A readable enquiry form 62
CSS practical 5 Output and code explanation 63
Part 05 JavaScript makes a page respond 64
JavaScript help Find the first error 65
JavaScript foundation Values decisions and actions 66
JavaScript practical 1 A visitor greeting on a page 67
JavaScript practical 1 Output and code explanation 68
JavaScript practical 2 A printing cost calculator 69
JavaScript practical 2 Output and code explanation 70
JavaScript practical 3 A small service catalogue 71
JavaScript practical 3 Output and code explanation 72
JavaScript practical 4 A frequently asked questions control 73
JavaScript practical 4 Output and code explanation 74
JavaScript practical 5 A catalogue search page 75
JavaScript practical 5 Output and code explanation 76
Part 06 PHP works on the server 77
PHP preparation Meet the terminal slowly 78
PHP foundation Install and check PHP 79
PHP help Start a server and fix setup errors 80
PHP foundation Where the code runs 81
PHP practical 1 A server generated welcome page 82

Clickable lesson index

PHP practical 1 Output and code explanation 83
PHP practical 2 A server calculated quotation page 84
PHP practical 2 Output and code explanation 85
PHP practical 3 A dynamic price list page 86
PHP practical 3 Output and code explanation 87
PHP practical 4 A validated estimate form 88
PHP practical 4 Output and code explanation 89
PHP practical 5 A session based visit counter page 90
PHP practical 5 Output and code explanation 91
Part 07 SQL works with stored data 92
SQL foundation Understand records and relationships 93
SQL foundation Open DB Browser and run a query 94
SQL help Run one statement and read the result 95
SQL practical 1 Create a product table 96
SQL practical 1 Output and code explanation 97
SQL practical 2 Find the products a visitor needs 98
SQL practical 2 Output and code explanation 99
SQL practical 3 Correct and remove specific records 100
SQL practical 3 Output and code explanation 101
SQL practical 4 Join customer and order information 102
SQL practical 4 Output and code explanation 103
SQL practical 5 Build a report and undo a trial change 104
SQL practical 5 Output and code explanation 105
Foundation checkpoint Before combining the languages 106

Clickable lesson index

Part 08 Put the five subjects together 107
Five complete websites How the parts work together 108
Website 1 Build a bakery website 109
Website 1 File config php 110
Website 1 File schema sql 111
Website 1 File setup php 112
Website 1 File app php 113
Website 1 File header php 114
Website 1 File footer php 115
Website 1 File public index php 116
Website 1 File public catalog php 117
Website 1 File public request php 118
Website 1 File public request php continued 2 119
Website 1 File public request php continued 3 120
Website 1 File public thanks php 121
Website 1 File public style css 122
Website 1 File public app js 123
Website 1 Inspect the saved record 124
Website 1 Explain the HTML CSS and JavaScript 125
Website 1 Explain PHP SQL and the request flow 126
Website 1 Expected output and practical checks 127
Website 2 A tailoring and branding enquiry website 128
Website 2 Configuration code 129
Website 2 Expected output and practical checks 130

Clickable lesson index

Website 3 A training centre course website 131
Website 3 Configuration code 132
Website 3 Expected output and practical checks 133
Website 4 A guesthouse room enquiry website 134
Website 4 Configuration code 135
Website 4 Expected output and practical checks 136
Website 5 A community event registration website 137
Website 5 Configuration code 138
Website 5 Expected output and practical checks 139
Part 09 Build with AI and explore more tools 140
Prepare the next AI practice project 141
Further lesson 1 Ask ChatGPT for help in ordinary English 142
Further lesson 2 Install Codex in VS Code 143
Further lesson 3 Let Codex make one small improvement 144
Further lesson 4 Save versions with GitHub Desktop 145
Further lesson 5 Work like a careful website designer 146
Further lesson 6 Choose the right kind of website tool 147
Further lesson 7 Make a page on WordPress com 148
Further lesson 8 Try WordPress software on your computer 149
Further lesson 9 Change a WordPress theme and try a plugin 150
Further lesson 10 Publish a short article with Blogger 151
Further lesson 11 Build a simple site with Google Sites 152
Further lesson 12 Briefly try Dreamweaver and a design tool 153
Further lesson 13 Try building with ChatGPT Sites 154

Clickable lesson index

Further lesson 14 What is a stack 155
Further lesson 15 Meet the terminal slowly 156
Further lesson 16 Start a Laravel project 157
Further lesson 17 Give Laravel your own page 158
Further lesson 18 Let Laravel save an enquiry 159
Further lesson 19 Start a Django project 160
Further lesson 20 Give Django your own page 161
Further lesson 21 Start a Flutter web project 162
Further lesson 22 Change the Flutter screen 163
Further lesson 23 A short visit to React 164
Further lesson 24 Meet Qoder IDE 165
Further lesson 25 Your final Qoder website 166
Further lesson 26 Give AI a clear finished test 167
Further lesson 27 Put the simple website online 168
Further lesson 28 Show what you can do 169
Your course completion record 170
Optional tutor notes Teach the person in front of you 171
Optional tutor notes Prepare the room and accounts 172
Optional tutor notes Use AI like a professional colleague 173
Optional tutor notes Important current tool details 174
Optional tutor notes Publish the right kind of project 175
Quick help When something goes wrong 176
Small glossary Words you have already used 177
Official sources 1 178

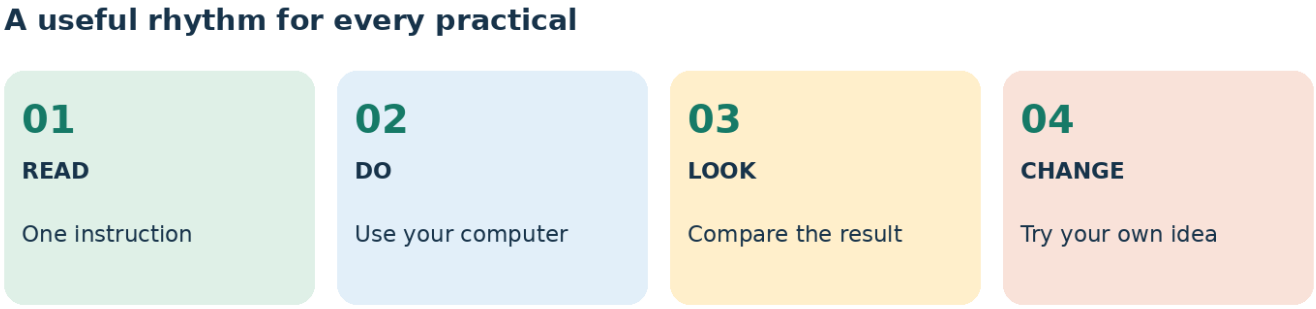
Clickable lesson index

Official sources 2 179
Official sources 3 180
Official sources 4 181
Optional tutor notes Assess the new foundation practicals 182
Foundation sources 1 183
Foundation sources 2 184

Welcome to your first website

You do not need to understand every word on the first day. Open the computer, follow one step and look at what changed. That small result is your starting point.

How this book teaches you



Teaching illustration • Simplified view

Read one instruction. Do it on your computer. Compare your screen with the result shown. Change one thing yourself. If it breaks, use the nearby fix box and try again.

Set your own pace

Allow 20–40 minutes for a small practice session. A page may take longer when typing is new. Stop at a working result, save your file and write the next step in notes.txt. There is no race to finish the book.

TRY IT When you reach Computer lesson 3, you will create notes.txt. Write your name and this sentence: I will build one small thing at a time.

The pictures explain controls and results. They are teaching diagrams unless a caption says browser capture. Sample names and prices are fictional. The five complete websites save local enquiries; they do not process payments or reserve rooms and seats.

READY TO CONTINUE You can use the book alone. If a computer belongs to a school or employer, you still need the owner's permission to install tools or use its accounts.

Meet your course author

Paul Victor

Paul Victor is a web developer, ICT trainer and digital media creator. His work connects practical computer education, website development and visual storytelling.

He previously served as a Lecturer and Technical Officer at Theofadek Business and Computer Studies. His teaching experience includes programming, website development, networking, database systems and desktop publishing.

His development portfolio includes BelovedSubP, a wallet and virtual top-up platform, and Beloved College, a school management project with student records, portals and result tracking. His listed web tools include HTML, CSS, JavaScript, PHP, Laravel, Git, GitHub and VS Code.

He also produces educational and technology content through Beloved Technology and Beloved Technology Media. This course brings those interests together: clear explanations, visible results and practice with real files.



Teaching illustration • Simplified view

Beloved Technology

This course is presented under the Beloved Technology name. Its learning approach is practical: begin with computer confidence, create small projects and build towards useful websites.

A public Facebook listing uses the name “Beloved Technology for Business and Computer Institute – BTECH”. Only the listed name is used here; no accreditation, founding date or campus claims are implied.

View the public BTECH listing ↗

Profile basis: Paul Victor’s supplied ICT teaching application, web-development portfolio and CVs. Personal contact details and private application information are not reproduced.

Download your course files

Everything you need for the practical work

01
READ THE BOOK
PDF or editable Word

02
GET THE ZIP
Extract the folder

03
OPEN A PROJECT
Keep your own copy



DOWNLOAD THE PRACTICE ZIP ↗

OPEN THE COURSE DOWNLOAD PAGE ↗

DOWNLOAD THE PDF HANDBOOK ↗

DOWNLOAD THE EDITABLE WORD HANDBOOK ↗

Save and open the ZIP

1. Click the practice ZIP link above. If your browser asks what to do, choose Save. In Word, hold Ctrl while clicking a link if a normal click only selects it.

2. Press Ctrl + J in your browser to see Downloads. Choose Show in folder beside Website_Design_Practice_Files.zip.

3. Right-click the ZIP → Extract All → Browse. Choose Documents → Website Class, then Extract. Do not work inside the unopened ZIP.

4. Open the extracted Website_Design_Practice_Files folder. Double-click START_HERE.html for a clickable list of exercises.

5. Create a separate My Practice folder. Copy just the exercise you need there before changing it.

IF IT DOES NOT WORK If a direct link opens a file page, use its Download raw file button. If a link is blocked in your document viewer, open the course download page in your browser.

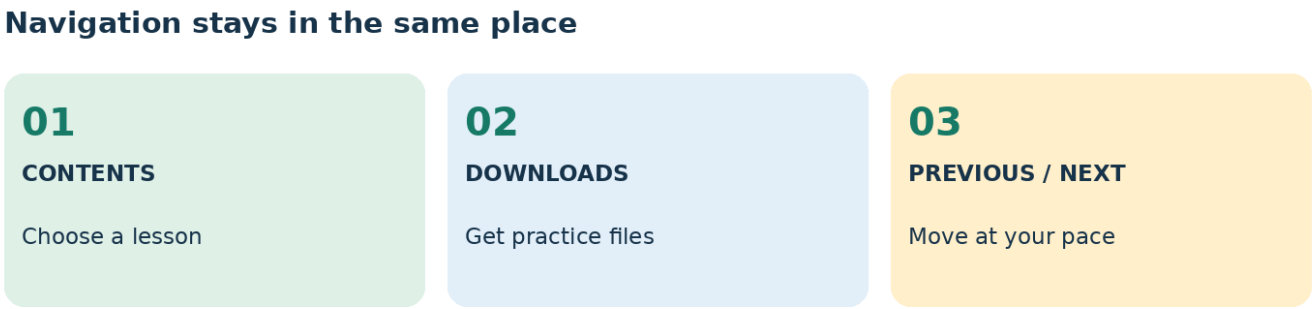
Use the PDF for reading and the files for typing practice. After downloading, the early HTML and CSS activities work offline. Tool installation, AI services and public publishing need internet access.

Find your way around the course

The coloured label at the top of each page tells you which part you are in. Click Contents to choose a lesson, Downloads to get the files, or Previous and Next to move through the book.

Click and jump

In the PDF, click a lesson in the contents or open your PDF reader’s Bookmarks panel. In Word, Ctrl + click a link if required. View → Navigation Pane also lists the headings.



Teaching illustration • Simplified view

Read these boxes as instructions

TRY IT Do this now. A small change helps you understand more than rereading the same paragraph.

IF IT DOES NOT WORK Something looks wrong? Check the suggested cause, change one thing and run it again.

READY TO CONTINUE Use this as a finish line. Move on when you can show the result, not only repeat the definition.

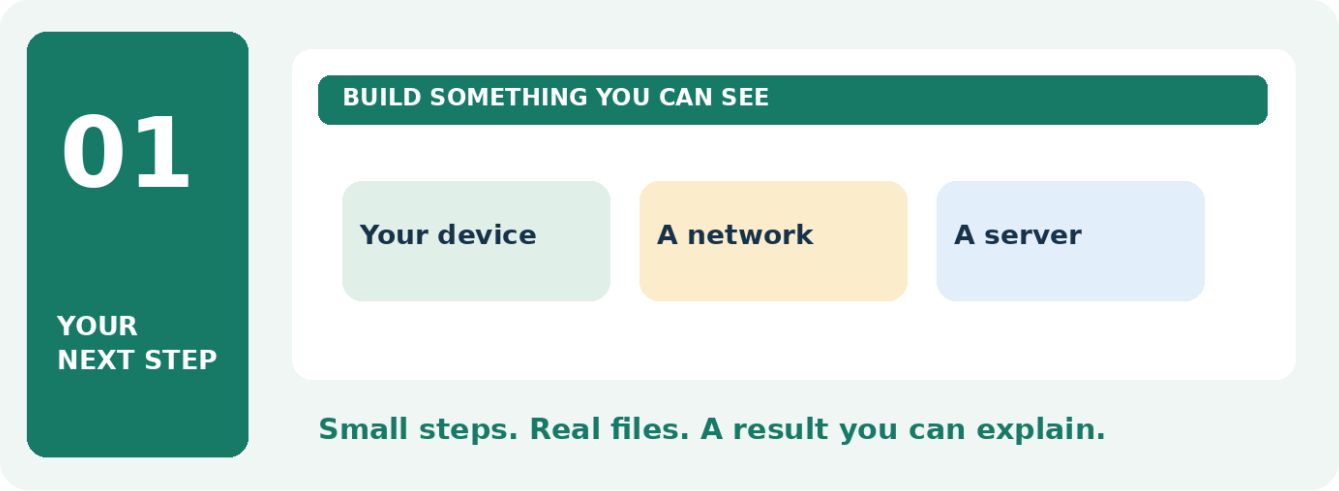
Keep a simple progress record

Write the lesson name, the date and one thing you made in notes.txt. Add the word DONE after completing its check. On a difficult day, improve one small part of a page you already built.

This is a reference book as well as a course. You can return to a code explanation whenever you need it.

Part 01 Computer and internet

Find your files and understand how a webpage reaches your screen.



Your finish line

Open Notepad and save a file. Follow a request through the browser, server and network.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

Computer lesson 1 Meet the mouse and keyboard

Use: your computer, mouse and keyboard.

Let us practise

1. Find the power button marked with a circle and a short line. Press it once and wait. Sign in with your own Windows account. On a shared computer, use the account provided by its owner.

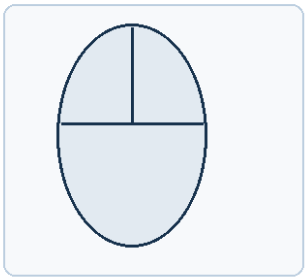
2. Move the mouse slowly. Watch the arrow move on the screen. That arrow is called the pointer.

3. Click an empty part of the desktop. The desktop is the main screen you see after signing in.

4. Find the Windows Start button on the taskbar, usually along the bottom. Click it once.

5. Type Notepad. Click the Notepad result. A writing window opens.

Find the controls before writing code



Left button: click

Right button: show choices

Ctrl + S Hold Ctrl, tap S

Teaching illustration • Simplified view

Type this

My name is Ada.

Press Enter. Type: I am learning to build websites.

Backspace removes the character just before the blinking line. That blinking line shows where your next letter will appear. Hold Shift while pressing a letter to make one capital letter.

Try it: change Ada to your name. Use the mouse to click where you want to type.

You are ready when: you can open Notepad and type two sentences without looking back at the steps.

Computer lesson 2 Select copy paste and undo

Use: Notepad. Keep the sentences from Computer lesson 1 open.

Follow these steps

1. Place the pointer just before your name.

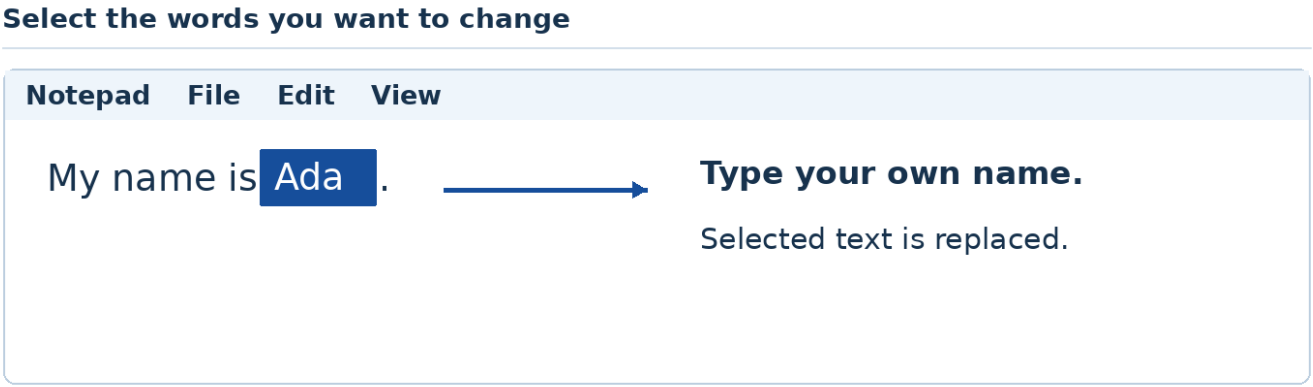
2. Hold the left mouse button. Move across your name. Release the button. The highlighted letters are selected.

3. Hold Ctrl and tap C. This copies the selected letters.

4. Click at the end of your second sentence. Press Enter, then Ctrl + V. Your copied name appears.

5. Press Ctrl + Z. Your last change is undone.

6. Press Ctrl + A. All the text is selected. Click once inside the text to remove the selection without deleting it.



Teaching illustration • Simplified view

Keep these shortcuts nearby

Shortcut	What it does
Ctrl + C	Copy the selected text
Ctrl + V	Paste the copied text
Ctrl + Z	Undo the last change
Ctrl + A	Select everything in the current writing area
Ctrl + S	Save your work

Be careful: typing while text is highlighted replaces that text. If you do it accidentally, press Ctrl + Z.

Your turn: copy one sentence and paste it on a new line. Then undo the paste.

Computer lesson 3 Make a folder and save your work

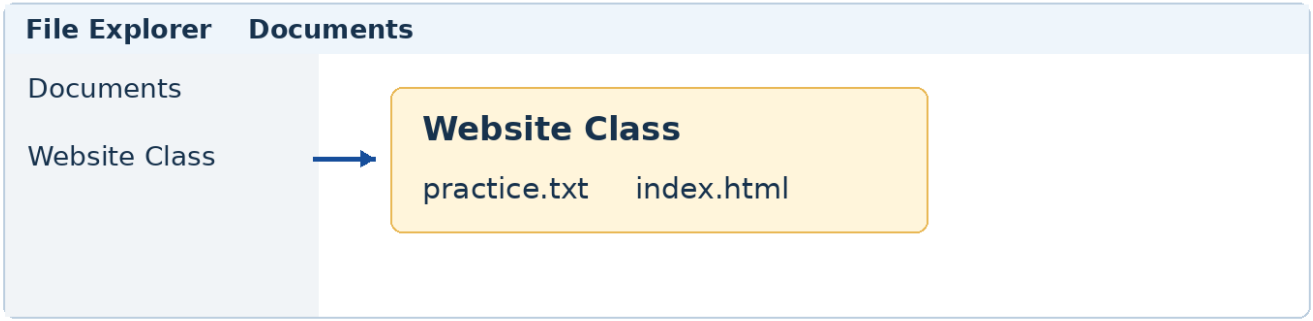
Use: File Explorer and Notepad.

A file holds one piece of work. A folder holds files. We will keep our website files together so we can find them again.

Make your course folder

1. Hold the Windows key and press E. This opens File Explorer.
2. Click Documents on the left.
3. Right-click an empty space. Choose New, then Folder.
4. Type Website Class and press Enter.
5. Double-click Website Class to open it.

A folder holds your files



Teaching illustration • Simplified view

Save your sentences

1. Return to Notepad by clicking its icon on the taskbar.
2. Click File, then Save as. On versions that show Save first, use that option for this new file.
3. Open Documents, then Website Class in the save window.
4. In File name, type practice.txt. Click Save.
5. Close Notepad using the X at the top right. In File Explorer, double-click practice.txt.

You should see: the same sentences you saved.

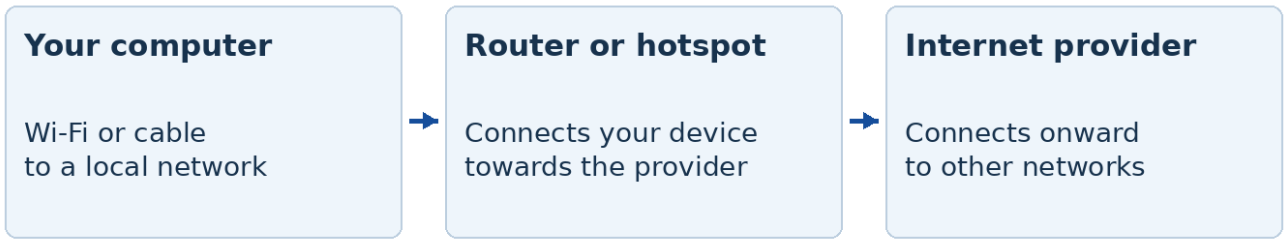
Your turn: add a third sentence. Press Ctrl + S. Close and reopen the file to check that the new sentence stayed.

Internet lesson 1 What the internet actually is

Use: your classroom computer and a simple drawing on paper.

The internet is a worldwide connection of networks. A network is a group of connected devices that can exchange information. Your phone, a school computer and a company's server can communicate through these connections.

Your device reaches other networks through a provider



Teaching illustration • Simplified view

Find the connection around you

1. Find the router that supplies your connection, or the phone whose hotspot you use. You do not need to change its settings.

2. On Windows, click the network symbol near the clock. Look at the connected network's name. Do not change its settings yet.

3. Draw your computer, the router or hotspot, and the internet provider. Join them with lines.

4. Say the route aloud or write it in your notes: “My computer connects here. This connection reaches our provider.”

Wi-Fi is a wireless connection to a nearby network. It does not guarantee internet access. A router can still be switched on when the provider's connection is down.

An ISP is an internet service provider: the company carrying your connection to other networks. Data can travel through fibre cables, other cables, radio links and mobile networks. The internet is not simply “inside the Wi-Fi box.”

Try it: identify whether your classroom connection uses a cable, Wi-Fi or a phone hotspot. If you know the network owner, ask which company provides the connection.

Check yourself: Wi-Fi can work while the router has no onward internet connection. A Wi-Fi symbol alone does not prove that websites are reachable.

Reference: N1.

Internet lesson 2 Open a browser and visit a website

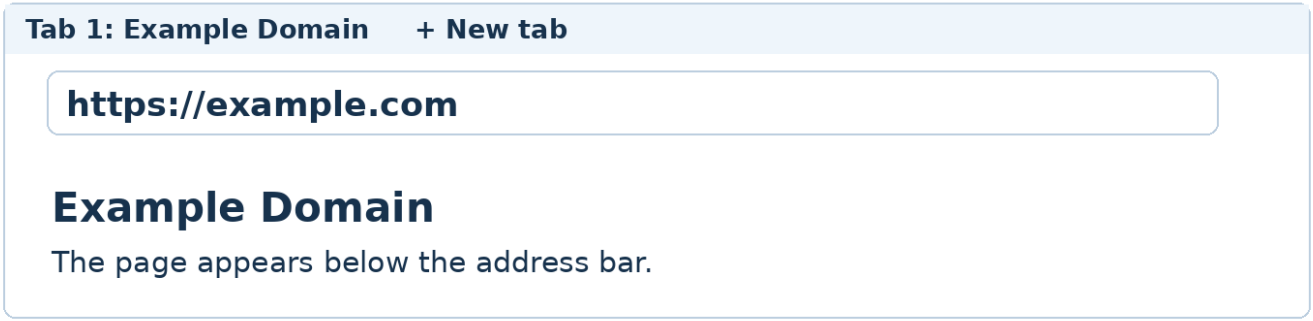
Use: Microsoft Edge or Google Chrome and a working internet connection.

The web is one service that uses the internet. Email and other services use it too. A browser is the program we use to open webpages. A search engine is a service that helps us find them.

Visit an address directly

1. Click Start. Type Edge and open Microsoft Edge. Chrome is also suitable if already installed.
2. Press Ctrl + L. This selects the address bar near the top.
3. Type https://example.com and press Enter. Wait for the page to load.
4. Press Ctrl + T to open another tab. Type a search such as what is a webpage and press Enter.
5. Compare the results page with the page you visited directly. Click the first tab to return.

A browser has tabs and an address bar



Teaching illustration • Simplified view

A tab keeps one page open while you visit another. The Back button returns to the previous page in that tab. Refresh asks the browser to load the current page again.

A website contains related pages. One page may be its Home page, another its Contact page. A link is an item you can click to move to a page or a place on a page.

Your turn: visit example.com without using a search result. Write the browser name and the search service name in notes.txt. They are different tools.

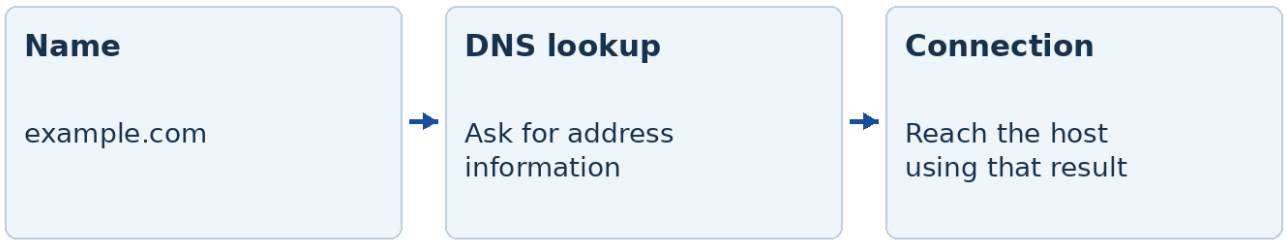
Reference: N2.

Internet lesson 3 Names addresses and places to store a site

Use: the browser and paper. No purchase is needed.

People remember a name such as example.com more easily than a numeric network address. DNS helps look up the network information needed to reach a name. An IP address helps networks deliver data to the right destination.

DNS helps resolve a host name to address information



Teaching illustration • Simplified view

A domain is a registered name. Hosting is the service running or storing the website. You can buy a name without yet having a working website. You can also test a website on your own computer without buying either.

Read this address together

https://example.com/cakes?size=small#prices

Part	Its job
https	Use HTTP through an encrypted connection
example.com	The host name to reach
/cakes	The path requested from that host
?size=small	Extra query information sent with the request
#prices	A place within the page; not sent as part of the HTTP request

This is an address example, not a promise that a cakes page exists there.

Your turn: in your notebook, invent a website name and two page paths. Circle the domain name and underline each path. Do not register or buy the name.

One name can use several IP addresses, and several websites can share infrastructure. Think of DNS as a lookup system, not a permanent one-name-one-computer rule.

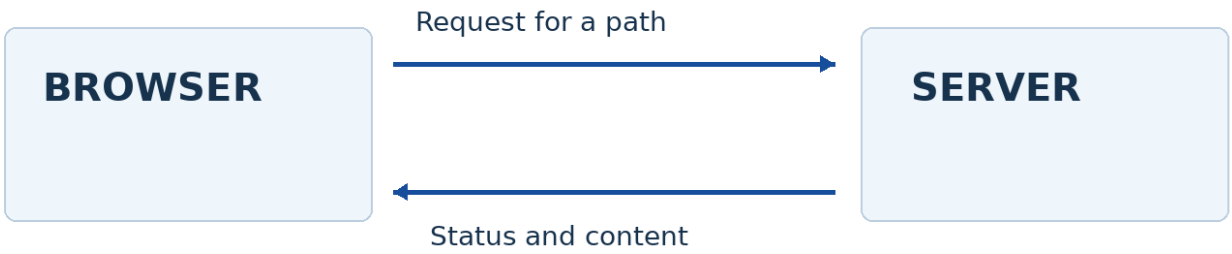
References: N2, N3.

Internet lesson 4 Follow a webpage from request to screen

Use: paper cards labelled Browser, DNS and Server.

A server is a computer or program that responds to requests. Your browser is the client asking for the page.

A request goes out and a response comes back



Teaching illustration • Simplified view

Act it out

1. Place three cards on a table: Browser, DNS and Server. Begin at Browser and ask DNS for the host’s address. A remembered result may already be available.

2. DNS provides address information. The browser connects to the destination.

3. The browser sends an HTTP request for a path, such as /.

4. The server sends an HTTP response: status information and usually some content.

5. The browser reads the HTML. It may request CSS, JavaScript, pictures and fonts separately. Then it builds the page you see.

HTTP is a set of rules for web requests and responses. HTTPS protects the connection using encryption and certificate checks. It does not prove that a seller is honest or that a webpage's claims are true.

A response code is a short result label. 200 usually means success. 404 means the requested resource was not found. 500 indicates a server-side failure. A redirect tells the browser to try another address.

Your turn: explain why a heading might appear before a large photograph finishes loading. The browser often needs more than one response to finish a page.

References: N2, N4.

Internet lesson 5 Watch real requests in the browser

Use: Edge or Chrome. The following steps open a panel built into the browser. You can close it without changing your page.

You do not need to understand the whole panel. Today we will find the page request and its response.

1. Open https://example.com.
2. Press F12, or Ctrl + Shift + I, to open developer tools. Some laptops also need Fn with F12.
3. Click Network. If it is hidden, use the extra-tab menu marked with arrows.
4. Press Ctrl + R to reload the page. Rows should appear in the panel.
5. Click the row for the document. Look under Headers for Request URL, Request Method and Status Code.
6. Read the actual method and status you see. Open Response or Preview to inspect the returned content.

Read your own request in the Network panel



Teaching illustration • Simplified view

Do not worry if a different website has many rows. One page can request many files. Some results may come from a cache, which is a saved copy used to reduce repeated downloads.

Your turn: write down the requested address and status. A 200 status usually means the request succeeded. Close the developer tools using its X when you finish.

If the panel is empty, open Network before refreshing. If a website fails, the panel may show a network error rather than an HTTP status because no response arrived.

Reference: N5. The illustration shows an example result; read your own screen.

Internet lesson 6 Data travels in pieces

Use: six small pieces of paper and a pencil.

Information travels across networks in small units called packets. Routers forward packets towards their destination. The receiving systems handle assembling the delivered data into something useful.

Send a paper message

1. Write WE, ARE, LEARNING, TO, BUILD, WEBSITES on six cards.
2. Number them 1–6 and write the same destination name on each card.
3. Let classmates pass them along two different routes across the room.
4. At the destination, arrange the cards in order. Notice whether one arrives late or is missing.

Packets are small units travelling across a network



Numbered cards model a message split into pieces.

Teaching illustration • Simplified view

This is a simplified model. Real networking adds addressing, checks and transport rules. For example, TCP can detect missing data and arrange retransmission. Different web connections may use different transport protocols.

Bandwidth describes how much data a connection can carry in a given time. Latency is the delay before information gets where it needs to go. A connection can have good capacity but still have noticeable delay.

A large image contains more data than a short sentence. That is one reason a page full of huge photos can feel slow and use more of a mobile data allowance.

Your turn: which should usually need more download data, one short paragraph or a high-resolution video? Explain your answer without using a speed-test number.

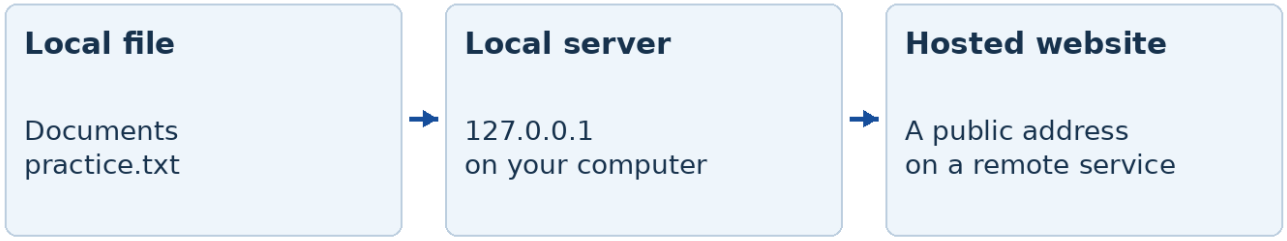
References: N1, N2.

Internet lesson 7 Online local upload and download

Use: your browser, File Explorer and practice.txt from the earlier folder lesson.

Download means receiving a copy from a remote service. Upload means sending a copy to a remote service. Saving a file in Documents is not automatically publishing it on the web.

Local files and hosted websites are different places



Teaching illustration • Simplified view

Compare two places

1. Open practice.txt in Notepad. It is stored on your computer.

2. Open example.com in your browser. You are visiting a remote website.

3. On your own practice computer, briefly disconnect your own Wi-Fi connection. Do not disconnect a shared classroom router.

4. Edit and save practice.txt. It should still work.

5. Try reloading the website. It may fail, or show stored content. A cached page does not prove that the internet is connected.

6. Reconnect Wi-Fi using the same network and the approved password arrangements.

Later, you will open a page using file:/// and run a server using http://127.0.0.1. Both can refer to work on your computer. A public https:// address usually reaches a hosted service.

If every site fails, check the connection. If just one path returns 404, check that address. If words load but a picture does not, check that picture's request.

Your turn: explain why sending someone C:\Users\Ada\Documents\index.html will not give them your website.

Reference: N2.

Internet lesson 8 Meet the five subjects we will practise

Use: this picture and a discussion. You will write each kind of code yourself.



Teaching illustration • Simplified view

Name	Its main job in our project
HTML	Describe the content and structure of the page
CSS	Control colours, spacing, type and layout
JavaScript	Respond to actions in the visitor's browser
PHP	Run on the server, check submitted information and prepare a response

HTML is a markup language. CSS is a style-sheet language. JavaScript and PHP are programming languages. They work together, but they do not do the same job.

Imagine a cake order page. HTML provides the labelled quantity field. CSS makes it readable. JavaScript can show a quick estimate as the visitor types. PHP checks the submitted quantity and calculates again on the server.

The browser can display HTML, apply CSS and run JavaScript. It does not execute PHP source code. A PHP server runs that code and sends the result back.

Your turn: name the tool you would use to change a blue heading, add a paragraph, respond to a button click and check a submitted form on the server.

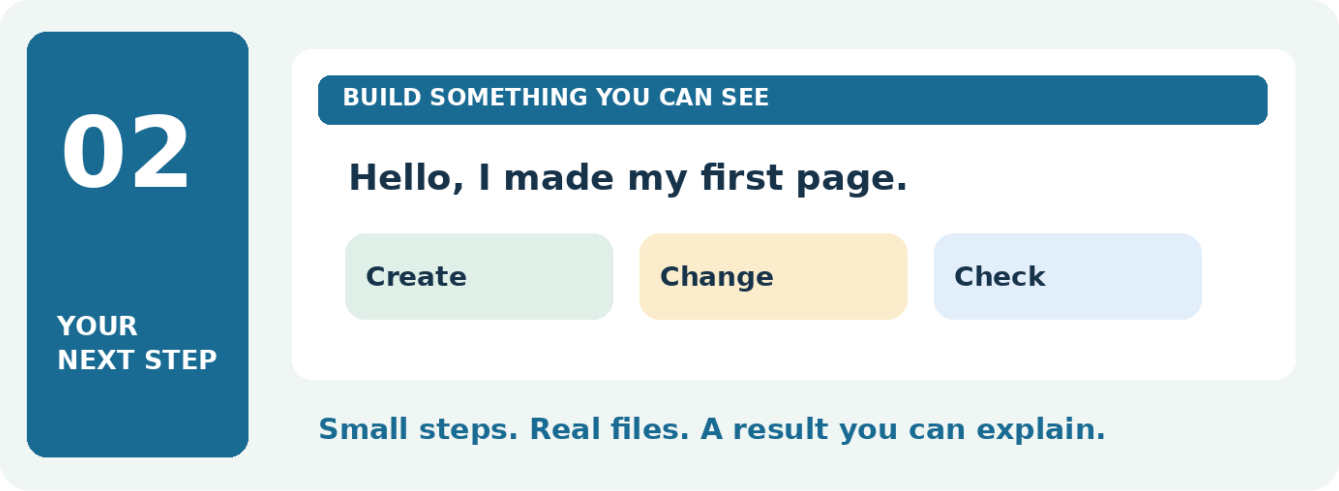
Answers after you try: CSS, HTML, JavaScript, PHP.

We will practise these foundations before moving into AI-assisted building and frameworks. You are not expected to remember all the syntax today.

References: N2, N6, N7, N8.

Part 02 Your first webpage

Start with Notepad, then make VS Code your everyday work space.



Your finish line

Save index.html, open it in a browser and change the visible words.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

Setup lesson 1 Make your first webpage in Notepad

Use: Notepad and your Website Class folder.

A webpage is a document that a browser can show. Today you will make one. Do not worry about remembering the symbols yet.

Type this carefully

Open a new Notepad document with File → New. In versions with tabs, New tab is also suitable. Type the whole example below.

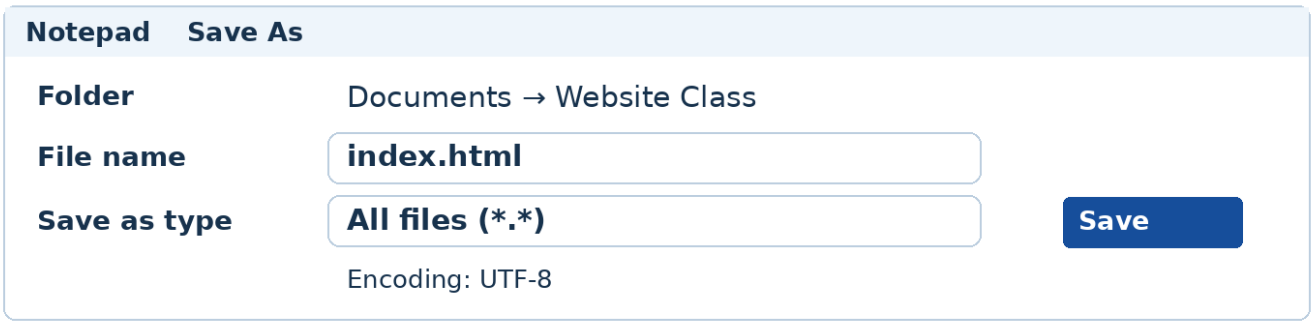
```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>My first website</title>
</head>
<body>
  <h1>Hello, I am Ada</h1>
  <p>This is my first webpage.</p>
</body>
</html>
```

On many English keyboards, hold Shift and press the comma key for <. Hold Shift and press the full-stop key for >. If these keys differ, press Windows + Ctrl + O to open the on-screen keyboard and look for the symbols.

Save it as a webpage

1. Click File → Save as. Open Documents → Website Class.
2. In File name, type index.html.
3. Change Save as type to All files. Choose UTF-8 if Encoding is shown.
4. Click Save. Keep this folder open for the next lesson.

Save a webpage with the right name and type



Teaching illustration • Simplified view

Important: index.html is different from index.html.txt. The ending .html is what we need.

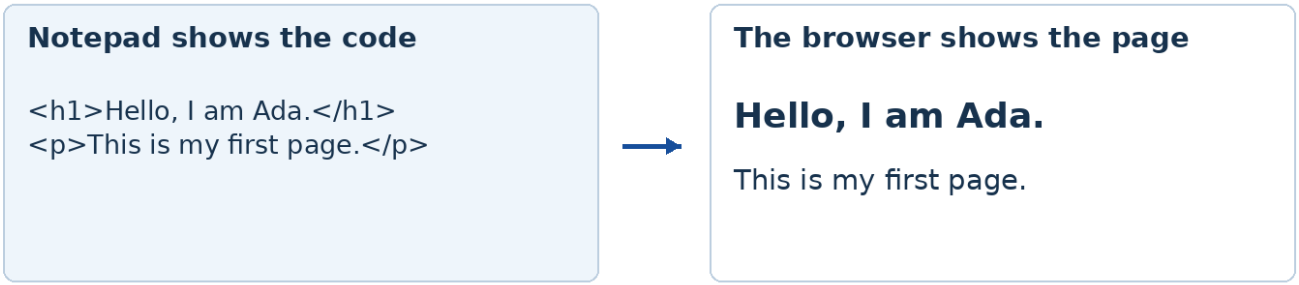
Setup lesson 2 Open your page and change it

Use: File Explorer, Notepad and Edge or Chrome.

See your page

1. In Website Class, right-click index.html.
2. Choose Open with, then Microsoft Edge or Google Chrome.
3. You should see a large greeting and a smaller sentence.

The same file has two useful views



Teaching illustration • Simplified view

Change the greeting

1. Return to Notepad. If the file is closed, use File → Open. Select All files in the file-type choice if index.html is hidden.
2. Change Ada to your own name. Leave <h1> and </h1> in place.
3. Press Ctrl + S to save.
4. Return to the browser. Press Ctrl + R to refresh the page.

If you still see Ada, check that you saved. Then check that the browser opened the index.html inside Website Class.

If you see the code itself, check the filename. In Windows 11, File Explorer → View → Show → File name extensions reveals file endings. In Windows 10, use the View tab → File name extensions. If the file ends in .html.txt, right-click it, choose Rename, remove only the final .txt, press Enter and confirm the extension change.

You have succeeded when: your own name appears in the browser.

Source: S1.

Setup lesson 3 Understand the few tags you used

Use: your index.html in Notepad.

HTML gives the browser instructions about the page. A tag is an instruction inside angle brackets, such as . For now, learn these four parts.

A tag gives a piece of text a job



Teaching illustration • Simplified view

Make two changes

1. Find <title>My first website</title>.

2. Replace only My first website with Ada School Project. Save and refresh. Look at the browser tab, not the page heading.

3. Find the sentence inside and .

4. Change it to I can create and edit a webpage. Save and refresh again.

Add another paragraph

Click just before </body>. Press Enter to make a new line. Type:

```
<p>I want to build a website for a small business.</p>
```

Save and refresh. You should now see two paragraphs beneath the greeting.

Say it back: “The title names the browser tab. The h1 is the main heading on the page. A p is a paragraph.”

Your turn: add one more paragraph about a skill you want to learn. Change the words, not the surrounding tags.

Setup lesson 4 Make a safe copy before changing more

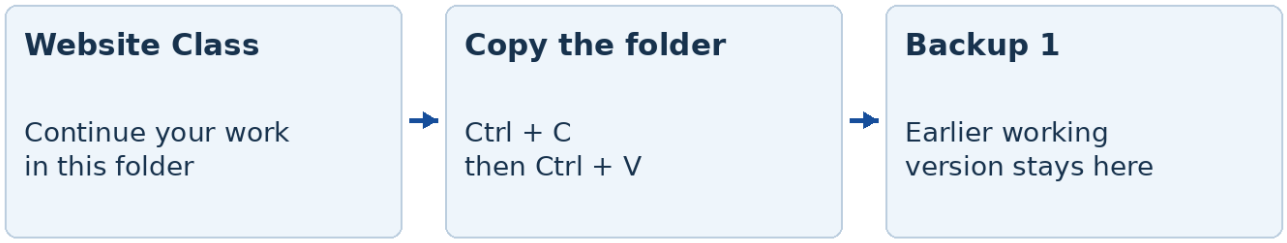
Use: File Explorer.

A backup is a spare copy. It helps you return to a working page if an experiment goes wrong.

Copy the folder

1. Save your page and close Notepad.
2. Open Documents in File Explorer.
3. Click Website Class once. Press Ctrl + C, then Ctrl + V.
4. Windows creates a copy. Right-click that copied folder and choose Rename.
5. Call it Website Class Backup 1. Press Enter.

Work in the original folder and keep a spare copy



Teaching illustration • Simplified view

Continue working in Website Class. Leave the backup alone.

A small confidence test

Open the original index.html in Notepad. Change the heading to I made this page. Save and open it in the browser.

Now open index.html inside Website Class Backup 1. It should still contain the earlier version. You have two separate copies.

If you lose your way: close the extra browser tabs and return to the original folder. Having five copies open at once makes it easy to edit the wrong file.

Ready to move on: open the correct file, change a sentence, save it and refresh without help. If this still feels difficult, repeat Computer lesson 3 and Setup lessons 1–2 before installing another tool.

Setup lesson 5 Install Visual Studio Code

Use: Edge or Chrome. You need permission from the owner before installing on a shared computer.

VS Code is an editor, like Notepad, with extra help for writing code. It is not the same product as Visual Studio.

Download and install

1. Open your browser. Click the address bar at the top.

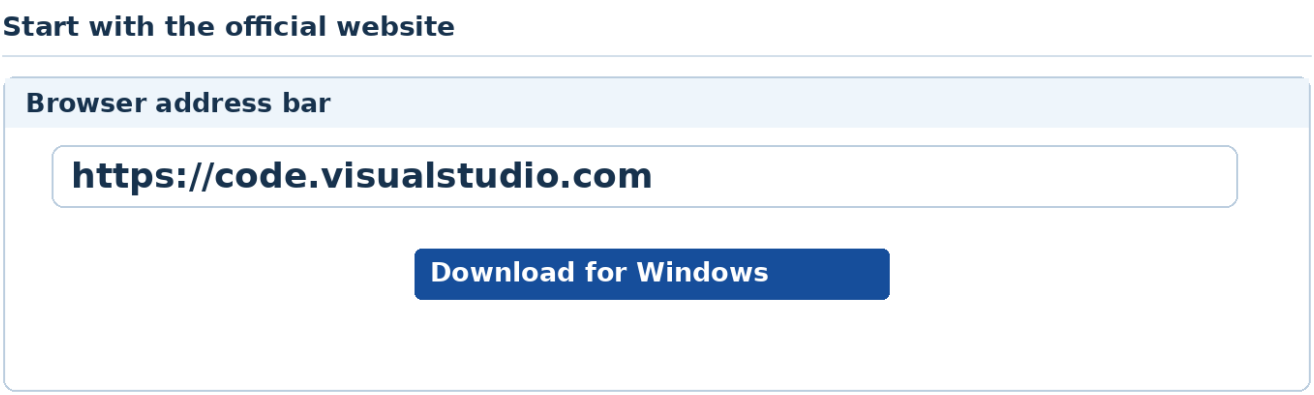
2. Type https://code.visualstudio.com and press Enter.

3. Choose Download for Windows. Use the official website, not an advertisement from a search result.

4. Open the downloaded installer from the browser's Downloads list or your Downloads folder.

5. Read the setup screens. Accept the licence only if you agree, then follow Next and Install. Keep the default location unless you have a reason to change it. Choose the User Installer when it is offered for your personal Windows account.

6. Choose Launch Visual Studio Code when installation finishes, or open it from Start.



Teaching illustration • Simplified view

If setup says the computer is unsupported, open Windows Settings → System → About and compare your system with the linked requirements. Continue the Notepad lessons on that computer; use a supported system for later tools. Do not use unofficial old installers.

You should see: the VS Code welcome screen.

There is no need to sign in or install an AI extension just to edit HTML files.

Sources: S2, S3.

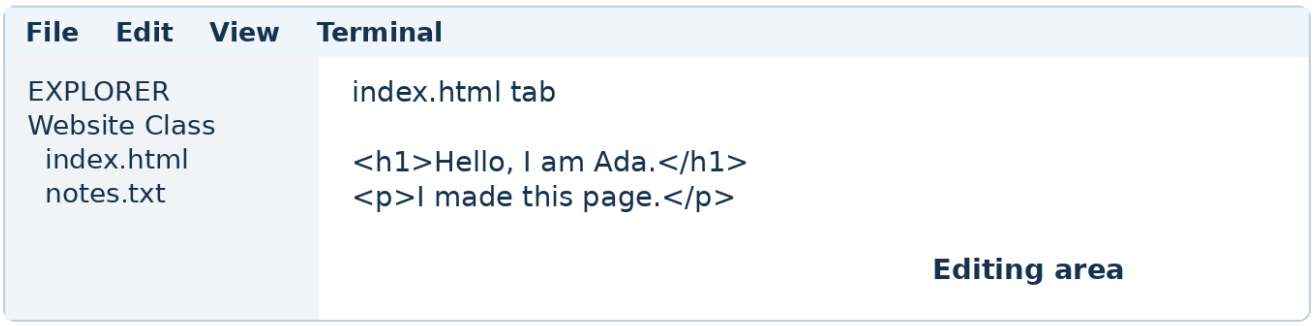
Setup lesson 6 Open your folder in VS Code

Use: VS Code and the original Website Class folder.

Find your page

1. In VS Code, click File → Open Folder.
2. Select Documents → Website Class. Click Select Folder.
3. If asked about trust, trust it only because this is your own classroom folder.
4. On the left, click index.html. Its code opens in the large middle area.
5. Change one sentence. Press Ctrl + S.
6. Open the page in your browser and refresh.

Three parts of the VS Code window



Teaching illustration • Simplified view

The left file list is called Explorer. A file's tab appears above the writing area. Clicking a tab switches between open files.

Create a new file

At the top of Explorer, click the New File icon. If you cannot find it, right-click the folder name and choose New File. Type notes.txt and press Enter.

Type: This is my VS Code practice folder. Save.

You should see: index.html and notes.txt in the left file list.

Your turn: close notes.txt using the X on its tab. Open it again by clicking its name in Explorer.

Source: S4.

Setup lesson 7 Add Live Server to VS Code

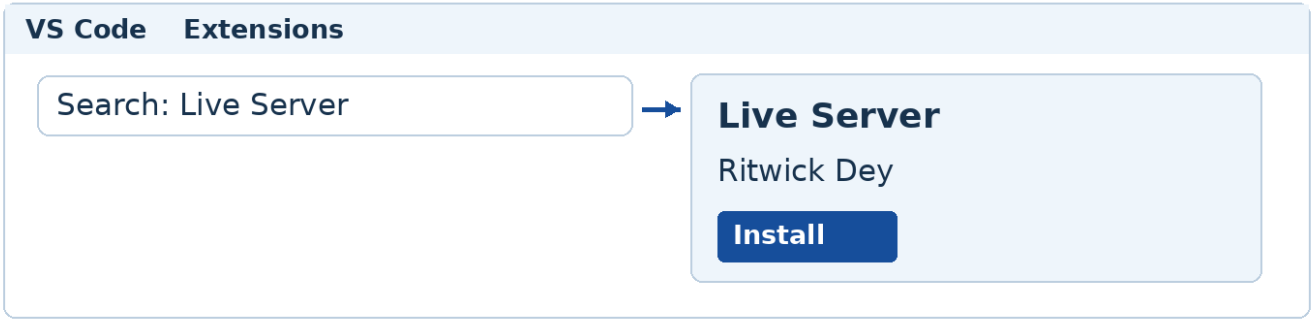
Use: VS Code with Website Class open. Internet is needed for installation.

An extension adds a tool to VS Code. Live Server opens your HTML page and refreshes it when you save. It is separate from the AI tools we will add later.

Install the correct extension

1. Click the Extensions icon on the left. It looks like a group of small squares. Or press Ctrl + Shift + X.
2. In the search box, type Live Server.
3. Select Live Server by Ritwick Dey. Check the publisher before installing.
4. Click Install. Return to Explorer using the files icon.
5. Right-click index.html and choose Open with Live Server.

Install the extension by the correct publisher



Teaching illustration • Simplified view

Your browser should open the page at a local address. It may begin with 127.0.0.1 and include a number such as 5500. Use the address that actually opens on your computer.

Change one sentence in VS Code and press Ctrl + S. Watch the browser update.

If the menu option is missing: check that installation finished and you opened a .html file. Close and reopen VS Code if needed. You can still use the manual browser method from Setup lesson 2.

Microsoft Live Preview is another useful extension. We use Live Server here so the class follows one set of instructions. Neither tool by itself runs Laravel or Django.

Source: S5.

Get the practice files ready

Use File Explorer and VS Code. The ZIP is a companion to this handout, not a replacement for typing and understanding the examples.

1. Click the Downloads link at the top of this page and choose the practice ZIP.

2. In File Explorer, right-click the ZIP → Extract All. Choose a folder inside Documents → Website Class.

3. Open the extracted Website_Design_Practice_Files folder. You should see html-1 through html-5, css-1 through css-5, javascript-1 through javascript-5, php-1 through php-5, sql and website-1 through website-5.

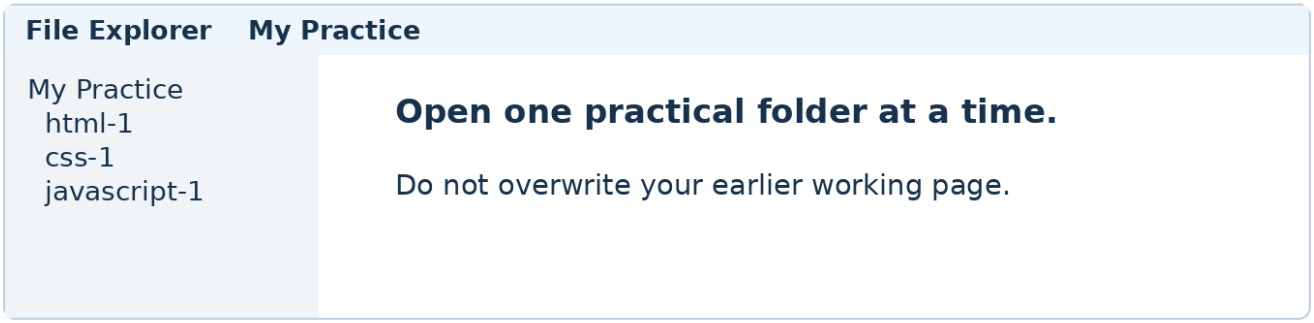
4. Make a separate folder called My Practice. Create your own exercise folders inside it using those same names.

5. In VS Code, use File → Open Folder to open the folder for the current practical.

For learning, type the displayed code into your own files. Use the supplied answer folder when comparing or recovering a mistake. Every supplied HTML, CSS, JavaScript and PHP page is complete.

The HTML snippets labelled “Inside body” replace only the content between <body> and </body> in the starter below. Keep its surrounding document structure. We show that repeated shell once so you can focus on each new lesson.

Keep every exercise in its own folder

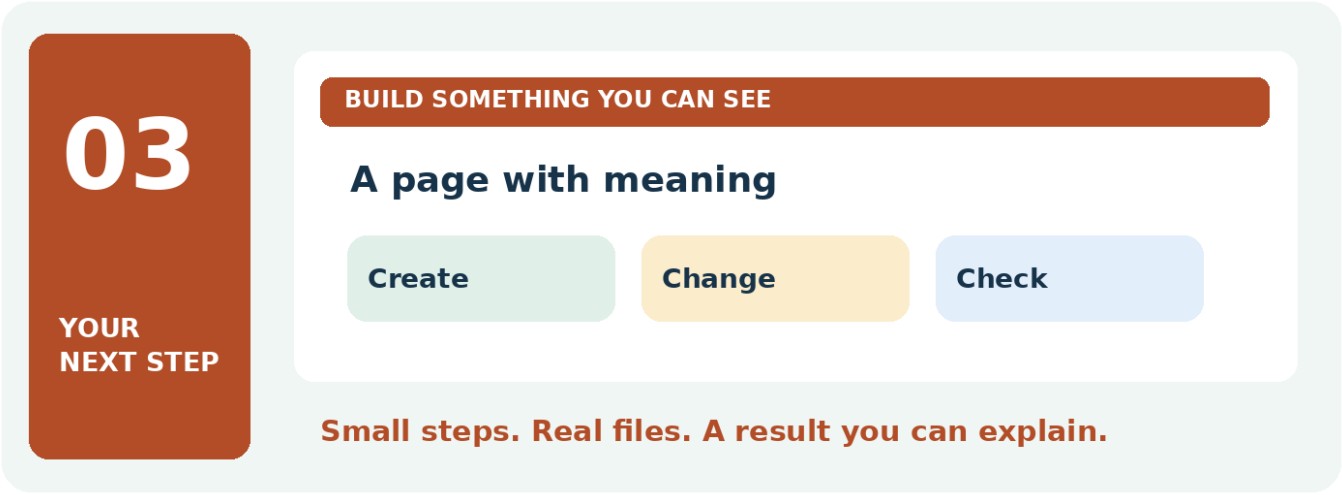


Teaching illustration • Simplified view

The books.svg image for HTML 3 is supplied. Copy it into your HTML 3 folder. It is an original classroom illustration, so no image search is required.

Part 03 HTML gives content a structure

Build five useful pages with headings, lists, images, links and forms.



Your finish line

Finish a profile, menu, event page, registration form and club homepage.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

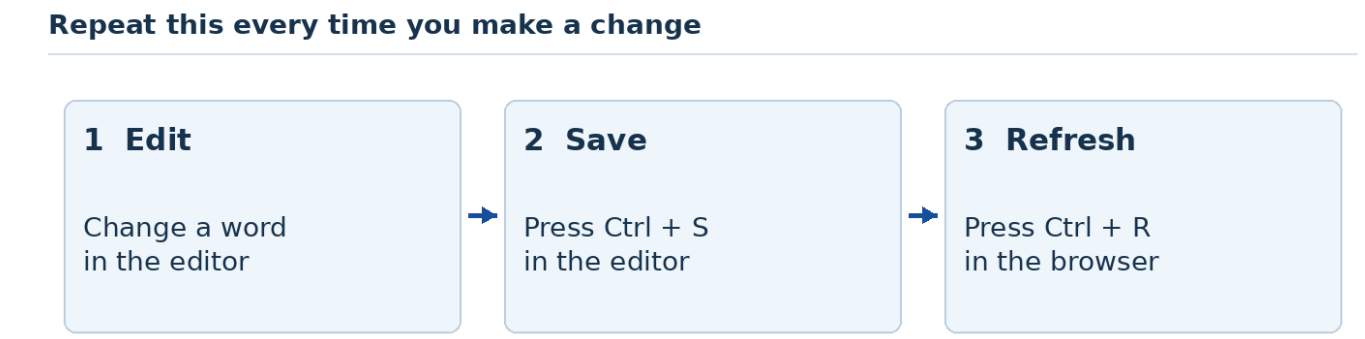
HTML help Save and reopen the right file

1. In File Explorer, turn on file extensions: View → Show → File name extensions on Windows 11, or View → File name extensions on Windows 10.

2. Confirm your file is index.html, not index.html.txt. In Notepad, Save As → All files → index.html → UTF-8.

3. To edit: right-click index.html → Open with → Notepad. To view: right-click → Open with → Edge or Chrome.

4. After every edit, Ctrl + S in Notepad and Ctrl + R in the browser.



Teaching illustration • Simplified view

What you see	Try this
Code appears as text	Check the .html ending and open it in a browser.
Old words remain	Save the editor file, then refresh the browser. Check the folder path.
Picture is missing	Match src to the exact filename, including its extension.
Link does nothing useful	Check href and confirm the destination file or id exists.

READY TO CONTINUE Create a heading with your name. Close the page. Find the file again and reopen it. You have made a webpage you can keep.

HTML foundation Build the document shell

HTML describes content and its meaning. It is a markup language. Tags are not decoration commands.

Create index.html in your first practical folder. Use Notepad with Save as type set to All files, or VS Code. Start every HTML practical with this complete shell:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>My practice page</title>
</head>
<body>
  <!-- Put the practical's body content here. -->
</body>
</html>
```

Save and open it in the browser. It will be blank until you add body content. Change title to match each exercise.

Code	Meaning
doctype and html	Use modern HTML and en for the document language.
head and title	Hold document information and the browser-tab title.
meta charset	Use UTF-8 character encoding.
meta viewport	Use the device width for the page viewport.
body	Contains the visible page content.
<!-- -->	A code comment; it is visible in source, so never put secrets there.

HTML foundation Read tags and attributes

An element usually has an opening tag, content and a closing tag. Close inner elements before outer ones.

The tag names the element and the attribute adds detail

```
<a href="about.html">About</a>
```

a: link element **href: destination** **About: text**

Teaching illustration • Simplified view

In About, a is the tag name. href is an attribute. Its value gives the destination. About is the visible text.

Use h1 for the page's main topic, h2 for sections and h3 for a smaller topic inside a section. These levels describe structure. Use CSS later to change visual sizes.

Some elements, including img, input, meta and link, do not have closing tags. Do not add </img> or </input>.

Your five HTML levels

1. Text and headings: a personal introduction.
2. Lists and table structure: a restaurant menu.
3. Links, images and meaningful sections: an event page.
4. Form controls and labels: workshop registration.
5. Navigation and interactive native elements: a club page.

After each practical, save and refresh. Right-click the page → View page source to compare the markup. A browser may recover from invalid HTML, so you can also visit https://validator.w3.org/nu/, choose File upload, select your fictional practice file and click Check.

References: N6 and the W3C page-structure source S36.

HTML practical 1 A personal introduction page

Job: Make a simple profile page for a student or craft worker.

Use: VS Code and the folder html-1. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

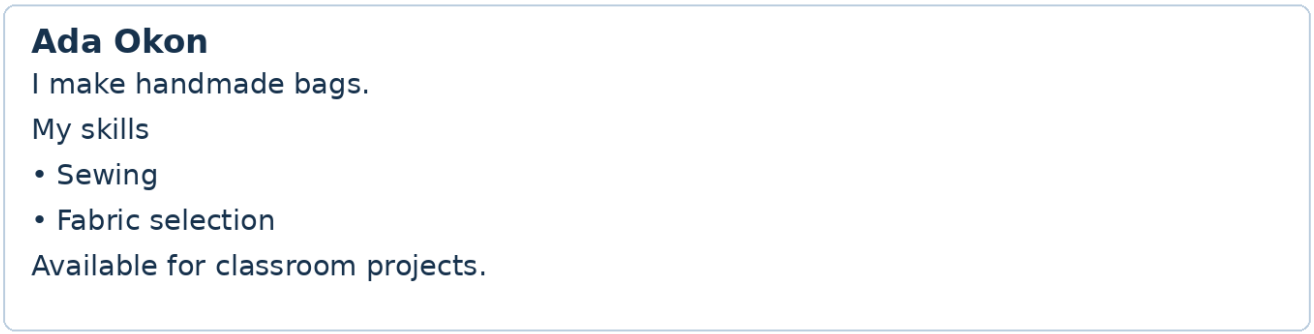
```
<main>
  <h1>Ada Okon</h1>
  <p>I make <strong>handmade bags</strong>.</p>
  <h2>My skills</h2>
  <ul>
    <li>Sewing</li>
    <li>Fabric selection</li>
  </ul>
  <p><em>Available for classroom projects.</em></p>
</main>
```

Now run the page or query and use the explanation and expected result on the next page.

HTML practical 1 Output and code explanation

What you should see

HTML 1 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
main	Contains the main content.
h1 and h2	Name the page and introduce a section.
p	Groups a paragraph of text.
strong and em	Mark importance and emphasis; usually bold and italic.
ul and li	Create an unordered list and its individual items.

TRY IT Replace the name and skills with your own. Add a third skill.

READY TO CONTINUE Name is visible. Two original skills appear as separate bullet points.

IF IT DOES NOT WORK Match opening and closing tags. Check filenames in src and href. Save, then refresh.

HTML practical 2 A restaurant menu page

Job: Present real-world menu information with categories and prices.

Use: VS Code and the folder html-2. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<header><h1>Market Kitchen</h1></header>
<main>
  <h2>Lunch menu</h2>
  <table>
    <caption>Sample prices in NGN</caption>
    <tr><th scope="col">Meal</th><th scope="col">Price</th></tr>
    <tr><td>Jollof rice</td><td>2500</td></tr>
    <tr><td>Vegetable soup</td><td>3000</td></tr>
  </table>
  <h2>How to order</h2>
  <ol><li>Choose a meal.</li><li>Ask the attendant.</li></ol>
</main>
<footer><p>Classroom menu only.</p></footer>
```

Now run the page or query and use the explanation and expected result on the next page.

HTML practical 2 Output and code explanation

What you should see

HTML 2 expected output in simplified form

Market Kitchen

Lunch menu

Meal	Price
Jollof rice	2500
Vegetable soup	3000

1 Choose a meal. 2 Ask the attendant.

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
header, main and footer	Separate the introduction, main content and closing note.
h1, h2 and p	Give the page and sections names, then normal text.
table and caption	Create and name a data table; not a page-layout tool.
tr, th and td	Make a row, heading cell and ordinary data cell.
scope="col"	Associates a heading with its column.
ol and li	Make ordered steps and each step.

TRY IT Add a third meal and its price. Keep each price in the same row as its meal.

READY TO CONTINUE Two meals and two prices appear. The ordering instructions are numbered.

IF IT DOES NOT WORK Match opening and closing tags. Check filenames in src and href. Save, then refresh.

HTML practical 3 An event information page

Job: Make an event page with a working internal link and meaningful image text.

Use: VS Code and the folder html-3. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

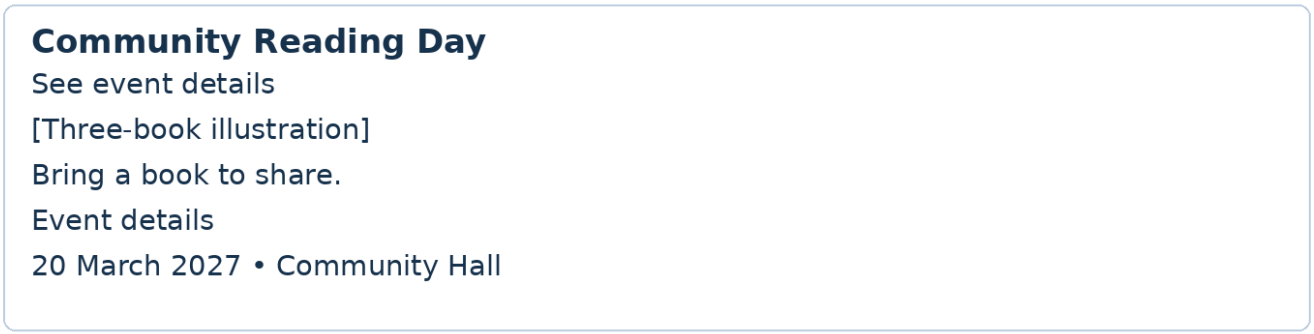
```
<header>
  <h1>Community Reading Day</h1>
  <a href="#details">See event details</a>
</header>
<main>
  <figure>
    <img src="books.svg" alt="Three books standing together"
         width="300" height="150">
    <figcaption>Bring a book to share.</figcaption>
  </figure>
  <section id="details">
    <h2>Event details</h2>
    <p><time datetime="2027-03-20">20 March 2027</time></p>
    <p>Practice venue: Community Hall.</p>
  </section>
</main>
```

Now run the page or query and use the explanation and expected result on the next page.

HTML practical 3 Output and code explanation

What you should see

HTML 3 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
header, main and section	Group introductory, main and related section content.
h1, h2 and p	Supply the visible heading hierarchy and paragraph.
a and href="#details"	Link to the element whose id is details; a short page may barely scroll.
figure and figcaption	Keep an illustration and its caption together.
img, src and alt	Load an image file and provide equivalent useful text.
width and height	Reserve the image dimensions; CSS can later make it responsive.
time and datetime	Show a human-readable date with a machine-readable value.
id	Identifies this section uniquely on the page.

TRY IT Change the event and date in both visible text and datetime. Check that they agree.

READY TO CONTINUE The image appears. The link changes the URL fragment to #details.

IF IT DOES NOT WORK Match opening and closing tags. Check filenames in src and href. Save, then refresh.

HTML practical 4 A workshop registration form

Job: Build correctly labelled fields and observe browser validation.

Use: VS Code and the folder html-4. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Register for a sewing workshop</h1>
  <form action="index.html" method="get">
    <label for="name">First name</label>
    <input id="name" name="name" required>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>
    <label for="level">Your level</label>
    <select id="level" name="level">
      <option value="new">Complete beginner</option>
      <option value="some">Some experience</option>
    </select>
    <label for="note">What would you like to learn?</label>
    <textarea id="note" name="note"></textarea>
    <button type="submit">Try registration</button>
  </form>
</main>
```

Now run the page or query and use the explanation and expected result on the next page.

HTML practical 4 Output and code explanation

What you should see

HTML 4 expected output in simplified form

Register for a sewing workshop
First name [Ada]
Email [ada@example.com]
Your level [Complete beginner]
[Try registration]
After submit: URL contains name=Ada

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
main and h1	Identify the main content and form purpose.
form, action and method	Submit to index.html using GET; this demo only reloads with query values.
label and for	Connect a visible label to an input id.
input, id and name	Create a field, identify it and name the submitted value.
type="email" and required	Apply browser email-format and missing-value checks.
select and option	Offer a menu; value is the submitted choice.
textarea	Provides a multiline writing field.
button type="submit"	Submits the enclosing form.

TRY IT Add a labelled phone field using type="tel". Do not collect real personal information.

READY TO CONTINUE Empty required fields are blocked. A valid fictional entry appears in the URL, but is not stored.

IF IT DOES NOT WORK Match opening and closing tags. Check filenames in src and href. Save, then refresh.

HTML practical 5 A school club home page

Job: Build navigation, a self-contained article and an expandable answer.

Use: VS Code and the folder html-5. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<header>
  <h1>Bright Readers Club</h1>
  <nav aria-label="Main navigation">
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
  </nav>
</header>
<main>
  <article>
    <h2>This week</h2>
    <p>Bring one short story for discussion.</p>
  </article>
  <details>
    <summary>Do I need to buy a book?</summary>
    <p>No. Bring a book you already have.</p>
  </details>
</main>
```

HTML practical 5 A school club home page code continued 2

Continue the same practical. Keep the earlier lines; add the following lines in the named file.

about html

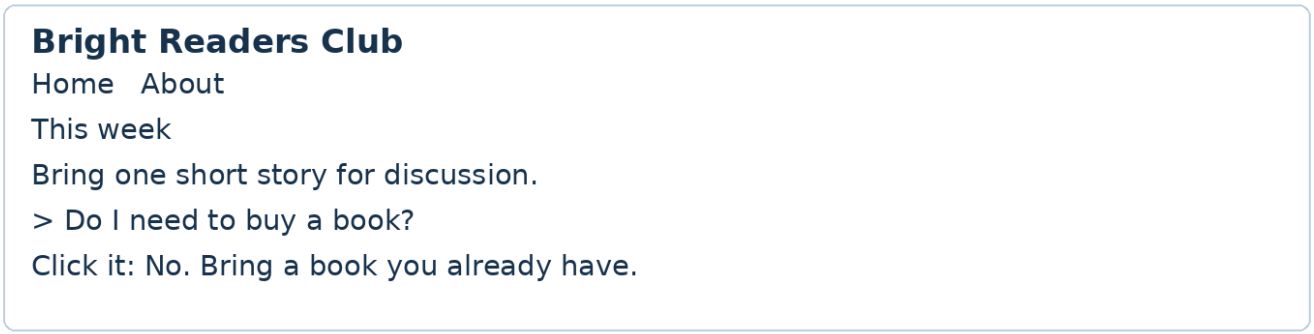
```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>About our club</title>
</head>
<body>
<main><h1>About our club</h1>
<p>We practise reading together.</p>
<a href="index.html">Home</a></main>
</body>
</html>
```

Now run the page or query and use the explanation and expected result on the next page.

HTML practical 5 Output and code explanation

What you should see

HTML 5 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
header, main, h1, h2 and p	Organise the page and its readable content.
nav and aria-label	Identify and name a navigation region.
a and relative href values	Link to files in the same folder.
article	Groups a self-contained club update.
details and summary	Provide an expandable answer with a clickable summary.

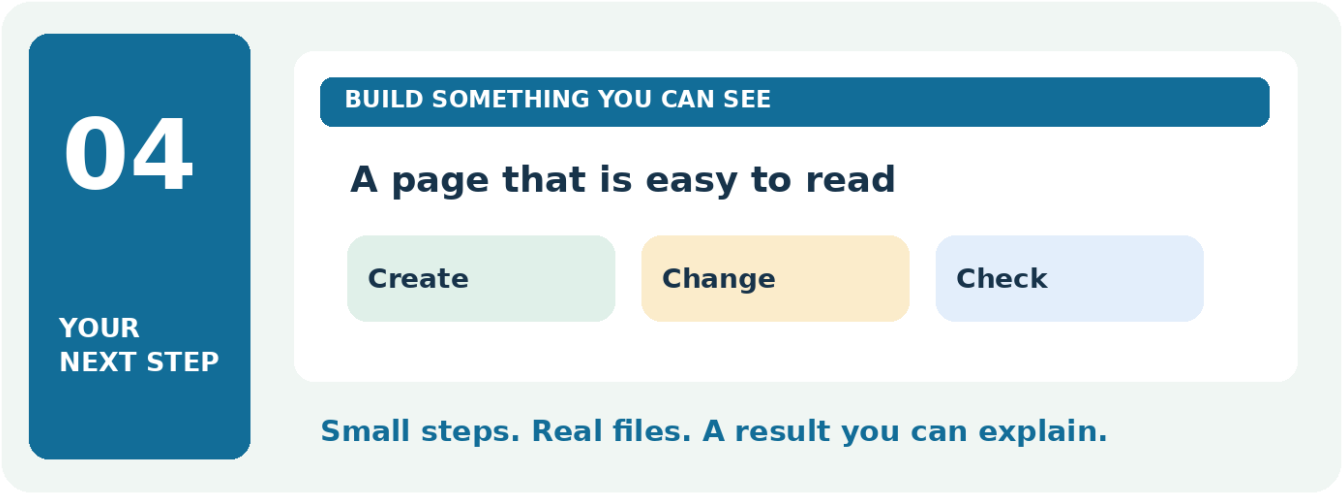
TRY IT Add a Contact link and contact.html. Test the return link to Home.

READY TO CONTINUE About opens and Home returns. The question expands without JavaScript.

IF IT DOES NOT WORK Match opening and closing tags. Check filenames in src and href. Save, then refresh.

Part 04 CSS gives the page its appearance

Use colour, spacing and layout to make the same content easier to read.



Your finish line

Build a business introduction, product card, navigation, course grid and enquiry form.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

CSS help See which rule changes the page

1. Open the practical’s folder in VS Code. index.html and style.css should appear together in Explorer.

2. Confirm the head contains <link rel="stylesheet" href="style.css">. This course uses style.css in the CSS practicals and styles.css in the later AI starter. Use the name shown for that project.

3. Change the body colour to #126b59. Save and refresh. If nothing changes, check the filename before changing more code.

4. Right-click a visible heading in the browser → Inspect. The Styles panel shows rules that apply to it. Untick one rule, then tick it again.

Read a CSS rule as a simple instruction

```
.card { padding: 24px; }
```

Selector: .card **Property: padding** **Value: 24px**

Teaching illustration • Simplified view

IF IT DOES NOT WORK Edits made in Inspect are temporary. Make the same change in your saved CSS file to keep it. A rule with a line through it is being overridden or cannot apply.

Your small experiment: try padding: 8px and then padding: 32px. Watch the space inside the box change. Restore the value from the practical before continuing.

CSS foundation Select and style the page

CSS controls appearance. It does not replace HTML's meaning. Use a separate style.css file connected by a link element in head.

Read a CSS rule as a simple instruction

```
.card { padding: 24px; }
```

Selector: .card **Property: padding** **Value: 24px**

Teaching illustration • Simplified view

A selector chooses the elements. A declaration gives a property and value. Braces surround the declarations. Colons separate properties from values; semicolons separate declarations.

h1 chooses heading elements. .card chooses elements with class="card". #notice chooses the element whose id is notice. Reuse a class; keep an id unique in a page.

A later rule can win when competing rules have equal priority and specificity. A class selector is more specific than an element selector in an ordinary style sheet. Inspect an element in the browser to see which rule won. Do not add !important whenever you are confused.

Your five CSS levels

1. Typography and colour.

2. Box model, borders and spacing.

3. Flexbox, navigation and interaction states.

4. Grid and responsive breakpoints.

5. Form styling and reusable custom properties.

Only CSS is the new subject here. Each practical supplies a small HTML fixture to give the styles something to act on. Source: N11.

CSS practical 1 A clear business introduction

Job: Style a service heading and short description.

Use: VS Code and the folder css-1. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

In head, add <link rel="stylesheet" href="style.css">. Create style.css beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main class="intro">
  <h1>Clean Home Services</h1>
  <p>Careful cleaning for small offices.</p>
  <a href="mailto:hello@example.com">Ask a question</a>
</main>
```

style css

```
body {
  font-family: Arial, sans-serif;
  color: #17324d;
  background: #f4f8fb;
  line-height: 1.6;
  padding: 24px;
}
h1 { font-size: 2.5rem; line-height: 1.15; }
.intro { max-width: 700px; margin: auto; }
a { color: #164e9b; }
```

Now run the page or query and use the explanation and expected result on the next page.

CSS practical 1 Output and code explanation

What you should see

CSS 1 expected output in simplified form

Clean Home Services
Careful cleaning for small offices.
Ask a question
Pale background • dark readable text
Large heading • centred content

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
body, h1 and a selectors	Choose all matching elements.
.intro	Chooses the element with class="intro".
font-family and font-size	Choose typeface and size; rem follows the root text size.
color and background	Set foreground and background colours.
line-height	Sets comfortable line spacing.
padding, max-width and margin	Add inner space, limit line length and centre the block.
HTML main, h1, p and a	Contain the introduction, heading, sentence and mail link; mailto opens a configured email app.

TRY IT Change the brand colour in a and compare. Keep text contrast readable.

READY TO CONTINUE The heading is larger than the paragraph. Content remains readable when the browser narrows.

IF IT DOES NOT WORK Check the linked CSS filename first. Then check the selector spelling, braces and semicolons.

CSS practical 2 A product card with breathing room

Job: Use the box model to make a merchandise card.

Use: VS Code and the folder css-2. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

In head, add <link rel="stylesheet" href="style.css">. Create style.css beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<article class="card">
  <h1>Canvas tote bag</h1>
  <p>Reusable bag for everyday essentials.</p>
  <p class="price">Sample price: NGN 6000</p>
</article>
```

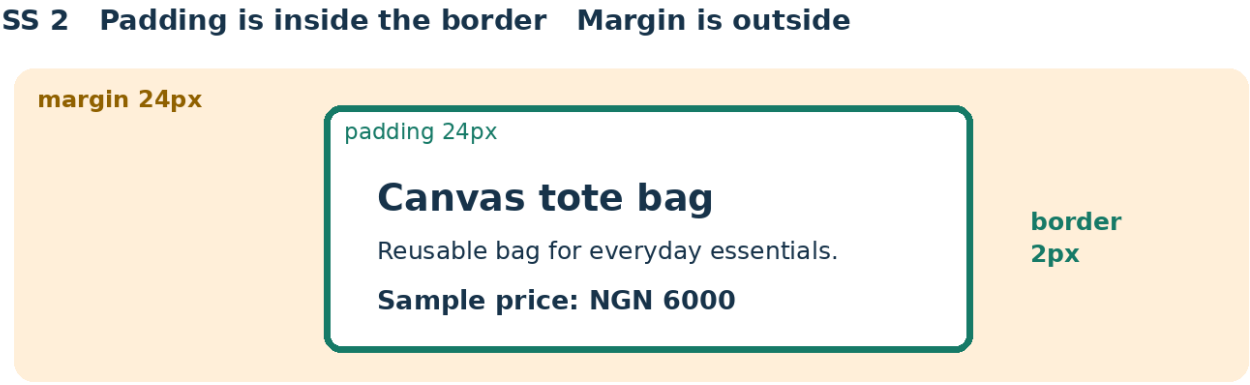
style css

```
* { box-sizing: border-box; }
body { font-family: Arial; padding: 24px; }
.card {
  max-width: 360px;
  margin: 24px auto;
  padding: 24px;
  border: 2px solid #164e9b;
  border-radius: 12px;
  background: #eef5fb;
}
.price { font-weight: bold; }
```

Now run the page or query and use the explanation and expected result on the next page.

CSS practical 2 Output and code explanation

What you should see



Teaching illustration • Layout comparison

What each part does

Tag or code	Usage in this practical
* and box-sizing	Apply border-box sizing to all elements so declared width includes padding and border.
body and .card	Choose the page body and the card class.
padding and margin	Space inside the card versus space outside it.
border and border-radius	Draw an edge and round its corners.
max-width and background	Limit card width and fill its background.
.price and font-weight	Make the price paragraph bold.
HTML article, h1, p and class	Group a standalone product, name it, describe it and attach style hooks.

TRY IT Change padding from 24px to 4px, observe, then restore it.

READY TO CONTINUE Text never touches the border. The price is bold.

IF IT DOES NOT WORK Check the linked CSS filename first. Then check the selector spelling, braces and semicolons.

CSS practical 3 A flexible navigation bar

Job: Arrange a studio name and navigation without fixed screen widths.

Use: VS Code and the folder css-3. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

In head, add <link rel="stylesheet" href="style.css">. Create style.css beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<header class="bar">
  <strong>Thread Studio</strong>
  <nav aria-label="Main">
    <a href="#services">Services</a>
    <a href="#contact">Contact</a>
  </nav>
</header>
<main>
  <h1 id="services">Tailoring services</h1>
  <p id="contact">Practice contact: hello@example.com</p>
</main>
```

style css

```
body { font-family: Arial; padding: 24px; }
.bar, nav { display: flex; gap: 16px; flex-wrap: wrap; }
.bar { justify-content: space-between; align-items: center; }
a { color: #164e9b; padding: 8px; }
a:hover { background: #eef5fb; }
a:focus-visible { outline: 3px solid #164e9b; }
```

Now run the page or query and use the explanation and expected result on the next page.

CSS practical 3 Output and code explanation

What you should see

CSS 3 expected output in simplified form

Thread Studio Services Contact
Tailoring services
Practice contact: hello@example.com
Narrow screen: links wrap
Tab: a visible outline appears

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
.bar, nav and comma	Apply the same rule to two selectors.
display: flex	Lay out direct children along an axis.
gap and flex-wrap	Add space and allow wrapping.
justify-content and align-items	Distribute items along the main axis and align them across it.
:hover and :focus-visible	Style pointer hover and visible keyboard focus.
outline, padding, color and background	Provide clear focus, space and colour cues.
HTML header, strong, nav, main, h1, p and a	Build a named studio header, navigation and two link targets; id matches each fragment.

TRY IT Add a third navigation link and matching section id.

READY TO CONTINUE Links wrap rather than overflow. Both links work using Tab then Enter.

IF IT DOES NOT WORK Check the linked CSS filename first. Then check the selector spelling, braces and semicolons.

CSS practical 4 A responsive course card grid

Job: Display training options in one column on a phone and three on a wide screen.

Use: VS Code and the folder css-4. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

In head, add <link rel="stylesheet" href="style.css">. Create style.css beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Weekend classes</h1>
  <div class="courses">
    <article><h2>Sewing</h2><p>Start with a simple bag.</p></article>
    <article><h2>Baking</h2><p>Make your first cupcakes.</p></article>
    <article><h2>Web design</h2><p>Build a first page.</p></article>
  </div>
</main>
```

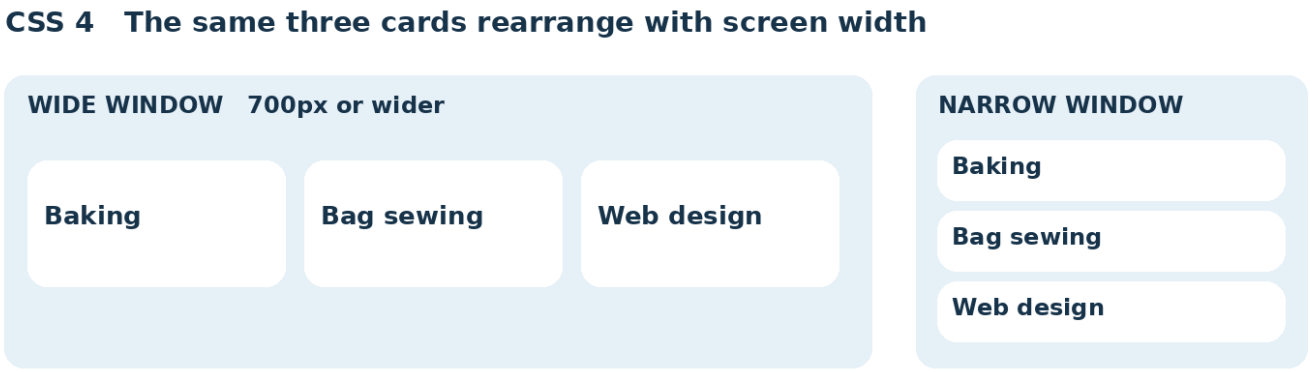
style css

```
body { font-family: Arial; padding: 24px; }
.courses { display: grid; gap: 16px; }
article { padding: 20px; background: #eef5fb; }
@media (min-width: 700px) {
  .courses { grid-template-columns: repeat(3, 1fr); }
}
```

Now run the page or query and use the explanation and expected result on the next page.

CSS practical 4 Output and code explanation

What you should see



Teaching illustration • Layout comparison

What each part does

Tag or code	Usage in this practical
display: grid and gap	Create a grid with spacing between cells.
@media and min-width	Apply the enclosed rule when the viewport is at least 700 CSS pixels wide.
grid-template-columns	Define the columns.
repeat(3, 1fr)	Create three equal fractional columns.
body and article rules	Set type and breathing room; give each card a fill.
HTML main, h1, div, article, h2 and p	Name the page, group the layout and describe each course; .courses selects the div.

TRY IT Add a fourth course. Observe the second row at wide sizes.

READY TO CONTINUE Below 700px: one column. At 700px or wider: three columns.

IF IT DOES NOT WORK Check the linked CSS filename first. Then check the selector spelling, braces and semicolons.

CSS practical 5 A readable enquiry form

Job: Style labels and controls without changing their behaviour.

Use: VS Code and the folder css-5. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

In head, add <link rel="stylesheet" href="style.css">. Create style.css beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Ask about a class</h1>
  <form action="index.html" method="get">
    <label for="name">First name</label>
    <input id="name" name="name" required>
    <label for="message">Your question</label>
    <textarea id="message" name="message" required></textarea>
    <button>Try the form</button>
  </form>
</main>
```

style css

```
:root { --brand: #164e9b; }
* { box-sizing: border-box; }
body { font-family: Arial; padding: 24px; }
form { max-width: 480px; }
label { display: block; margin-top: 16px; }
input, textarea { width: 100%; padding: 12px; font: inherit; }
button {
  margin-top: 16px; padding: 12px 18px;
  color: white; background: var(--brand); border: 0;
}
input:focus-visible, textarea:focus-visible,
button:focus-visible { outline: 3px solid #9c5c00; }
```

Now run the page or query and use the explanation and expected result on the next page.

CSS practical 5 Output and code explanation

What you should see

CSS 5 expected output in simplified form

Ask about a class
First name
[ ]
Your question
[ ]
[Try the form]

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
:root, --brand and var()	Define and reuse a custom colour property.
box-sizing, width and max-width	Keep controls within the form width including their padding.
display: block and margin-top	Put each label on its own line and separate fields.
font: inherit	Use the surrounding font in controls.
padding, color, background and border	Give comfortable controls and a distinct button.
:focus-visible and outline	Show which control the keyboard is using.
HTML form, label, input, textarea and button	Use the same field-and-label rules from HTML 4; the default button type in a form is submit.

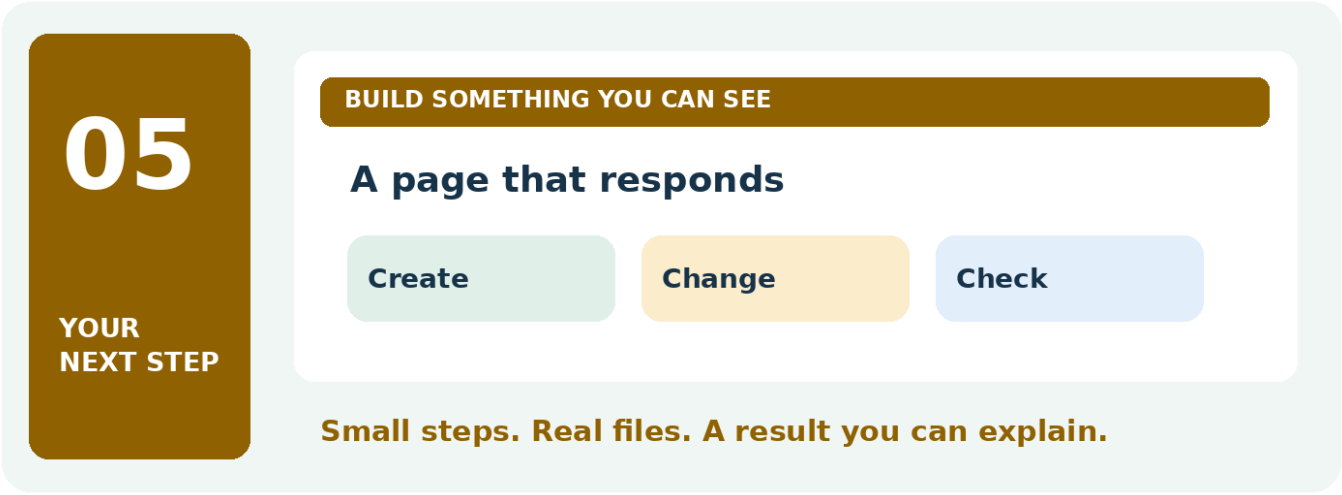
TRY IT Change --brand to #166649. Test keyboard focus and empty submission again.

READY TO CONTINUE Labels are above their fields. Controls fit a narrow screen.

IF IT DOES NOT WORK Check the linked CSS filename first. Then check the selector spelling, braces and semicolons.

Part 05 JavaScript makes a page respond

Connect a visitor’s action to a visible result.



Your finish line

Make a greeting, calculator, catalogue, answer toggle and search filter.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

JavaScript help Find the first error

1. Save index.html and app.js. Confirm the HTML includes the script line shown in the practical.

2. Open your page in the browser. Press F12, then click Console. If F12 changes volume, try Fn + F12 or Ctrl + Shift + J.

3. Click the page's button. If a red error appears, read the first one. Click its filename and line number to locate the problem.

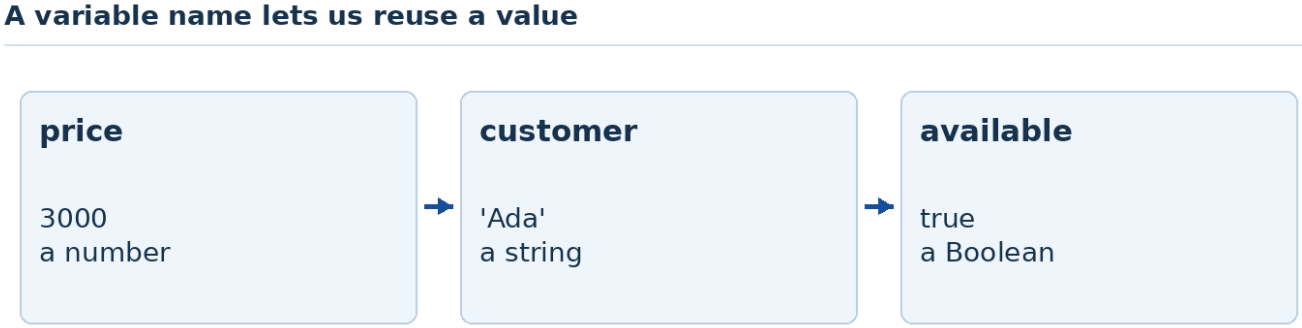
4. Compare the spelling of the id in HTML with the #name used by querySelector. They must match.

Message or symptom	Likely first check
Unexpected token	Missing quote, bracket or extra punctuation near the reported line.
Cannot read properties of null	The selector found no element. Check its spelling and script position.
Button does nothing	Confirm the script filename, save both files and refresh.
Code works only once	Repeat the action and check which value changes.

TRY IT Temporarily misspell an id in a copy of your project. Observe the error, restore the spelling and confirm the button works. You have practised fixing a real bug.

JavaScript foundation Values decisions and actions

JavaScript can respond to a visitor inside the browser. Use app.js connected by a script element with defer. The script file itself does not contain <script> tags.



Teaching illustration • Simplified view

const holds a binding you do not reassign. let allows reassignment. Text uses quotes; numbers do not. true and false are Boolean values. = assigns; === compares strictly.

A function groups instructions. Calling it runs them. if chooses whether to run a block. Arrays hold lists; objects hold named details. Events such as click and input tell us when a visitor acts.

Braces group blocks or define objects depending on context. Parentheses hold function arguments or conditions. Square brackets define arrays or access values. Semicolons end the statements in these examples.

Practise before the projects

Open a practice page, press F12 and choose Console. Type console.log(2 + 3), then Enter. Expect 5. Type console.log('2' + 3). Expect 23 because a string was joined. Close the Console when done.

Our five levels are a greeting, a calculator, an array-driven list, state that toggles, and live filtering. In each case, inspect the real page result. Source: N7.

JavaScript practical 1 A visitor greeting on a page

Job: Read a name from a field and update the page safely.

Use: VS Code and the folder javascript-1. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Just before </body>, add <script src="app.js" defer></script>. Create app.js beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Welcome to our workshop</h1>
  <label for="name">First name</label>
  <input id="name">
  <button id="greet" type="button">Greet me</button>
  <p id="result" aria-live="polite"></p>
</main>
```

app js

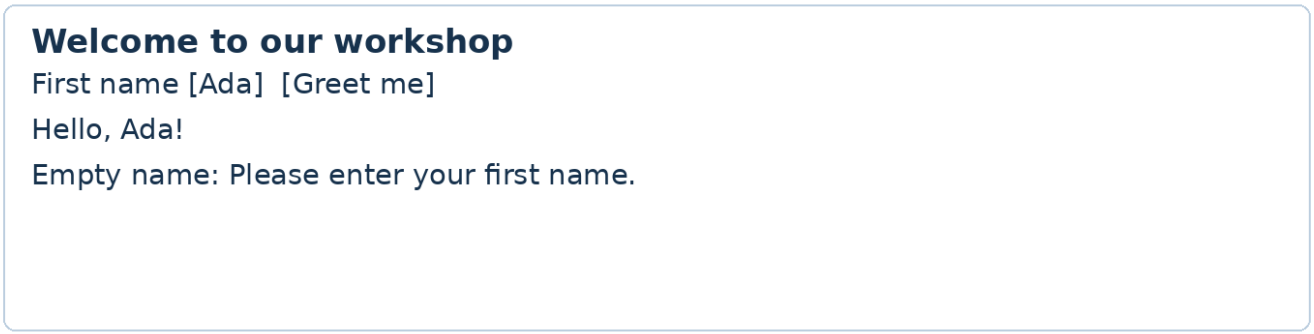
```
const nameField = document.querySelector('#name');
const button = document.querySelector('#greet');
const result = document.querySelector('#result');
button.addEventListener('click', function () {
  const name = nameField.value.trim();
  if (name === '') {
    result.textContent = 'Please enter your first name.';
    return;
  }
  result.textContent = 'Hello, ' + name + '!';
});
```

Now run the page or query and use the explanation and expected result on the next page.

JavaScript practical 1 Output and code explanation

What you should see

JavaScript 1 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
const and =	Create a binding to a value; the binding is not reassigned here.
document.querySelector and #	Find an element by its id in the DOM, the browser representation of HTML.
addEventListener, click and function	Run the enclosed instructions when the button is clicked.
value and trim()	Read field text and remove outer whitespace.
if, === and return	Test strict equality with empty text and stop early if it is missing.
textContent and +	Set text safely and join strings; typed HTML is not executed.
HTML label, input, button, p and aria-live	Collect a name, trigger an action and announce changing text politely; main and h1 identify the page.

TRY IT Change the greeting wording. Try a name containing <b> and observe literal text.

READY TO CONTINUE Ada produces Hello, Ada!. Spaces only produce the missing-name message.

IF IT DOES NOT WORK Open Console and read its first error. Check the script filename and the matching HTML ids.

JavaScript practical 2 A printing cost calculator

Job: Multiply a valid quantity by a fixed classroom unit price.

Use: VS Code and the folder javascript-2. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Just before </body>, add <script src="app.js" defer></script>. Create app.js beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Print estimate</h1>
  <label for="quantity">Number of flyers</label>
  <input id="quantity" type="number" min="1" step="1">
  <button id="calculate" type="button">Calculate</button>
  <p id="result" aria-live="polite"></p>
</main>
```

app js

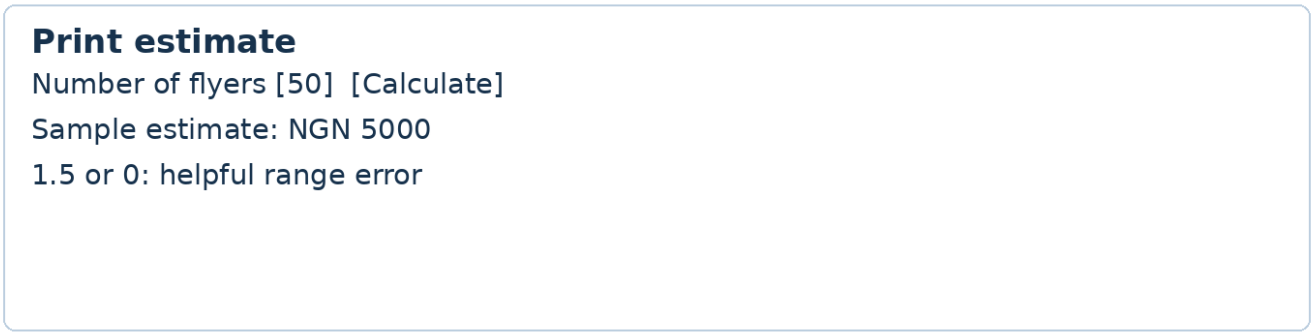
```
const field = document.querySelector('#quantity');
const result = document.querySelector('#result');
function totalFor(quantity, price) {
  return quantity * price;
}
document.querySelector('#calculate').addEventListener('click', function () {
  const quantity = Number(field.value);
  if (!Number.isInteger(quantity) || quantity < 1 || quantity > 1000) {
    result.textContent = 'Enter a whole number from 1 to 1000.';
    return;
  }
  result.textContent = 'Sample estimate: NGN ' + totalFor(quantity, 100);
});
```

Now run the page or query and use the explanation and expected result on the next page.

JavaScript practical 2 Output and code explanation

What you should see

JavaScript 2 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
querySelector, const and addEventListener	Find controls, retain references and respond to a click.
function, parameters and return	Define a reusable calculation receiving quantity and price.
Number() and field.value	Convert a field string to a number; an empty string becomes 0 and is rejected by the range test.
Number.isInteger(), ! and / /	Reject a non-integer or any value outside the range.
<, > and *	Compare lower/upper limits and multiply values.
textContent and +	Show an error or join the label to the calculated total.
HTML number input, min and step	Suggest whole positive values; JavaScript still checks them. Label, button and live paragraph provide accessible interaction.

TRY IT Change the sample price to 150 and calculate 20 flyers. Expected: 3000.

READY TO CONTINUE 50 gives 5000. Empty, 0, 1.5 and 1001 are rejected.

IF IT DOES NOT WORK Open Console and read its first error. Check the script filename and the matching HTML ids.

JavaScript practical 3 A small service catalogue

Job: Render an array of service objects as page content.

Use: VS Code and the folder javascript-3. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Just before </body>, add <script src="app.js" defer></script>. Create app.js beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Our services</h1>
  <ul id="services"></ul>
</main>
```

app js

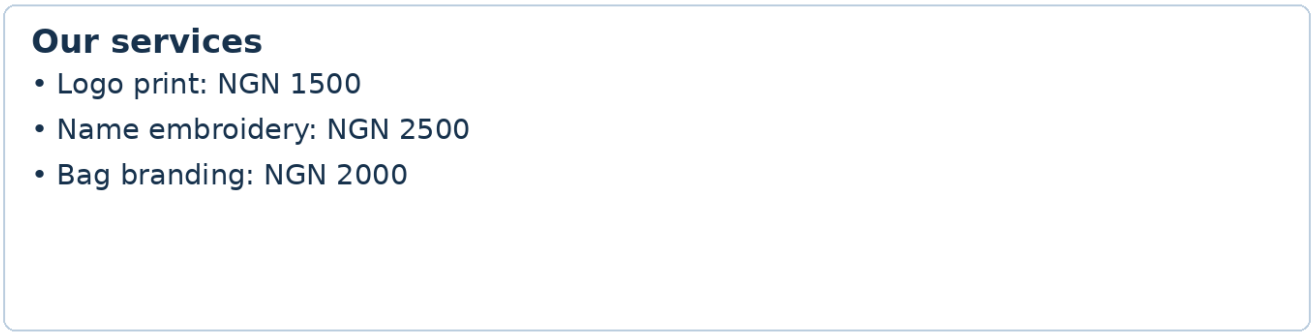
```
const services = [
  { name: 'Logo print', price: 1500 },
  { name: 'Name embroidery', price: 2500 },
  { name: 'Bag branding', price: 2000 }
];
const list = document.querySelector('#services');
for (const service of services) {
  const item = document.createElement('li');
  item.textContent = service.name + ': NGN ' + service.price;
  list.append(item);
}
```

Now run the page or query and use the explanation and expected result on the next page.

JavaScript practical 3 Output and code explanation

What you should see

JavaScript 3 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
[ ] and commas	Define an array containing several values.
{ name, price } and colon	Define an object with named properties for one service.
const, querySelector and #services	Keep the catalogue and find the destination list.
for...of	Repeat the block once for each service object.
createElement, li and append	Create an HTML list item and attach it to the list.
service.name and service.price	Read object properties using dot notation.
textContent and +	Construct and insert plain text, not interpreted HTML.
HTML main, h1 and ul	Provide a page heading and an initially empty unordered list.

TRY IT Add a fourth object. The loop should show it without another append statement.

READY TO CONTINUE Three list items appear. Changing one price changes only that item.

IF IT DOES NOT WORK Open Console and read its first error. Check the script filename and the matching HTML ids.

JavaScript practical 4 A frequently asked questions control

Job: Toggle extra information and keep its button state understandable.

Use: VS Code and the folder javascript-4. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Just before </body>, add <script src="app.js" defer></script>. Create app.js beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Delivery information</h1>
  <button id="toggle" type="button"
          aria-expanded="false" aria-controls="answer">Show answer</button>
  <p id="answer" hidden>Sample delivery time: 3 working days.</p>
</main>
```

app js

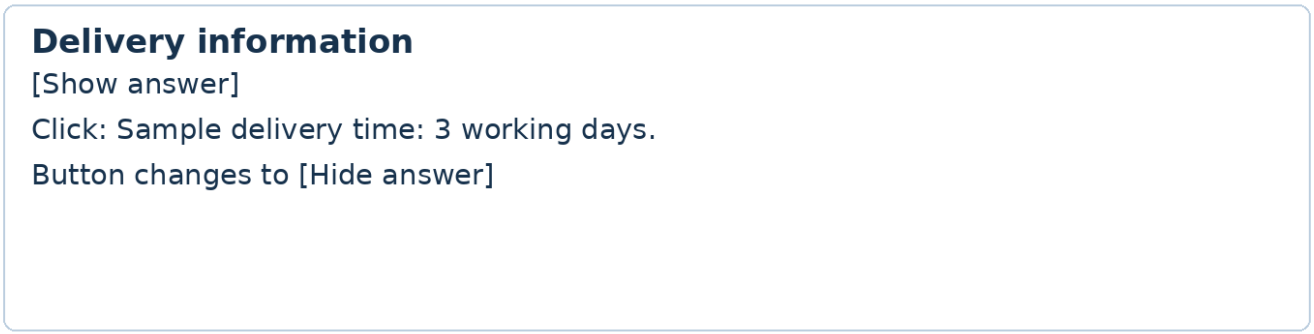
```
const button = document.querySelector('#toggle');
const answer = document.querySelector('#answer');
let open = false;
button.addEventListener('click', function () {
  open = !open;
  answer.hidden = !open;
  button.setAttribute('aria-expanded', String(open));
  button.textContent = open ? 'Hide answer' : 'Show answer';
});
```

Now run the page or query and use the explanation and expected result on the next page.

JavaScript practical 4 Output and code explanation

What you should see

JavaScript 4 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
const versus let	References stay bound; open is reassigned after every click.
false, true and !	Boolean state and logical negation switch between closed and open.
hidden	Hide or reveal the answer element.
setAttribute and String()	Update the accessibility state as the text true or false.
? : conditional operator	Choose one button label according to the current state.
querySelector, event listener and textContent	Connect the existing HTML to the click behaviour and label.
HTML button, aria-controls, aria-expanded and hidden	Identify the controlled answer and starting closed state. A real button supports keyboard activation.

TRY IT Write a different question and answer. Test using Tab and Enter only.

READY TO CONTINUE First click reveals; second click hides. aria-expanded matches the visible state.

IF IT DOES NOT WORK Open Console and read its first error. Check the script filename and the matching HTML ids.

JavaScript practical 5 A catalogue search page

Job: Filter visible services as the visitor types.

Use: VS Code and the folder javascript-5. Create your own index.html using the HTML shell. Replace its body content as shown. Keep each practical in its own folder.

Just before </body>, add <script src="app.js" defer></script>. Create app.js beside index.html.

Save your files. Right-click index.html → Open with Live Server. After each edit, save and observe the browser.

Inside body in index html

```
<main>
  <h1>Find a class</h1>
  <label for="search">Search classes</label>
  <input id="search" type="search">
  <ul id="results" aria-live="polite"></ul>
</main>
```

app js

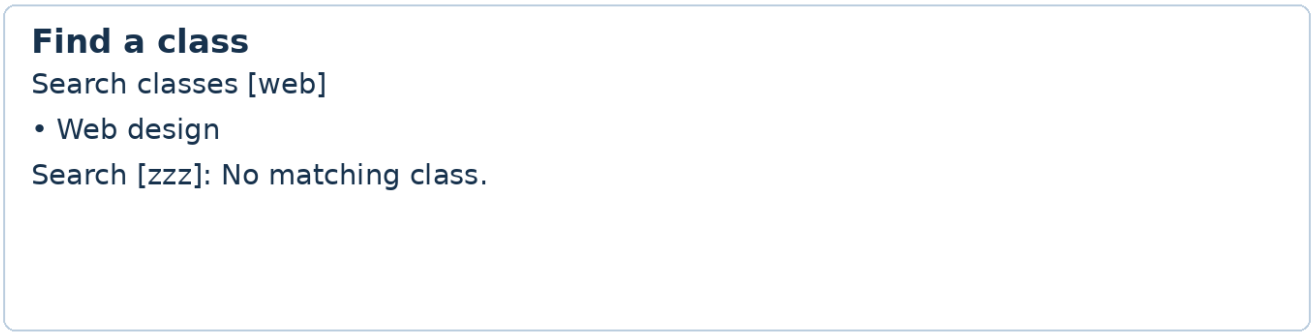
```
const classes = ['Baking basics', 'Bag sewing', 'Web design'];
const field = document.querySelector('#search');
const list = document.querySelector('#results');
function showClasses() {
  const query = field.value.trim().toLowerCase();
  const matches = classes.filter(function (name) {
    return name.toLowerCase().includes(query);
  });
  list.textContent = '';
  for (const name of matches) {
    const item = document.createElement('li');
    item.textContent = name;
    list.append(item);
  }
  if (matches.length === 0) list.textContent = 'No matching class.';
}
field.addEventListener('input', showClasses);
showClasses();
```

Now run the page or query and use the explanation and expected result on the next page.

JavaScript practical 5 Output and code explanation

What you should see

JavaScript 5 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
array, const and selectors	Store names and locate the search field and result list.
function showClasses and its final call	Define the update routine and run it once for the initial full list.
trim and toLowerCase	Normalise the query; normalise each name too.
filter, callback, return and includes	Keep names containing the query and return a new array.
textContent = empty string	Clear old results before creating the next set.
for...of, createElement and append	Render each match as a list item.
length, === and if	Show useful feedback when there are zero matches.
input event and HTML type="search"	Update while the visitor types; label and aria-live explain the field and changing results.

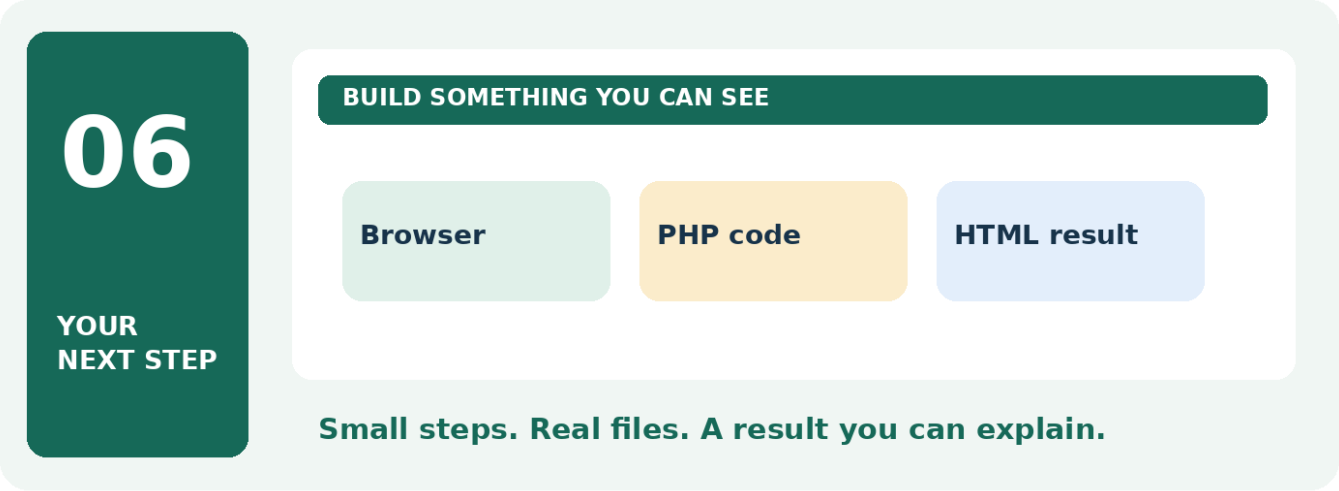
TRY IT Add Photography to the array. Search with upper-case letters and verify it still works.

READY TO CONTINUE Empty search shows all three. web shows only Web design. zzz shows the empty-result message.

IF IT DOES NOT WORK Open Console and read its first error. Check the script filename and the matching HTML ids.

Part 06 PHP works on the server

Generate pages, validate forms and remember a session.



Your finish line

Create five PHP pages and check their server-generated results.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

PHP preparation Meet the terminal slowly

Use: VS Code. Read a command first, type it once, then compare the result.

The terminal is a place to give the computer written instructions. Each instruction is called a command.

1. Open VS Code. Choose File → Open Folder and open Website Class.

2. Choose Terminal → New Terminal. A panel opens at the bottom.

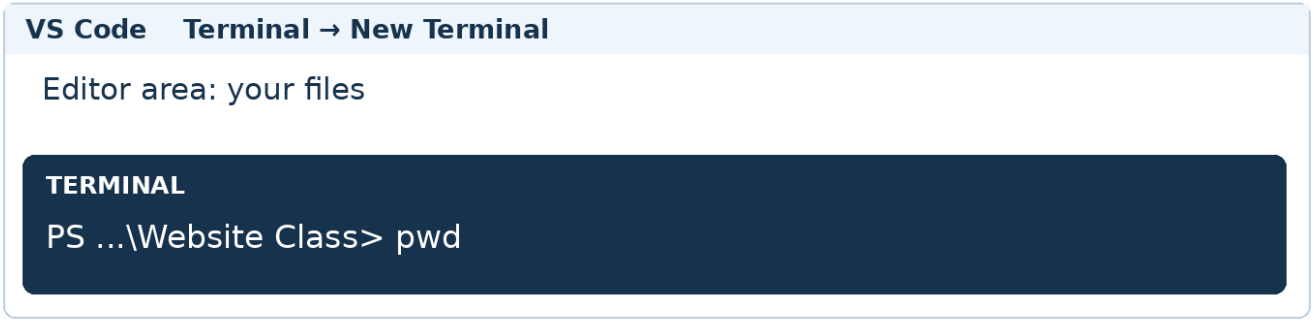
3. Click beside the blinking cursor in that panel.

4. Type the command below and press Enter once.

```
pwd
```

In Windows PowerShell, this shows your current folder. If your terminal is Command Prompt instead, use cd with no extra words.

Commands go in the bottom terminal panel



Teaching illustration • Simplified view

Three useful habits

Type one command at a time. Wait for it to finish before typing the next one. Commands go in the terminal, not inside index.html.

When a command starts a local website, it may keep running. Leave that terminal open. To stop the website, click inside that terminal and press Ctrl + C.

An address such as http://127.0.0.1:8000 points to a service on your own computer. It is not a public link you can send to a customer.

Your turn: close the terminal panel using its panel controls. Open a new terminal again. Find the folder name it shows.

PHP foundation Install and check PHP

Use: VS Code, the preceding terminal practice and Laravel Herd for Windows. No Laravel project is needed yet.

PHP needs a program that can run it. Live Server previews static files; it does not execute PHP. We will use PHP's local development server for these exercises.

1. Open https://herd.laravel.com/windows. Check the Windows requirements, download the installer and open it. Follow setup, then launch Herd and finish its welcome screens. Approve an administrator prompt only on a computer you are allowed to manage.

2. Close and reopen VS Code after installation. Choose Terminal → New Terminal.

3. Type the following, then press Enter:

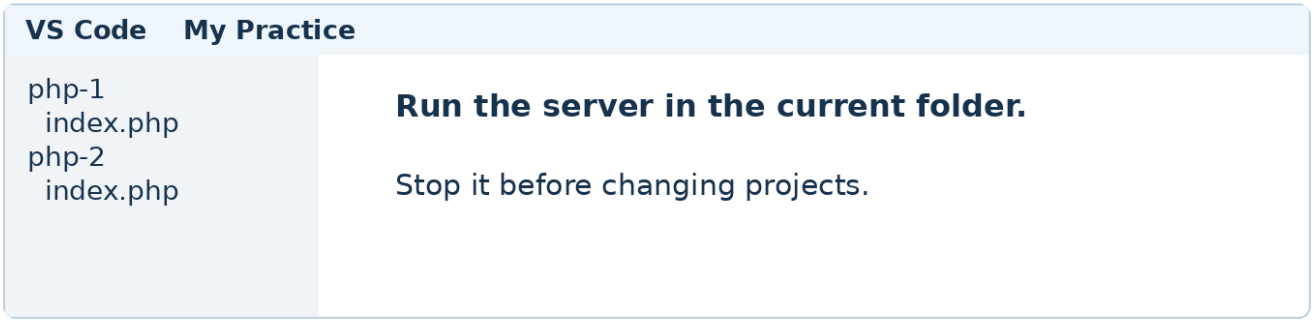
```
php --version
```

4. You should see a PHP version. If PHP is not recognised, close VS Code, open Herd once, reopen VS Code and create a new terminal. Try php --version again. Use the PHP troubleshooting page if it still fails.

5. In File Explorer, create Website Class → php-bakery. This folder will be useful later. For the five separate PHP practicals, create php-1 through php-5 inside My Practice.

6. In VS Code, choose File → Open Folder → My Practice → php-1.

Separate practice projects contain their own PHP page



Teaching illustration • Simplified view

Our separate PHP practicals use one public demonstration file each. The combined projects later separate public files from the database.

If you already have a working PHP installation, use it. Keep one active setup. Do not install XAMPP, WampServer and Herd all together just for this lesson.

Ready to move on: php --version succeeds and you can open php-1 as the project folder.

References: S28, N8, N14.

PHP help Start a server and fix setup errors

Check the tools before the code

In VS Code choose Terminal → New Terminal. Type each command on its own line and press Enter:

```
php --version
php -m
php --ini
```

The first prints a version. The second lists modules; the complete websites need PDO and pdo_sqlite. The third identifies the configuration file PHP uses.

Problem	What to do
php is not recognised	Close VS Code. Open Herd and finish setup. Reopen VS Code and a new terminal. Run php --version again.
Could not find driver	Run php -m and check pdo_sqlite. Use your PHP provider's extension instructions to enable the matching SQLite driver; restart the terminal and recheck.
Address already in use	Stop your earlier server with Ctrl + C. Or choose port 8001 and use 8001 in the browser too.
Browser says connection refused	Keep the terminal server running. Match its address and port exactly.
PHP source appears in browser	You opened a file directly or used Live Server. Open the http://127.0.0.1 address printed by PHP instead.

Open the official Herd Windows guide ↗

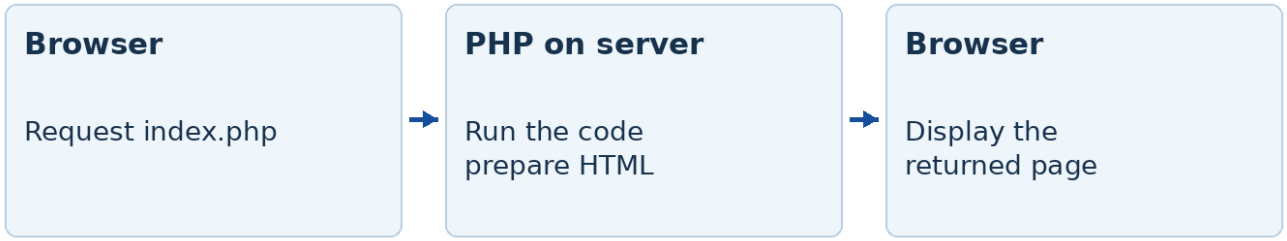
Read the PHP SQLite driver guide ↗

READY TO CONTINUE You are ready for PHP when its version command works. You are ready for the combined websites when PDO and pdo_sqlite also appear. Do not paste later commands over an unresolved setup error.

PHP foundation Where the code runs

The browser sends a request. PHP runs on the server. The server returns generated HTML. Your browser does not execute a PHP source file opened by double-clicking it.

The browser receives PHP output instead of its source



Teaching illustration • Simplified view

For each separate practical, open its folder in VS Code. Create index.php. Choose Terminal → New Terminal and run:

```
php -S 127.0.0.1:8000
```

Open http://127.0.0.1:8000/index.php. Leave the terminal running. To change projects, click the terminal, press Ctrl + C, open the next folder and restart. If port 8000 is busy, use 8001 in both command and browser address.

PHP variables start with $. Text uses quotes. A dot joins text. Most statements end with a semicolon. <?php begins code; ?> returns to surrounding HTML. <?= outputs a value.

Our five levels are a safe greeting, a calculation, arrays and loops, POST validation, and sessions. SQL is taught separately after these.

The built-in server is for local development. Later website projects run it with -t public so private code and databases are outside the served folder. Sources: N8, N14.

PHP practical 1 A server generated welcome page

Job: Use variables and safe output inside an HTML document.

Use: VS Code, PHP and your php-1 folder. Create index.php with this complete code. In Terminal → New Terminal, run php -S 127.0.0.1:8000. Open http://127.0.0.1:8000/index.php. Stop any earlier server first.

This single-file practice folder contains no database or secret files. Later complete projects use a separate public web root.

index php

```
<?php
$name = 'Ada';
function e(string $text): string {
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
}
?>
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><title>Welcome</title></head>
<body>
  <h1>Welcome, <?= e($name) ?></h1>
  <p>This greeting was prepared by PHP.</p>
</body>
</html>
```

Now run the page or query and use the explanation and expected result on the next page.

PHP practical 1 Output and code explanation

What you should see

PHP 1 expected output in simplified form

Welcome, Ada
This greeting was prepared by PHP.
View source: the greeting text appears
The PHP function itself is not sent to the browser

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
<?php, ?> and <?= ?>	Enter PHP, return to HTML and echo an expression in the short output form.
$name, =, quotes and semicolon	Name a text value, assign it and finish the statement.
function e, string and return	Define a reusable text-escaping function with string input and output.
htmlspecialchars, ENT_QUOTES and UTF-8	Escape special characters and both quote types for HTML text with this encoding.
HTML doctype, html, head, meta and title	Declare HTML, language, character encoding and browser-tab title.
body, h1 and p	Show the greeting and explanation in the page body.

TRY IT Change $name to your first name. Then try Ada and check it appears as literal text.

READY TO CONTINUE The server URL shows the heading. View Source does not contain the PHP assignment.

IF IT DOES NOT WORK Use the local server address, not a file address. Read errors in the terminal and check the named line.

PHP practical 2 A server calculated quotation page

Job: Calculate a fixed example and use a condition and reusable function.

Use: VS Code, PHP and your php-2 folder. Create index.php with this complete code. In Terminal → New Terminal, run php -S 127.0.0.1:8000. Open http://127.0.0.1:8000/index.php. Stop any earlier server first.

This single-file practice folder contains no database or secret files. Later complete projects use a separate public web root.

index php

```
<?php
$quantity = 12;
$unitPrice = 4000;
function subtotal(int $quantity, int $price): int {
    return $quantity * $price;
}
$total = subtotal($quantity, $unitPrice);
if ($quantity >= 10) {
    $note = 'Bulk enquiry';
} else {
    $note = 'Small enquiry';
}
?>
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><title>Cap quote</title></head>
<body>
  <h1>Practice cap quotation</h1>
  <p>Quantity: <?= $quantity ?></p>
  <p>Total: NGN <?= $total ?></p>
  <p><?= $note ?></p>
</body>
</html>
```

Now run the page or query and use the explanation and expected result on the next page.

PHP practical 2 Output and code explanation

What you should see

PHP 2 expected output in simplified form

Practice cap quotation
Quantity: 12
Total: NGN 48000
Bulk enquiry

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
$quantity, $unitPrice and $total	Store two whole-number inputs and the calculated result.
function, int parameters and return	Define a reusable whole-number multiplication and return its result.
*, = and semicolons	Multiply, assign and end statements.
if, >=, else and braces	Choose the note according to the quantity.
<?= ?>	Output controlled numeric values and fixed strings; user-provided strings would need escaping.
HTML structure, title, h1 and p	Create a valid page and display the three results.

TRY IT Change quantity to 5. Predict 20000 and Small enquiry, then verify.

READY TO CONTINUE 12 produces 48000. 5 selects the small-enquiry branch.

IF IT DOES NOT WORK Use the local server address, not a file address. Read errors in the terminal and check the named line.

PHP practical 3 A dynamic price list page

Job: Use an array and loop to generate repeated HTML.

Use: VS Code, PHP and your php-3 folder. Create index.php with this complete code. In Terminal → New Terminal, run php -S 127.0.0.1:8000. Open http://127.0.0.1:8000/index.php. Stop any earlier server first.

This single-file practice folder contains no database or secret files. Later complete projects use a separate public web root.

index php

```
<?php
$services = [
    ['name' => 'Haircut', 'price' => 3000],
    ['name' => 'Hair wash', 'price' => 2000],
    ['name' => 'Beard trim', 'price' => 1500]
];
function e(string $text): string {
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
}
?>
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><title>Service prices</title></head>
<body>
<h1>Practice salon prices</h1>
<ul>
<?php foreach ($services as $service): ?>
  <li><?= e($service['name']) ?>: NGN <?= $service['price'] ?></li>
<?php endforeach; ?>
</ul>
</body>
</html>
```

Now run the page or query and use the explanation and expected result on the next page.

PHP practical 3 Output and code explanation

What you should see

PHP 3 expected output in simplified form

Practice salon prices

- Haircut: NGN 3000
- Hair wash: NGN 2000
- Beard trim: NGN 1500

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
[ ], commas and =>	Build an array of associative arrays; each key maps to a value.
foreach...as, colon and endforeach	Repeat a template once per service; this is PHP alternative control syntax.
$service['name'] and ['price']	Read the current row values by key.
e(), htmlspecialchars and <?= ?>	Escape name text and output the template values; prices are controlled integers here.
HTML doctype, head, meta, title and body	Provide the basic page document.
h1, ul and li	Name the page and display each service in the generated list.

TRY IT Add a fourth service. Do not copy another li; let the loop do the work.

READY TO CONTINUE Three list items render. The added service appears after refreshing.

IF IT DOES NOT WORK Use the local server address, not a file address. Read errors in the terminal and check the named line.

PHP practical 4 A validated estimate form

Job: Process a real form on the server without storing customer data.

Use: VS Code, PHP and your php-4 folder. Create index.php with this complete code. In Terminal → New Terminal, run php -S 127.0.0.1:8000. Open http://127.0.0.1:8000/index.php. Stop any earlier server first.

This single-file practice folder contains no database or secret files. Later complete projects use a separate public web root.

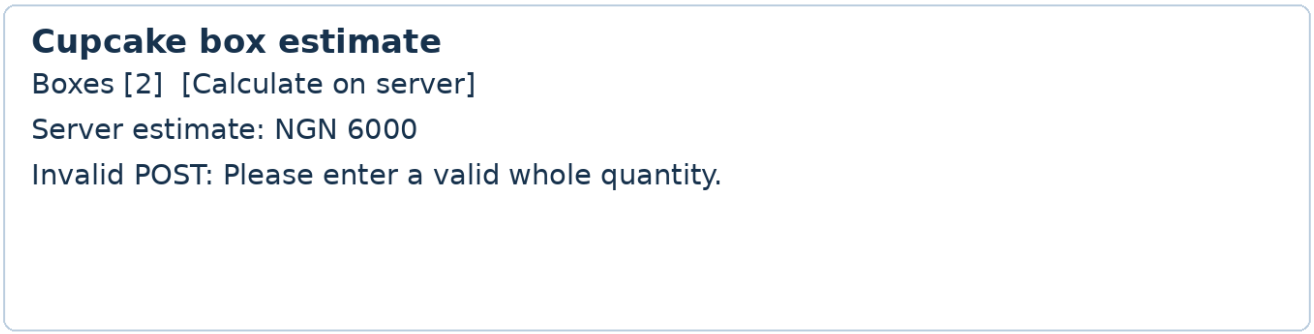
index php

```
<?php
$message = 'Enter a quantity from 1 to 100.';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $raw = $_POST['quantity'] ?? '';
    $quantity = is_string($raw) ? filter_var($raw,
        FILTER_VALIDATE_INT, ['options' => [
            'min_range' => 1, 'max_range' => 100]]) : false;
    if ($quantity === false) {
        $message = 'Please enter a valid whole quantity.';
    } else {
        $message = 'Server estimate: NGN ' . $quantity * 3000;
    }
}
?>
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><title>Bakery estimate</title></head>
<body>
<h1>Cupcake box estimate</h1>
<form method="post" action="index.php">
  <label for="qty">Boxes</label>
  <input id="qty" name="quantity" type="number" min="1" max="100" required>
  <button>Calculate on server</button>
</form>
<p><?= htmlspecialchars($message, ENT_QUOTES, 'UTF-8') ?></p>
</body>
</html>
```

PHP practical 4 Output and code explanation

What you should see

PHP 4 expected output in simplified form



Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
$_SERVER and REQUEST_METHOD	Detect whether the request is a form POST.
$_POST and ??	Read the submitted quantity or an empty default.
is_string and ? :	Reject unexpected array-shaped input before validating the scalar value.
filter_var and FILTER_VALIDATE_INT	Validate a whole number within min_range and max_range.
if, === false and else	Distinguish validation failure from valid numeric values.
dot, * and htmlspecialchars	Join the fixed message to a server calculation and escape output.
form method, action and name	POST to this PHP file and identify the submitted field.
label, id, number type, min, max and required	Provide associated labels and browser checks; PHP checks again.
button, h1, p and document shell	Submit, name the page and show the resulting message.

TRY IT Change the PHP unit price to 3500 and test two boxes. Do not add a client-controlled price.

READY TO CONTINUE 2 produces 6000. A direct invalid POST is rejected by PHP.

IF IT DOES NOT WORK Use the local server address, not a file address. Read errors in the terminal and check the named line.

PHP practical 5 A session based visit counter page

Job: Learn how a server remembers state across requests.

Use: VS Code, PHP and your php-5 folder. Create index.php with this complete code. In Terminal → New Terminal, run php -S 127.0.0.1:8000. Open http://127.0.0.1:8000/index.php. Stop any earlier server first.

This single-file practice folder contains no database or secret files. Later complete projects use a separate public web root.

index php

```
<?php
session_start();
$_SESSION['visits'] = ($_SESSION['visits'] ?? 0) + 1;
$visits = $_SESSION['visits'];
?>
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><title>Session practice</title></head>
<body>
<h1>Welcome back to our reading club</h1>
<p>Visits in this session: <?= $visits ?></p>
<p>This is a counter, not a sign-in system.</p>
</body>
</html>
```

Now run the page or query and use the explanation and expected result on the next page.

PHP practical 5 Output and code explanation

What you should see

PHP 5 expected output in simplified form

Welcome back to our reading club

Visits in this session: 1

Refresh: 2, then 3

Separate private window: usually starts at 1

Teaching illustration • Simplified view

What each part does

Tag or code	Usage in this practical
session_start()	Start or resume a session before any HTML output.
$_SESSION and visits key	Keep a server-side value associated with this session.
?? 0, + 1 and assignment	Use zero on the first visit, increment and save.
$visits and <?= ?>	Read and display the controlled integer counter.
Cookie relationship	A browser cookie usually carries an identifier, while this value is stored server-side.
HTML document shell, h1 and p	Provide a valid page with a heading and two paragraphs.

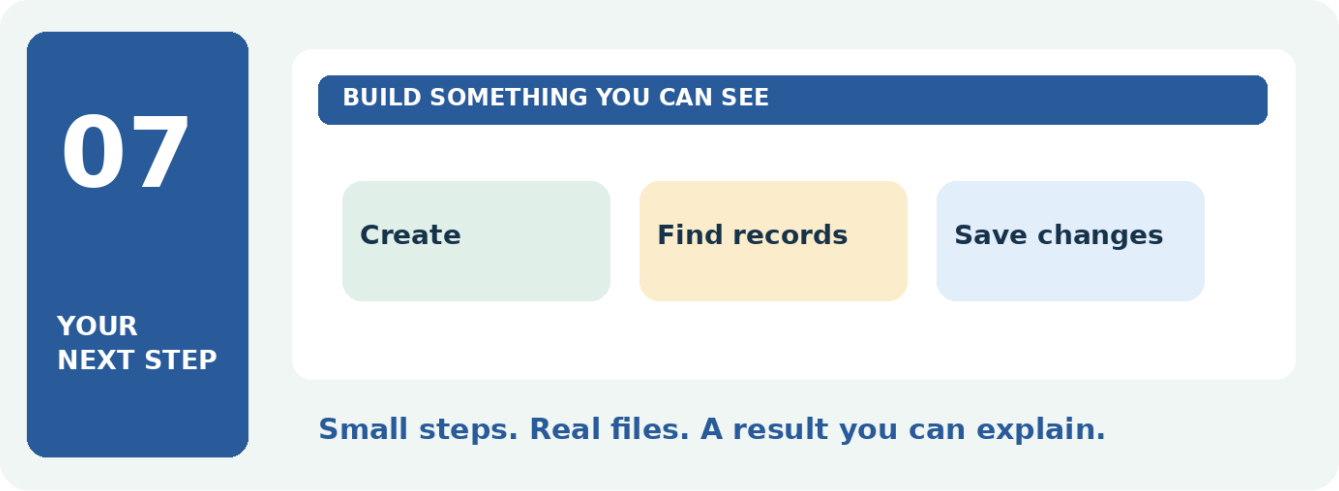
TRY IT Refresh three times. Compare a normal window and a private window, then explain why this is not authentication.

READY TO CONTINUE Counter increases in one session. PHP starts the session before HTML output.

IF IT DOES NOT WORK Use the local server address, not a file address. Read errors in the terminal and check the named line.

Part 07 SQL works with stored data

Create tables, select records and understand related data.



Your finish line

Complete five database practicals before connecting SQL to a website.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

SQL foundation Understand records and relationships

SQL is the language used to define, read and change data in a relational database. It does not colour headings or draw buttons.

A database table stores one record per row

Database products

id	name	price
1	Polo shirt	7500
2	Canvas bag	6000
3	Cap	4000

Teaching illustration • Simplified view

A table holds rows and columns. A row is one record, such as a product. A column is one kind of detail, such as its price. A primary key identifies a row. A foreign key links a row to a record in another table.

We use SQLite for these beginner exercises because the database is a local file. MySQL and PostgreSQL are other database systems that use SQL. Their setup, types and some syntax differ, so do not assume every SQLite command works unchanged in them.

Our five levels are creating and inserting, selecting and filtering, updating and deleting, joining related records, and reporting with transactions.

Use invented products and customer names. Prices are whole-number NGN examples. For money requiring fractional units, decide on a smallest-unit integer representation such as kobo.

When SQL is connected to PHP later, the server will use prepared statements to bind values. A browser does not get direct database access.

Sources: N10, N12.

SQL foundation Open DB Browser and run a query

1. Visit https://sqlitebrowser.org/dl/. In Windows, open Settings → System → About and read System type. Choose the matching Windows installer, download it, open it and follow setup. Launch DB Browser for SQLite from Start.

2. Open DB Browser for SQLite. Click New Database. Save class-practice.sqlite inside My Practice → sql. If a Create Table dialog appears, cancel it; we will create the table using SQL.

3. Click the Execute SQL tab. This is where the commands go. Do not put SQL into index.html.

4. Type SELECT 2 + 3 AS answer; and click the execute control, usually a play triangle. You should see one result cell: 5.

5. Replace that query with the SQL 1 code. Run it once. Use Browse Data to inspect products and Write Changes to save pending changes.

Type commands above and read the result below



Teaching illustration • Simplified view

Before the next practical, clear only the query editor text. Keep the database tables. Run SQL 1 to SQL 5 in order. Do not run the table-creation scripts repeatedly on the same database.

For a fresh start, create a new empty database and run the scripts again in order. This is safer than deleting tables you do not recognise. Click Write Changes before beginning the transaction exercise in SQL 5.

The query result grid is the output for this separate SQL section. The combined websites later turn database results into HTML pages.

Sources: N9, N10.

SQL help Run one statement and read the result

Open DB Browser → Open Database and choose class-practice.sqlite. Choose Execute SQL. This is the box where you type SQL; the result table appears below it.

1. Type SELECT 2 + 3 AS answer; and run it. Expect one column named answer with the value 5.

2. If an exercise has several SELECT statements, highlight one complete statement, including its semicolon, then use Execute selected SQL. Read that result before selecting the next statement.

3. Use Write Changes after completing changes. Before SQL practical 5, use Write Changes so no earlier transaction remains open.

4. Run practicals 1–5 in order, once each, in the same fresh database. If you want to restart, create a new database with a new filename.

Type commands above and read the result below



Teaching illustration • Simplified view

IF IT DOES NOT WORK "Table already exists" means the table was created earlier. Do not keep rerunning the setup. Continue with the next statement, or start a new practice database. "No such table" usually means a different database is open or an earlier practical was skipped.

TRY IT Change 2 + 3 to 10 + 5. Predict 15, run the query and compare. A query is simply a request you write to the database.

SQL practical 1 Create a product table

Job: Build the data behind a small merchandise catalogue.

Use: DB Browser for SQLite and the same classroom database from SQL 1. Run these practicals in order. Open Execute SQL, clear the previous query text and type this exercise. Run selected statements when comparing more than one SELECT. Write Changes after each completed practical.

Start with a new empty database named class-practice.sqlite. Do not rerun CREATE TABLE on the already-created tables.

practical 1 sql

```
PRAGMA foreign_keys = ON;
CREATE TABLE products (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  category TEXT NOT NULL,
  price INTEGER NOT NULL CHECK (price >= 0)
);
INSERT INTO products (id, name, category, price) VALUES
  (1, 'Polo shirt', 'Clothing', 7500),
  (2, 'Canvas bag', 'Bags', 6000),
  (3, 'Cap', 'Clothing', 4000);
SELECT id, name, category, price FROM products;
```

Now run the page or query and use the explanation and expected result on the next page.

SQL practical 1 Output and code explanation

What you should see

SQL 1 expected output in simplified form

```
id | name | category | price
1 | Polo shirt | Clothing | 7500
2 | Canvas bag | Bags | 6000
3 | Cap | Clothing | 4000
```

Teaching illustration • Simplified view

id	name	category	price
1	Polo shirt	Clothing	7500
2	Canvas bag	Bags	6000
3	Cap	Clothing	4000

What each part does

Tag or code	Usage in this practical
PRAGMA foreign_keys	Enables SQLite foreign-key enforcement for this connection.
CREATE TABLE, parentheses and commas	Define a table and separate its columns.
INTEGER, TEXT and PRIMARY KEY	Choose whole numbers or text and identify each row uniquely.
NOT NULL and CHECK	Require a value and reject negative prices.
INSERT INTO, column list and VALUES	Add three records in the specified column order.
SELECT and FROM	Choose columns and the table to read.
Quotes and semicolons	Single quotes enclose SQL text; semicolons end statements.

TRY IT Add a fictional fourth product with a new id. Then remove it or reset from the supplied starting file before SQL 2.

READY TO CONTINUE Compare every row with the expected output. Write Changes after completing the practical.

IF IT DOES NOT WORK Check the open database and exercise order. Run one selected statement when comparing results.

SQL practical 2 Find the products a visitor needs

Job: Select only products that meet a condition and order the result.

Use: DB Browser for SQLite and the same classroom database from SQL 1. Run these practicals in order. Open Execute SQL, clear the previous query text and type this exercise. Run selected statements when comparing more than one SELECT. Write Changes after each completed practical.

practical 2 sql

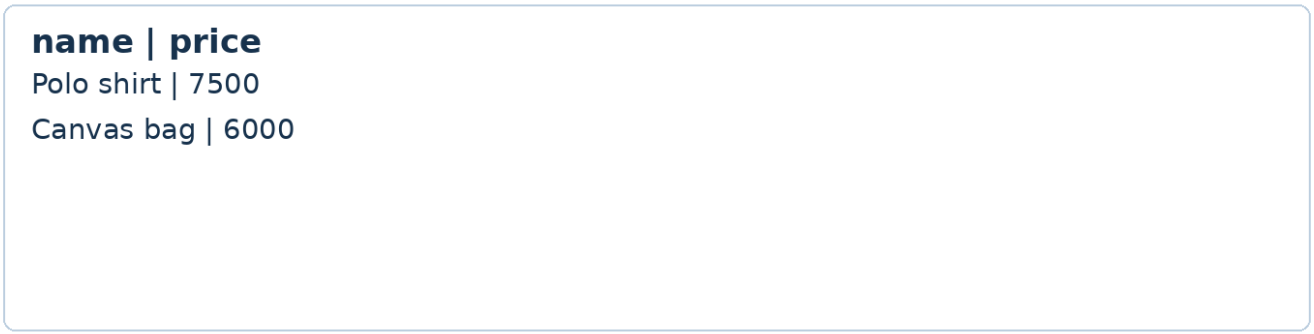
```
SELECT name, price
FROM products
WHERE price >= 6000
ORDER BY price DESC;
SELECT name
FROM products
WHERE name LIKE '%bag%';
```

Now run the page or query and use the explanation and expected result on the next page.

SQL practical 2 Output and code explanation

What you should see

SQL 2 expected output in simplified form



Teaching illustration • Simplified view

name	price
Polo shirt	7500
Canvas bag	6000

The second SELECT returns Canvas bag. Run it separately to see that result.

What each part does

Tag or code	Usage in this practical
SELECT and FROM	Return only the named columns from products.
WHERE and >=	Include only rows whose price is at least 6000.
ORDER BY and DESC	Sort from the highest price to the lowest.
LIKE and %	Match a text pattern; % stands for any sequence of characters.
Two statements	Each SELECT has its own result grid; run one at a time to inspect both.

TRY IT Change the minimum price to 7000. Expect only Polo shirt. Restore the original query.

READY TO CONTINUE Compare every row with the expected output. Write Changes after completing the practical.

IF IT DOES NOT WORK Check the open database and exercise order. Run one selected statement when comparing results.

SQL practical 3 Correct and remove specific records

Job: Practise changes with a WHERE clause on a local classroom database.

Use: DB Browser for SQLite and the same classroom database from SQL 1. Run these practicals in order. Open Execute SQL, clear the previous query text and type this exercise. Run selected statements when comparing more than one SELECT. Write Changes after each completed practical.

practical 3 sql

```
UPDATE products
SET price = 4500
WHERE id = 3;
INSERT INTO products (id, name, category, price)
VALUES (4, 'Practice only', 'Test', 0);
DELETE FROM products WHERE id = 4;
SELECT id, name, price FROM products ORDER BY id;
```

Now run the page or query and use the explanation and expected result on the next page.

SQL practical 3 Output and code explanation

What you should see

SQL 3 expected output in simplified form

id | name | price
1 | Polo shirt | 7500
2 | Canvas bag | 6000
3 | Cap | 4500

Teaching illustration • Simplified view

id	name	price
1	Polo shirt	7500
2	Canvas bag	6000
3	Cap	4500

What each part does

Tag or code	Usage in this practical
UPDATE and SET	Change the selected record value.
WHERE id = 3	Limit the update to the cap row; without WHERE, all product prices would change.
INSERT INTO and VALUES	Create a disposable classroom record.
DELETE FROM and WHERE	Remove only that disposable row. DELETE removes rows, not the table definition.
SELECT and ORDER BY	Inspect the remaining rows in a predictable order.

TRY IT Before running a new UPDATE, use SELECT with the same WHERE condition to inspect the target.

READY TO CONTINUE Compare every row with the expected output. Write Changes after completing the practical.

IF IT DOES NOT WORK Check the open database and exercise order. Run one selected statement when comparing results.

SQL practical 4 Join customer and order information

Job: Connect separate records instead of repeating a customer name in every order.

Use: DB Browser for SQLite and the same classroom database from SQL 1. Run these practicals in order. Open Execute SQL, clear the previous query text and type this exercise. Run selected statements when comparing more than one SELECT. Write Changes after each completed practical.

practical 4 sql

```
CREATE TABLE customers (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL
);
CREATE TABLE orders (
  id INTEGER PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES customers(id),
  product_id INTEGER NOT NULL REFERENCES products(id),
  quantity INTEGER NOT NULL CHECK (quantity > 0)
);
INSERT INTO customers VALUES (1, 'Ada'), (2, 'Tunde');
INSERT INTO orders VALUES (1, 1, 2, 5), (2, 2, 1, 2);
SELECT c.name, p.name AS product, o.quantity
FROM orders AS o
JOIN customers AS c ON c.id = o.customer_id
JOIN products AS p ON p.id = o.product_id
ORDER BY o.id;
```

Now run the page or query and use the explanation and expected result on the next page.

SQL practical 4 Output and code explanation

What you should see

SQL 4 expected output in simplified form

```
name | product | quantity
Ada | Canvas bag | 5
Tunde | Polo shirt | 2
```

Teaching illustration • Simplified view

name	product	quantity
Ada	Canvas bag	5
Tunde	Polo shirt	2

What each part does

Tag or code	Usage in this practical
REFERENCES	Declare links from each order to an existing customer and product.
CHECK quantity > 0	Require positive quantities.
INSERT without a column list	Uses the table column order; explicit column lists are clearer in longer applications.
AS and aliases o, c and p	Give tables short names and rename the product result column.
JOIN and ON	Combine matching rows using their related keys.
c.name, p.name and o.quantity	Qualify which table each column comes from.
ORDER BY o.id	Present orders in their id order.

TRY IT Try adding an order using a nonexistent customer id with foreign keys enabled. Expect a constraint error, not a saved order.

READY TO CONTINUE Compare every row with the expected output. Write Changes after completing the practical.

IF IT DOES NOT WORK Check the open database and exercise order. Run one selected statement when comparing results.

SQL practical 5 Build a report and undo a trial change

Job: Summarise orders and see how a transaction protects an experiment.

Use: DB Browser for SQLite and the same classroom database from SQL 1. Run these practicals in order. Open Execute SQL, clear the previous query text and type this exercise. Run selected statements when comparing more than one SELECT. Write Changes after each completed practical.

practical 5 sql

```
SELECT c.name, SUM(o.quantity * p.price) AS total
FROM orders AS o
JOIN customers AS c ON c.id = o.customer_id
JOIN products AS p ON p.id = o.product_id
GROUP BY c.id, c.name
ORDER BY total DESC;
BEGIN TRANSACTION;
UPDATE products SET price = 1 WHERE id = 2;
ROLLBACK;
SELECT name, price FROM products WHERE id = 2;
```

Now run the page or query and use the explanation and expected result on the next page.

SQL practical 5 Output and code explanation

What you should see

SQL 5 expected output in simplified form

name | total
Ada | 30000
Tunde | 15000

Teaching illustration • Simplified view

name	total
Ada	30000
Tunde	15000

After ROLLBACK, the final SELECT returns Canvas bag with price 6000. Write Changes before BEGIN if DB Browser has pending edits.

What each part does

Tag or code	Usage in this practical
SUM, multiplication and AS total	Calculate each line value and add values into a named result.
GROUP BY	Produce one summary per customer rather than one row per order.
ORDER BY total DESC	Show the largest summary first.
BEGIN TRANSACTION and ROLLBACK	Start a group of changes and cancel them; COMMIT would keep them.
Final SELECT	Proves the bag price returned to 6000.
Reporting limitation	This teaching report uses current catalogue prices. The combined websites save quoted unit prices so later price changes do not rewrite old totals.

TRY IT Explain when to choose COMMIT and when to choose ROLLBACK. Do not experiment on a real business database.

READY TO CONTINUE Compare every row with the expected output. Write Changes after completing the practical.

IF IT DOES NOT WORK Check the open database and exercise order. Run one selected statement when comparing results.

Foundation checkpoint Before combining the languages

Demonstrate these tasks without asking AI to replace the whole project.

Subject	Prove it to yourself
Internet	Explain a browser request, DNS and a server response
HTML	Create a labelled field and a working link
CSS	Change spacing and make a layout adapt to a narrow screen
JavaScript	Read input, validate it and update visible text
PHP	Process POST and reject an invalid value on the server
SQL	Insert a record and retrieve it with a condition

A useful way to explain your work

“This tag creates the input. This rule styles it. This event reacts to typing. This PHP code checks the submitted value. This SQL statement saves the record.”



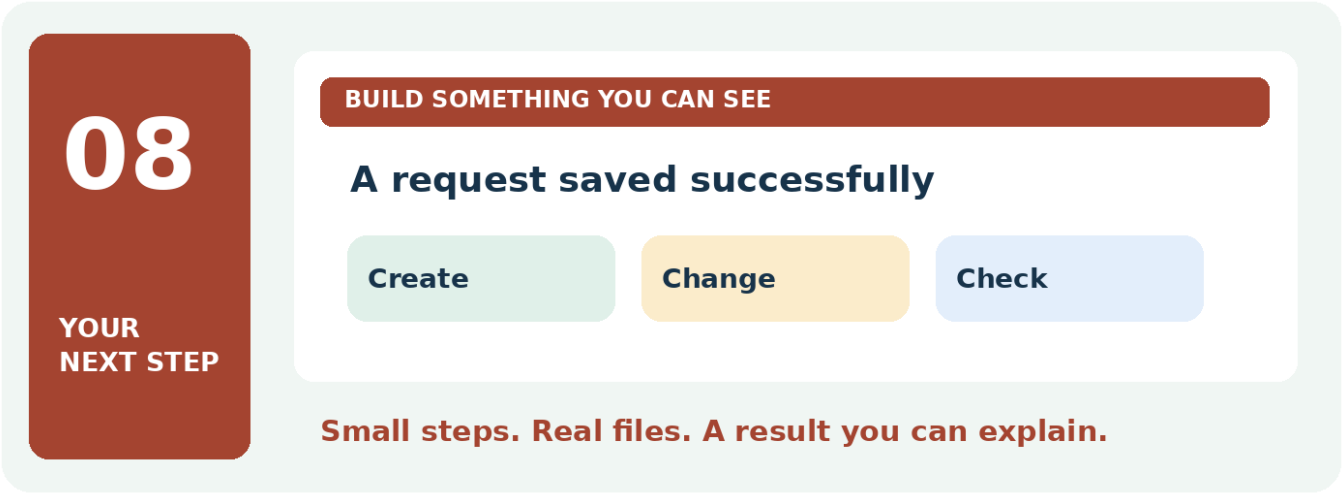
Teaching illustration • Simplified view

A page working in the browser is a good start. A saved record that you can read back is stronger evidence that the whole flow works.

Keep a backup of your successful exercises. The five websites next reuse the same small application structure so you learn how to adapt it, not simply copy five unrelated piles of code.

Part 08 Put the five subjects together

Follow one enquiry from a page to a saved database record.



Your finish line

Build the bakery site, then adapt it into four more complete enquiry websites.

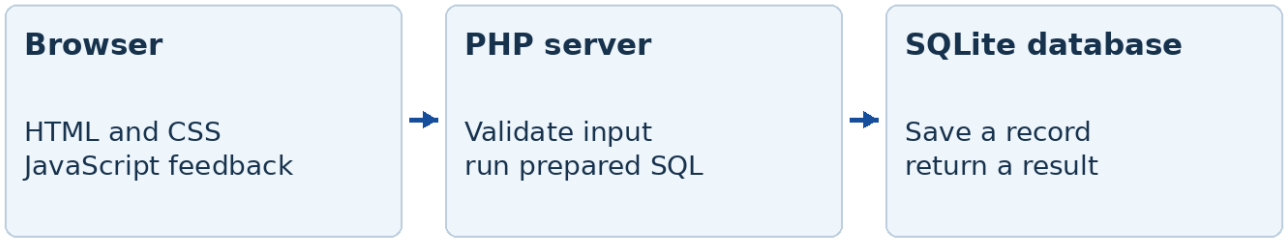
TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

Five complete websites How the parts work together

Each website has Home, Catalogue, Request and Result pages. Its catalogue comes from SQLite. JavaScript filters the cards. PHP validates submissions and saves records using SQL. The result page confirms the saved request.

Follow a form from the screen to a saved record



Teaching illustration • Simplified view

These are five complete small enquiry websites, with different subjects and rules. They are not full e-commerce, school management or hotel reservation systems. Their forms save real local records; they do not send email, take payment or confirm availability.

Build one then adapt it carefully

Website 1 teaches the complete shared source code. Websites 2–5 reuse that code and change a configuration file, content and business rules. Each has its own standalone folder, database, test cases and student task. Reuse is deliberate: it is a practical professional skill.

Type Website 1's files in the shown folders, or open the supplied website-1 folder to compare. Never serve the whole project folder. The command must include -t public because private PHP files and the database live above that folder.

The code uses a CSRF token, server validation, escaped HTML, prepared SQL and a redirect after POST. These are useful foundations. Real public deployment still requires a production environment, secure sessions, operational checks and experienced review.

Website 1 Build a bakery website

Job: create a bakery catalogue with a searchable product list and a saved enquiry.

1. Make a folder named website-1. Inside it, create public and data. Open website-1 in VS Code.

2. Create the named files on the following pages. Root-level files sit beside public, not inside it. Only names beginning public/ go inside public.

3. Check php --version and php -m in Terminal. The modules must include PDO and pdo_sqlite. If a driver is missing, follow the PHP troubleshooting page before continuing.

4. After saving all files, run php setup.php from website-1. Expect “Setup complete. Start the local server.”

5. Run php -S 127.0.0.1:8000 -t public. Visit http://127.0.0.1:8000/index.php. Keep the terminal running.

Only the public folder is exposed by the local server



Teaching illustration • Simplified view

Use the supplied files as the answer copy if you lose a bracket. Read the explanation after the code before moving to Website 2.

Sample acceptance test: choose Vanilla box, enter Ada and quantity 2. The result should say NGN 6000. A row should appear in the database. This will be a request, not a real order.

Website 1 File config php

File: config.php. Create this file at the exact path shown.

```
<?php
return [
    'name' => 'Ada Bakes',
    'headline' => 'Cakes for your next celebration',
    'intro' => 'Explore our fictional bakery catalogue and practise making an
enquiry.',
    'quantityLabel' => 'Number of boxes',
    'min' => 1,
    'max' => 100,
    'notice' => 'Sample bakery enquiry. No payment or delivery is arranged.',
    'items' => [
        [1, 'Vanilla box', 3000],
        [2, 'Chocolate box', 3500],
        [3, 'Fruit box', 4000],
    ]
];
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File schema sql

File: schema.sql. Create this file at the exact path shown.

```
CREATE TABLE IF NOT EXISTS items (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  price INTEGER NOT NULL CHECK (price >= 0)
);
CREATE TABLE IF NOT EXISTS requests (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer TEXT NOT NULL,
  item_id INTEGER NOT NULL REFERENCES items(id),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price INTEGER NOT NULL,
  total INTEGER NOT NULL
);
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File setup php

File: setup.php. Create this file at the exact path shown.

```
<?php
$config = require __DIR__ . '/config.php';
if (!is_dir(__DIR__ . '/data')) mkdir(__DIR__ . '/data');
$db = new PDO('sqlite:' . __DIR__ . '/data/site.sqlite');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('PRAGMA foreign_keys = ON');
$db->exec(file_get_contents(__DIR__ . '/schema.sql'));
$insert = $db->prepare(
    'INSERT OR IGNORE INTO items (id, name, price) VALUES (?, ?, ?)'
);
foreach ($config['items'] as $item) $insert->execute($item);
echo "Setup complete. Start the local server.\n";
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File app php

File: app.php. Create this file at the exact path shown.

```
<?php
session_start();
$config = require __DIR__ . '/config.php';
function e(string $text): string {
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
}
if (!is_file(__DIR__ . '/data/site.sqlite')) {
    exit('Run php setup.php from the project folder first.');
}
$db = new PDO('sqlite:' . __DIR__ . '/data/site.sqlite');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('PRAGMA foreign_keys = ON');
if (!isset($_SESSION['token'])) {
    $_SESSION['token'] = bin2hex(random_bytes(32));
}
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File header php

File: header.php. Create this file at the exact path shown.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title><?= e($config['name']) ?></title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<header>
  <p class="brand"><?= e($config['name']) ?></p>
  <nav aria-label="Main navigation">
    <a href="index.php">Home</a>
    <a href="catalog.php">Catalogue</a>
    <a href="request.php">Make a request</a>
  </nav>
</header>
<main>
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File footer php

File: footer.php. Create this file at the exact path shown.

```
</main>
<footer><p>Fictional classroom website. Requests are not confirmed orders.</p></footer>
<script src="app.js" defer></script>
</body>
</html>
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public index php

File: public/index.php. Create this file at the exact path shown.

```
<?php
require __DIR__ . '/../app.php';
require __DIR__ . '/../header.php';
?>
<h1><?= e($config['headline']) ?></h1>
<p><?= e($config['intro']) ?></p>
<p><a class="button" href="catalog.php">Explore our options</a></p>
<section>
  <h2>How it works</h2>
  <ol>
    <li>Choose an option from our catalogue.</li>
    <li>Send a fictional practice request.</li>
    <li>Review the saved request confirmation.</li>
  </ol>
</section>
<?php require __DIR__ . '/../footer.php'; ?>
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public catalog php

File: public/catalog.php. Create this file at the exact path shown.

```
<?php
require __DIR__ . '/../app.php';
$items = $db->query('SELECT * FROM items ORDER BY id')
    ->fetchAll(PDO::FETCH_ASSOC);
require __DIR__ . '/../header.php';
?>
<h1>Choose an option</h1>
<label for="search">Search the catalogue</label>
<input id="search" type="search">
<p id="empty" hidden>No matching option.</p>
<div class="cards">
<?php foreach ($items as $item): ?>
  <article data-name="<?= e(strtolower($item['name'])) ?>">
    <h2><?= e($item['name']) ?></h2>
    <p>Sample unit price: NGN <?= (int) $item['price'] ?></p>
    <a href="request.php?item=<?= (int) $item['id'] ?>">Request this</a>
  </article>
<?php endforeach; ?>
</div>
<?php require __DIR__ . '/../footer.php'; ?>
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public request php

File: public/request.php. Create this file at the exact path shown.

```
<?php
require __DIR__ . '/../app.php';
$items = $db->query('SELECT * FROM items ORDER BY id')
    ->fetchAll(PDO::FETCH_ASSOC);
$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $token = $_POST['token'] ?? '';
    if (!is_string($token) || !hash_equals($_SESSION['token'], $token)) {
        http_response_code(403);
        exit('Form expired. Return to the request page.');
    }
    $name = $_POST['customer'] ?? '';
    $itemId = $_POST['item'] ?? '';
    $raw = $_POST['quantity'] ?? '';
    if (!is_string($name) || !is_string($itemId) || !is_string($raw)) {
        http_response_code(400);
        exit('Invalid input.');
    }
    $name = trim($name);
    $qty = filter_var($raw, FILTER_VALIDATE_INT, ['options' => [
        'min_range' => $config['min'], 'max_range' => $config['max']]]);
    $find = $db->prepare('SELECT * FROM items WHERE id = ?');
    $find->execute([$itemId]);
    $item = $find->fetch(PDO::FETCH_ASSOC);
    if ($name === '' || strlen($name) > 60
        || $qty === false || !$item) {
        $error = 'Check your name, option and quantity range.';
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public request php continued 2

File: public/request.php. Append these lines below the previous part in the same file.

```
} else {
        $total = $qty * (int) $item['price'];
        $save = $db->prepare('INSERT INTO requests
          (customer, item_id, quantity, unit_price, total)
          VALUES (?, ?, ?, ?, ?)');
        $save->execute([$name, $item['id'], $qty, $item['price'], $total]);
        $_SESSION['receipt'] = ['name' => $name, 'total' => $total,
            'id' => $db->lastInsertId()];
        $_SESSION['token'] = bin2hex(random_bytes(32));
        header('Location: thanks.php', true, 303);
        exit;
    }
}
$selected = filter_input(INPUT_GET, 'item', FILTER_VALIDATE_INT);
require __DIR__ . '/../header.php';
?>
<h1>Make a practice request</h1>
<p><?= e($config['notice']) ?></p>
<p role="alert"><?= e($error) ?></p>
<form method="post" action="request.php">
  <input type="hidden" name="token" value="<?= e($_SESSION['token']) ?>">
  <label for="customer">First name</label>
  <input id="customer" name="customer" maxlength="60" required>
  <label for="item">Choose an option</label>
  <select id="item" name="item">
  <?php foreach ($items as $row): ?>
    <option value="<?= (int) $row['id'] ?>"
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public request php continued 3

File: public/request.php. Append these lines below the previous part in the same file.

```
<?= $selected === (int) $row['id'] ? 'selected' : '' ?>>
      <?= e($row['name']) ?>
    </option>
  <?php endforeach; ?>
  </select>
  <label for="quantity"><?= e($config['quantityLabel']) ?></label>
  <input id="quantity" name="quantity" type="number" step="1"
    min="<?= $config['min'] ?>" max="<?= $config['max'] ?>" required>
  <button type="submit">Save practice request</button>
</form>
<?php require __DIR__ . '/../footer.php'; ?>
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public thanks php

File: public/thanks.php. Create this file at the exact path shown.

```
<?php
require __DIR__ . '/../app.php';
$receipt = $_SESSION['receipt'] ?? null;
require __DIR__ . '/../header.php';
?>
<h1>Request result</h1>
<?php if ($receipt): ?>
  <p>Saved request <?= (int) $receipt['id'] ?></p>
  <p>Thank you, <?= e($receipt['name']) ?>.</p>
  <p>Recorded estimate: NGN <?= (int) $receipt['total'] ?></p>
  <p>This is a request, not a confirmed booking or payment.</p>
<?php else: ?>
  <p>No request has been submitted in this session.</p>
<?php endif; ?>
<?php require __DIR__ . '/../footer.php'; ?>
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public style css

File: public/style.css. Create this file at the exact path shown.

```
* { box-sizing: border-box; }
body {
  margin: 0; color: #17324d; background: #f4f8fb;
  font-family: Arial, sans-serif; line-height: 1.6;
}
header, main, footer { max-width: 1050px; margin: auto; padding: 24px; }
.brand { font-weight: bold; font-size: 1.4rem; }
nav { display: flex; gap: 20px; flex-wrap: wrap; }
a { color: #164e9b; }
h1 { font-size: 2.3rem; line-height: 1.2; }
.cards { display: grid; gap: 20px; }
article { padding: 24px; background: white; border: 1px solid #bdcddd; }
form { max-width: 500px; }
label { display: block; margin-top: 16px; }
input, select { width: 100%; padding: 12px; font: inherit; }
button, .button {
  display: inline-block; padding: 12px 18px; margin-top: 16px;
  background: #164e9b; color: white; border: 0; font: inherit;
}
a:focus-visible, input:focus-visible,
select:focus-visible, button:focus-visible {
  outline: 3px solid #9c5c00; outline-offset: 3px;
}
@media (min-width: 720px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 File public app js

File: public/app.js. Create this file at the exact path shown.

```
const search = document.querySelector('#search');
if (search) {
  const cards = document.querySelectorAll('[data-name]');
  const empty = document.querySelector('#empty');
  search.addEventListener('input', function () {
    const query = search.value.trim().toLowerCase();
    let visible = 0;
    for (const card of cards) {
      const match = card.dataset.name.includes(query);
      card.hidden = !match;
      if (match) visible += 1;
    }
    empty.hidden = visible !== 0;
  });
}
```

Save this file. The complete project runs after all the listed files are present. The shared code walkthrough follows the file listings.

Website 1 Inspect the saved record

Create inspect.php in website-1, outside public:

```
<?php
require __DIR__ . '/app.php';
$rows = $db->query('SELECT id, customer, quantity, unit_price, total
                    FROM requests ORDER BY id')->fetchAll(PDO::FETCH_ASSOC);
foreach ($rows as $row) echo implode(' | ', $row) . PHP_EOL;
```

Open a second terminal in website-1. Run php inspect.php. A clean database after the Ada test should print:

```
1 | Ada | 2 | 3000 | 6000
```

The columns are request id, customer, quantity, saved unit price and total. Each successful submission adds another row. Refreshing thanks.php should not add a row because the handler redirected after the POST.

You can also open data/site.sqlite in DB Browser and inspect the requests table. Do not edit it while PHP is writing. Close DB Browser before changes if it holds a lock.

Stop and restart the local server. Run php inspect.php again. The saved row should remain. Session receipt data and persistent database records are different: the database is the lasting evidence.

TRY IT deliberately submit an invalid quantity with browser limits removed using Inspect. Confirm the server rejects it and the database row count stays unchanged.

Website 1 Explain the HTML CSS and JavaScript

Follow the visible side

Code and file	Why it is there
header.php and footer.php	Share the document shell, navigation and closing content on every page. require loads them.
lang, charset, viewport and title	Declare document language, encoding, device viewport and browser title.
header, nav, main, section, article and footer	Describe the roles of the content regions. h1 and h2 create a heading hierarchy.
a and href	Move between pages; ?item= passes an item id to preselect the request option.
label, for, id and name	Give controls readable labels and submitted field names. select and option list database products.
input type hidden and token value	Send the session form token back. Hidden fields are not secret or trusted by themselves.
min, max, step and required	Offer quick browser checks; the PHP handler also checks.
role alert and e($error)	Announce validation feedback and render it as escaped text.
style.css selectors and properties	Set readable typography, page width, spacing, form controls and focus outlines. The media rule changes the grid to three columns.

Read the catalogue filter

app.js first finds #search. if (search) prevents errors on pages without that field. querySelectorAll collects cards with data-name. An input listener normalises the search text, tests includes(), changes each hidden property and counts visible cards. The final comparison controls the empty-result message.

textContent is used for text in the separate language lessons; this catalogue already has escaped server-rendered text. dataset.name reads the data-name attribute. No database query is needed for this small browser filter.

Website 1 Explain PHP SQL and the request flow

Follow one submission

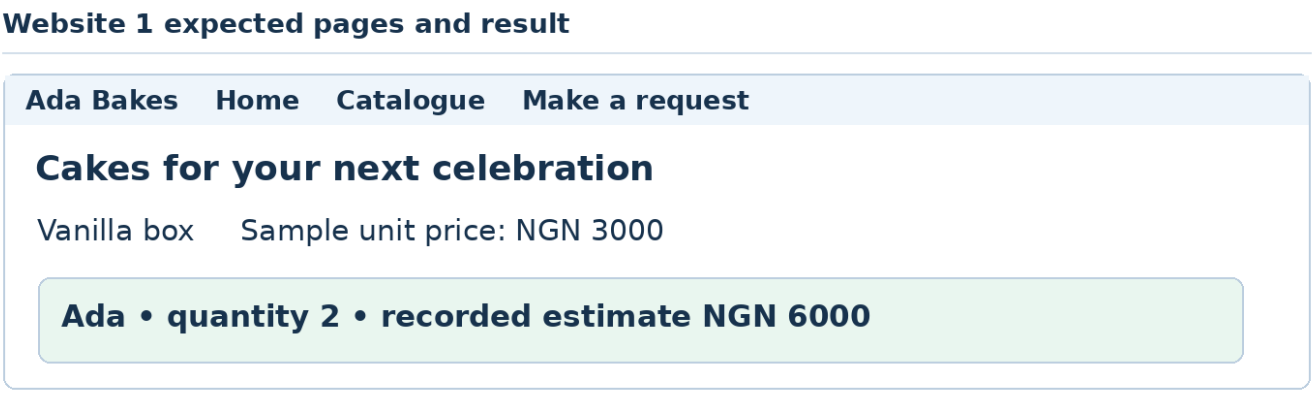
Code and file	Why it is there
config.php return array	Holds trusted site content, quantity limits and seed items. Each later website changes this file.
setup.php and schema.sql	Create a private database and tables. INSERT OR IGNORE seeds each id once; it does not update later config price changes.
PDO, attributes and PRAGMA	Connect to SQLite, turn SQL errors into exceptions and enable foreign keys.
app.php session_start and e	Start server-side session state and escape text for HTML using htmlspecialchars.
random_bytes, bin2hex and token	Generate an unpredictable form token. hash_equals compares the submitted token with the session token.
POST checks, is_string and trim	Require scalar text inputs and clean outer name whitespace. strlen limits bytes to 60 in this demo.
filter_var and config min/max	Validate a whole quantity on the server, including each website business limit.
prepare, ? and execute	Keep SQL separate from values. Fetch the selected item and its trusted database price.
INSERT and saved unit_price	Store an enquiry with a price snapshot; later price changes do not rewrite that quoted amount.
lastInsertId and receipt	Keep a short receipt for this session; no public customer list is created.
header with 303 and exit	Redirect after saving, then stop. Refreshing the result performs GET, not another insertion.

SELECT reads items. ORDER BY gives a predictable sequence. fetchAll produces arrays for foreach to render. PRIMARY KEY identifies a row; REFERENCES relates a request to its product. CHECK and NOT NULL add database constraints.

The database is outside public. The HTML escaping helper is for text/attribute contexts here; SQL safety comes from bound parameters, not HTML escaping.

Website 1 Expected output and practical checks

Use the site's Home, Catalogue, Request and Result pages in that order.



Teaching illustration • Simplified view

Try this	Expected result
Search chocolate	Only the matching catalogue option remains.
Search zzz	No matching option is shown.
First item with Ada and quantity 2	Recorded estimate: NGN 6000.
Quantity 0 or 101	Browser blocks it; server also rejects a bypassed invalid submission.
Run php inspect.php	A saved row shows the same quantity and total.
Refresh the Result page	No duplicate request is inserted.
Narrow the window and use Tab	Cards stack; links and form controls remain usable.

Explain the combined result

HTML provides the pages and labelled form. CSS lays them out. JavaScript filters cards. PHP validates the name, item and the 1–100 quantity range. SQL selects catalogue records and saves the checked request. The shared handler always reads the price from the database.

The code walkthrough after Website 1 explains every shared file. Use it while tracing this site; changing a config value does not replace the validation or database code.

Your demonstration: open one file from each language and explain its role. Then show the saved record. Sample bakery enquiry. No payment or delivery is arranged.

Website 2 A tailoring and branding enquiry website

Job: adapt the complete application to Thread Studio.

1. Stop the running PHP server with Ctrl + C.

2. Copy the source files from Website 1 into a new website-2 folder, including public. Start with an EMPTY data folder. Do not copy the earlier site.sqlite database.

3. Replace config.php with the complete code on the next page. Alternatively, open the supplied website-2 answer folder.

4. Open website-2 in VS Code. Run php setup.php, then php -S 127.0.0.1:8000 -t public.

5. Open http://127.0.0.1:8000/index.php. Check the name on all pages and click every navigation link.

This is a separate working website with its own database. Its shared source files are exactly those explained in Website 1. The change is its content and enforced business rules, not just a colour swap.

Website 2 expected pages and result



Teaching illustration • Simplified view

Student task: Set a new minimum of 20 in config.php. Confirm both the HTML field and server reject 10. Do not change only the browser field.

Keep all data fictional. Minimum 10 garments. This is an enquiry, not an accepted order.

Website 2 Configuration code

```
<?php
return [
    'name' => 'Thread Studio',
    'headline' => 'Branded clothing for your team',
    'intro' => 'Compare sample garment services and submit a bulk enquiry.',
    'quantityLabel' => 'Number of garments',
    'min' => 10,
    'max' => 500,
    'notice' => 'Minimum 10 garments. This is an enquiry, not an accepted order.',
    'items' => [
        [1, 'Polo branding', 2500],
        [2, 'Apron branding', 1800],
        [3, 'Cap embroidery', 1500],
    ]
];
```

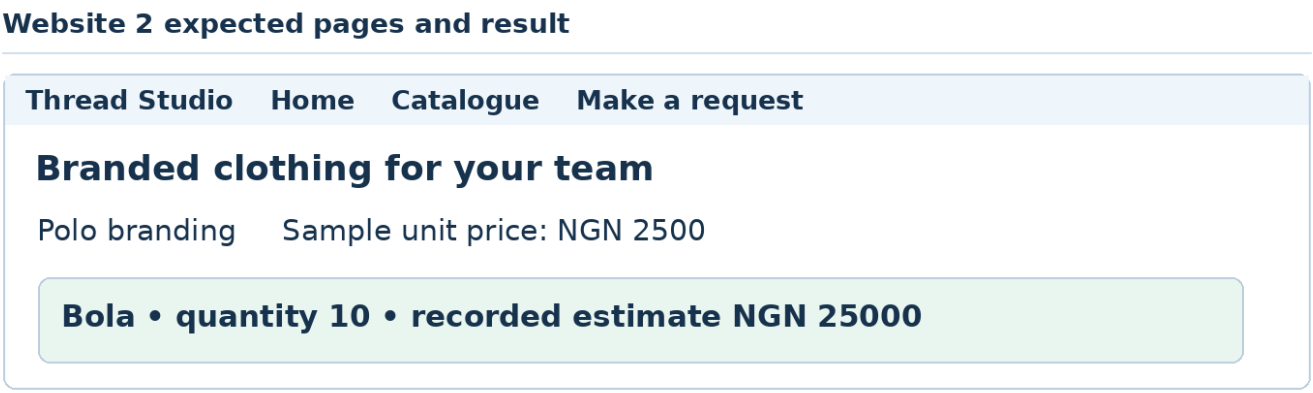
What each configuration value does

Key	Usage
name, headline and intro	Set site identity and Home content.
quantityLabel, min and max	Name the quantity and set browser plus server limits.
notice	Explain what a request does and does not confirm.
items	Seed rows as id, name and whole-number price. The shared SQL uses these values.
return and PHP array syntax	Return the settings to require; => maps each key to its value.

Use a fresh database for this exercise. setup.php deliberately does not overwrite existing rows or erase earlier requests.

Website 2 Expected output and practical checks

Use the site's Home, Catalogue, Request and Result pages in that order.



Teaching illustration • Simplified view

Try this	Expected result
Search cap	Only the matching catalogue option remains.
Search zzz	No matching option is shown.
First item with Bola and quantity 10	Recorded estimate: NGN 25000.
Quantity 9 or 501	Browser blocks it; server also rejects a bypassed invalid submission.
Run php inspect.php	A saved row shows the same quantity and total.
Refresh the Result page	No duplicate request is inserted.
Narrow the window and use Tab	Cards stack; links and form controls remain usable.

Explain the combined result

HTML provides the pages and labelled form. CSS lays them out. JavaScript filters cards. PHP validates the name, item and the 10–500 quantity range. SQL selects catalogue records and saves the checked request. The shared handler always reads the price from the database.

The code walkthrough after Website 1 explains every shared file. Use it while tracing this site; changing a config value does not replace the validation or database code.

Your demonstration: open one file from each language and explain its role. Then show the saved record. Minimum 10 garments. This is an enquiry, not an accepted order.

Website 3 A training centre course website

Job: adapt the complete application to SkillStart Academy.

1. Stop the running PHP server with Ctrl + C.

2. Copy the source files from Website 1 into a new website-3 folder, including public. Start with an EMPTY data folder. Do not copy the earlier site.sqlite database.

3. Replace config.php with the complete code on the next page. Alternatively, open the supplied website-3 answer folder.

4. Open website-3 in VS Code. Run php setup.php, then php -S 127.0.0.1:8000 -t public.

5. Open http://127.0.0.1:8000/index.php. Check the name on all pages and click every navigation link.

This is a separate working website with its own database. Its shared source files are exactly those explained in Website 1. The change is its content and enforced business rules, not just a colour swap.

Website 3 expected pages and result



Teaching illustration • Simplified view

Student task: Add a page describing course prerequisites and link it from the shared header. State what is required without claiming a place is reserved.

Keep all data fictional. One place per request. The request does not confirm admission.

Website 3 Configuration code

```
<?php
return [
    'name' => 'SkillStart Academy',
    'headline' => 'Practical weekend learning',
    'intro' => 'Browse sample workshops and request one learner place.',
    'quantityLabel' => 'Learner places',
    'min' => 1,
    'max' => 1,
    'notice' => 'One place per request. The request does not confirm admission.',
    'items' => [
        [1, 'Baking basics', 15000],
        [2, 'Bag sewing', 12000],
        [3, 'Web design basics', 18000],
    ]
];
```

What each configuration value does

Key	Usage
name, headline and intro	Set site identity and Home content.
quantityLabel, min and max	Name the quantity and set browser plus server limits.
notice	Explain what a request does and does not confirm.
items	Seed rows as id, name and whole-number price. The shared SQL uses these values.
return and PHP array syntax	Return the settings to require; => maps each key to its value.

Use a fresh database for this exercise. setup.php deliberately does not overwrite existing rows or erase earlier requests.

Website 3 Expected output and practical checks

Use the site's Home, Catalogue, Request and Result pages in that order.



Teaching illustration • Simplified view

Try this	Expected result
Search web	Only the matching catalogue option remains.
Search zzz	No matching option is shown.
First item with Chika and quantity 1	Recorded estimate: NGN 15000.
Quantity 0 or 2	Browser blocks it; server also rejects a bypassed invalid submission.
Run php inspect.php	A saved row shows the same quantity and total.
Refresh the Result page	No duplicate request is inserted.
Narrow the window and use Tab	Cards stack; links and form controls remain usable.

Explain the combined result

HTML provides the pages and labelled form. CSS lays them out. JavaScript filters cards. PHP validates the name, item and the 1–1 quantity range. SQL selects catalogue records and saves the checked request. The shared handler always reads the price from the database.

The code walkthrough after Website 1 explains every shared file. Use it while tracing this site; changing a config value does not replace the validation or database code.

Your demonstration: open one file from each language and explain its role. Then show the saved record. One place per request. The request does not confirm admission.

Website 4 A guesthouse room enquiry website

Job: adapt the complete application to Palm Rest Guesthouse.

1. Stop the running PHP server with Ctrl + C.

2. Copy the source files from Website 1 into a new website-4 folder, including public. Start with an EMPTY data folder. Do not copy the earlier site.sqlite database.

3. Replace config.php with the complete code on the next page. Alternatively, open the supplied website-4 answer folder.

4. Open website-4 in VS Code. Run php setup.php, then php -S 127.0.0.1:8000 -t public.

5. Open http://127.0.0.1:8000/index.php. Check the name on all pages and click every navigation link.

This is a separate working website with its own database. Its shared source files are exactly those explained in Website 1. The change is its content and enforced business rules, not just a colour swap.

Website 4 expected pages and result



Teaching illustration • Simplified view

Student task: Add a facilities section to Home. Do not label the result as a confirmed reservation: no date or availability system has been built.

Keep all data fictional. Dates and availability are not checked. This is a quote request only.

Website 4 Configuration code

```
<?php
return [
    'name' => 'Palm Rest Guesthouse',
    'headline' => 'Plan a comfortable short stay',
    'intro' => 'Compare sample nightly room rates and request a stay estimate.',
    'quantityLabel' => 'Number of nights',
    'min' => 1,
    'max' => 14,
    'notice' => 'Dates and availability are not checked. This is a quote request
only.',
    'items' => [
        [1, 'Standard room', 25000],
        [2, 'Family room', 40000],
        [3, 'Garden room', 30000],
    ]
];
```

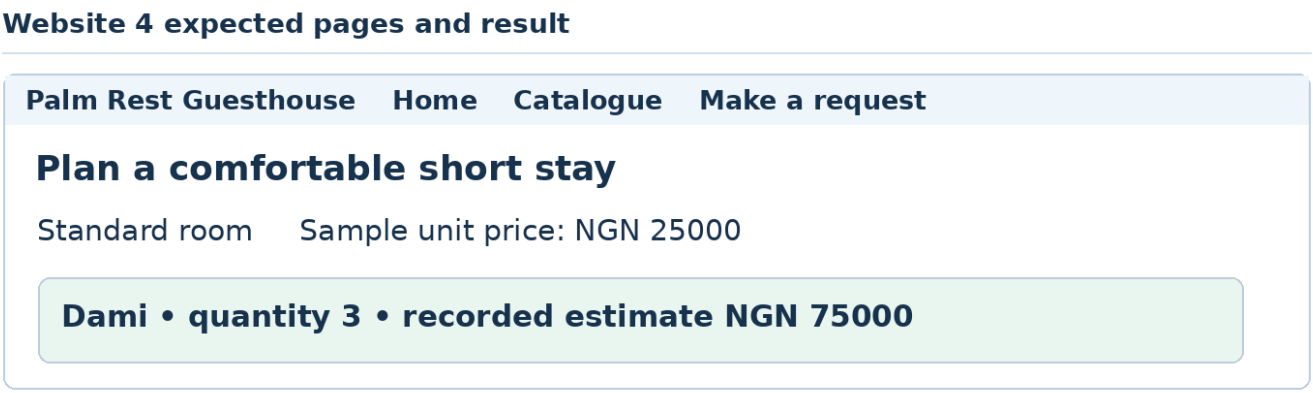
What each configuration value does

Key	Usage
name, headline and intro	Set site identity and Home content.
quantityLabel, min and max	Name the quantity and set browser plus server limits.
notice	Explain what a request does and does not confirm.
items	Seed rows as id, name and whole-number price. The shared SQL uses these values.
return and PHP array syntax	Return the settings to require; => maps each key to its value.

Use a fresh database for this exercise. setup.php deliberately does not overwrite existing rows or erase earlier requests.

Website 4 Expected output and practical checks

Use the site's Home, Catalogue, Request and Result pages in that order.



Teaching illustration • Simplified view

Try this	Expected result
Search family	Only the matching catalogue option remains.
Search zzz	No matching option is shown.
First item with Dami and quantity 3	Recorded estimate: NGN 75000.
Quantity 0 or 15	Browser blocks it; server also rejects a bypassed invalid submission.
Run php inspect.php	A saved row shows the same quantity and total.
Refresh the Result page	No duplicate request is inserted.
Narrow the window and use Tab	Cards stack; links and form controls remain usable.

Explain the combined result

HTML provides the pages and labelled form. CSS lays them out. JavaScript filters cards. PHP validates the name, item and the 1–14 quantity range. SQL selects catalogue records and saves the checked request. The shared handler always reads the price from the database.

The code walkthrough after Website 1 explains every shared file. Use it while tracing this site; changing a config value does not replace the validation or database code.

Your demonstration: open one file from each language and explain its role. Then show the saved record. Dates and availability are not checked. This is a quote request only.

Website 5 A community event registration website

Job: adapt the complete application to Community Learning Day.

1. Stop the running PHP server with Ctrl + C.

2. Copy the source files from Website 1 into a new website-5 folder, including public. Start with an EMPTY data folder. Do not copy the earlier site.sqlite database.

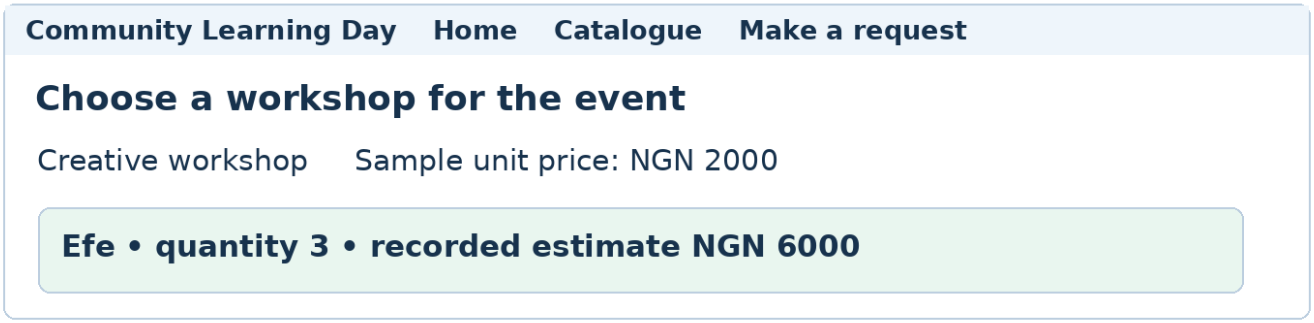
3. Replace config.php with the complete code on the next page. Alternatively, open the supplied website-5 answer folder.

4. Open website-5 in VS Code. Run php setup.php, then php -S 127.0.0.1:8000 -t public.

5. Open http://127.0.0.1:8000/index.php. Check the name on all pages and click every navigation link.

This is a separate working website with its own database. Its shared source files are exactly those explained in Website 1. The change is its content and enforced business rules, not just a colour swap.

Website 5 expected pages and result



Teaching illustration • Simplified view

Student task: Add an event schedule page using an HTML table. Link it from every page through header.php and test the links.

Keep all data fictional. No payment, seat allocation or email ticket is issued.

Website 5 Configuration code

```
<?php
return [
    'name' => 'Community Learning Day',
    'headline' => 'Choose a workshop for the event',
    'intro' => 'Explore event sessions and submit a sample ticket request.',
    'quantityLabel' => 'Tickets requested',
    'min' => 1,
    'max' => 6,
    'notice' => 'No payment, seat allocation or email ticket is issued.',
    'items' => [
        [1, 'Creative workshop', 2000],
        [2, 'Business talk', 1500],
        [3, 'Digital skills session', 2500],
    ]
];
```

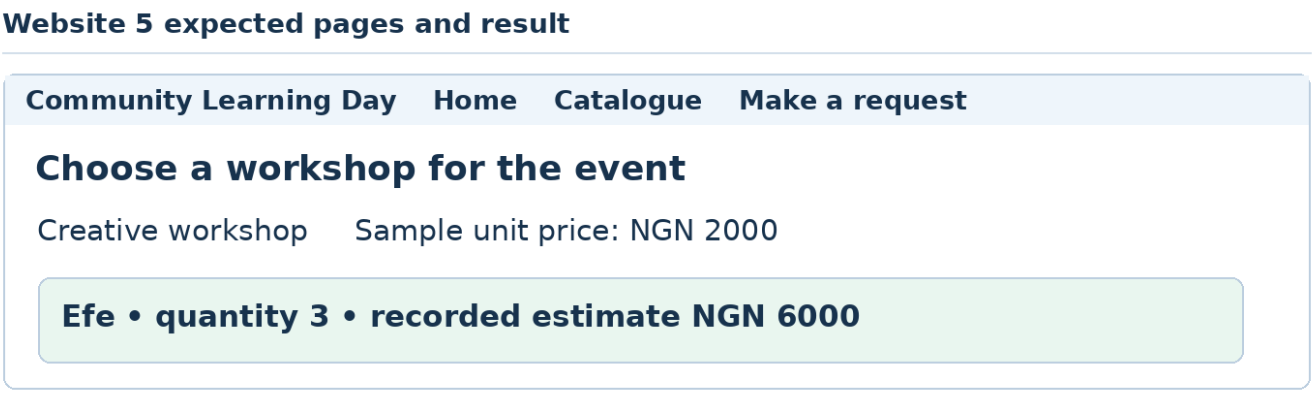
What each configuration value does

Key	Usage
name, headline and intro	Set site identity and Home content.
quantityLabel, min and max	Name the quantity and set browser plus server limits.
notice	Explain what a request does and does not confirm.
items	Seed rows as id, name and whole-number price. The shared SQL uses these values.
return and PHP array syntax	Return the settings to require; => maps each key to its value.

Use a fresh database for this exercise. setup.php deliberately does not overwrite existing rows or erase earlier requests.

Website 5 Expected output and practical checks

Use the site's Home, Catalogue, Request and Result pages in that order.



Teaching illustration • Simplified view

Try this	Expected result
Search digital	Only the matching catalogue option remains.
Search zzz	No matching option is shown.
First item with Efe and quantity 3	Recorded estimate: NGN 6000.
Quantity 0 or 7	Browser blocks it; server also rejects a bypassed invalid submission.
Run php inspect.php	A saved row shows the same quantity and total.
Refresh the Result page	No duplicate request is inserted.
Narrow the window and use Tab	Cards stack; links and form controls remain usable.

Explain the combined result

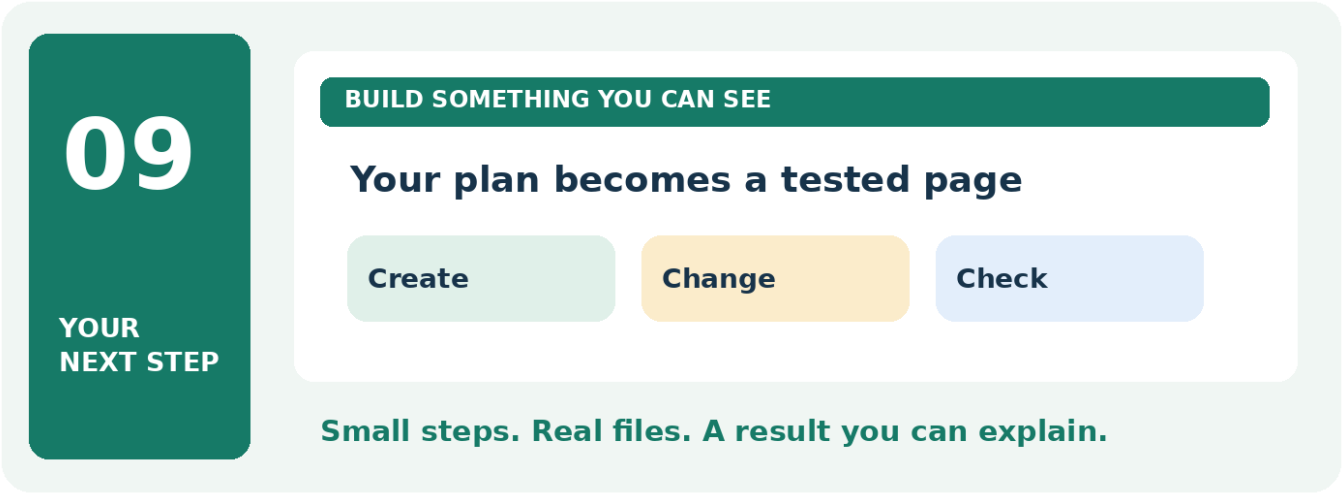
HTML provides the pages and labelled form. CSS lays them out. JavaScript filters cards. PHP validates the name, item and the 1–6 quantity range. SQL selects catalogue records and saves the checked request. The shared handler always reads the price from the database.

The code walkthrough after Website 1 explains every shared file. Use it while tracing this site; changing a config value does not replace the validation or database code.

Your demonstration: open one file from each language and explain its role. Then show the saved record. No payment, seat allocation or email ticket is issued.

Part 09 Build with AI and explore more tools

Use your foundation skills to guide, inspect and test AI-assisted work.



Your finish line

Try Codex, publishing platforms and frameworks; finish with a Qoder project.

TRY IT Take this part one practical at a time. Keep your previous work and change only the file named in the current step.

READY TO CONTINUE When you finish, write one sentence explaining what this subject does. Then show its result on your computer.

Prepare the next AI practice project

You have now built with the languages yourself. We will use AI to help improve a small familiar project.

1. Copy the ai-bakery folder from the practice files into Website Class.

2. In VS Code, choose File → Open Folder → ai-bakery.

3. Open index.html with Live Server. Read the page and click its message button.

4. Open styles.css and app.js. Identify the styles and click handler before asking an AI to change them.

5. Make a backup of ai-bakery. In the next lessons, “your bakery project” means this folder.

The five complete PHP websites remain separate. Do not upload their private database files to a static host.

The following lessons retain the earlier practical visits to Codex, WordPress, Blogger, Google Sites, Dreamweaver, ChatGPT Sites, Laravel, Django, Flutter, React and Qoder. They come after the foundation work, not before it.

Further lesson 1 Ask ChatGPT for help in ordinary English

Use: your browser and ChatGPT. Internet and an account may be required.

A prompt is simply the message you give an AI tool. You do not need special words. Describe the small thing you want help with.

Try your first request

1. Open a new browser tab with Ctrl + T.

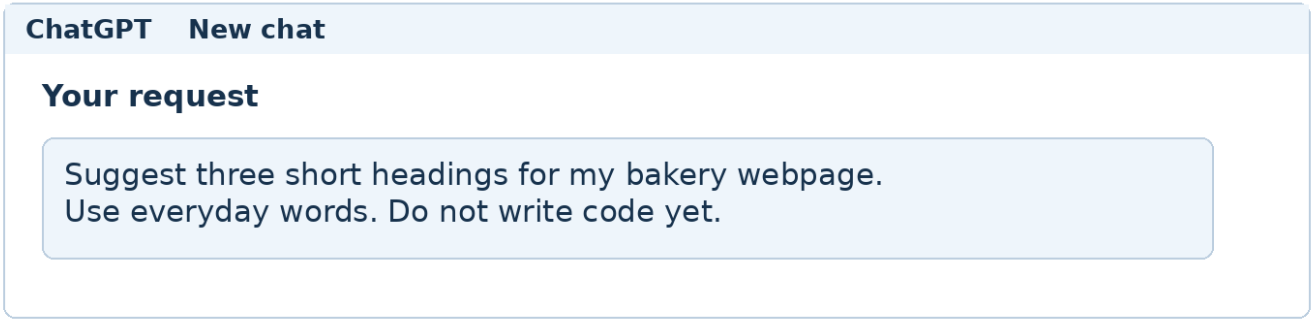
2. In the address bar, enter https://chatgpt.com. Sign in with your own approved account.

3. Start a new chat. Click the message box near the bottom.

4. Type the message below and send it using the arrow button.

“Help me improve my bakery website. Give me three short headings for a small bakery. Use simple English. Do not invent reviews or awards.”

Ask for help with one real task



Teaching illustration • Simplified view

Read the suggestions. Choose one you like. Return to VS Code and change the words inside your h1. Save and view the page.

You remain the designer: you choose what fits the business. The AI may suggest words that sound good but are not true.

If you copy code later, ask: “Which file does this go in? Am I adding it or replacing something?” Do not paste code until that is clear.

Your turn: ask for a one-sentence description of a product. Edit the answer so it sounds natural to you.

Further lesson 2 Install Codex in VS Code

Use: VS Code, internet and an account with access to Codex.

Codex can work beside your files. It can explain code and help change it. Account access and usage limits vary; check your account before starting.

Set it up

1. Open ai-bakery in VS Code.

2. Press Ctrl + Shift + X to open Extensions.

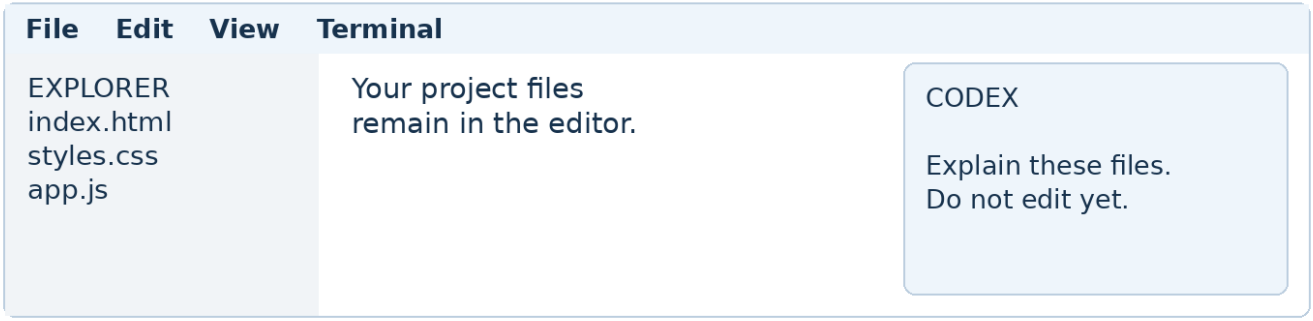
3. Search for Codex. Choose the official extension published by OpenAI. Use the official installation link in S6 if you are unsure which result is correct.

4. Click Install. Click the Codex icon when it appears.

5. If you cannot see it, press Ctrl + Shift + P. Type Codex: Open Codex Sidebar and select that command.

6. Follow the sign-in instructions. Choose Sign in with ChatGPT when offered and return to VS Code after completing the browser sign-in.

Open your project and then the Codex sidebar



Teaching illustration • Simplified view

Begin with a question

Type in the Codex panel:

“Look at index.html, styles.css and app.js. Explain what each file does in three short sentences. Do not change any files.”

You should receive an explanation before making an edit. If the tool cannot see the files, check that the correct folder is open.

Do not buy API credits for this lesson by accident. If a screen requires unexpected payment, cancel it and check your account plan. You can continue by asking ChatGPT to explain code and making edits yourself.

Source: S6.

Further lesson 3 Let Codex make one small improvement

Use: VS Code, Codex and the working bakery project.

First make another backup using the folder-copy steps in Setup lesson 4. A small change is easier to judge than an entire new website.

Ask for this change

In the Codex panel, type:

“Make the existing message button blue with white text and comfortable padding. Keep the words and button behaviour. Change only styles.css. Explain the change briefly.”

1. Read requests to edit files or run commands. If a command is unfamiliar, ask the tool to explain what it changes before you approve it.

2. When Codex finishes, open its changed-file view or compare styles.css with your backup.

3. Save if the editor shows unsaved changes.

4. Look at the browser. The button should look different.

5. Click it. The original thank-you message should still appear.

Read the change before accepting the result

Button style before

background: white;

Button style after

background: #164e9b;

Teaching illustration • Simplified view

If the result is wrong

Tell Codex exactly what you observed: “The colour changed, but clicking no longer shows the message. Restore the original click behaviour. Do not redesign the page.”

You should keep a change only after trying it yourself. “Done” in a chat does not prove that a button works.

Your turn: ask for one spacing improvement. Compare the changed file with the visible result.

Source: S6.

Further lesson 4 Save versions with GitHub Desktop

Use: GitHub Desktop, VS Code and your finished static files. Work in your own practice repository.

A saved version is a point you can return to. Git calls it a commit. GitHub Desktop lets you create these versions using buttons.

Make a practice repository

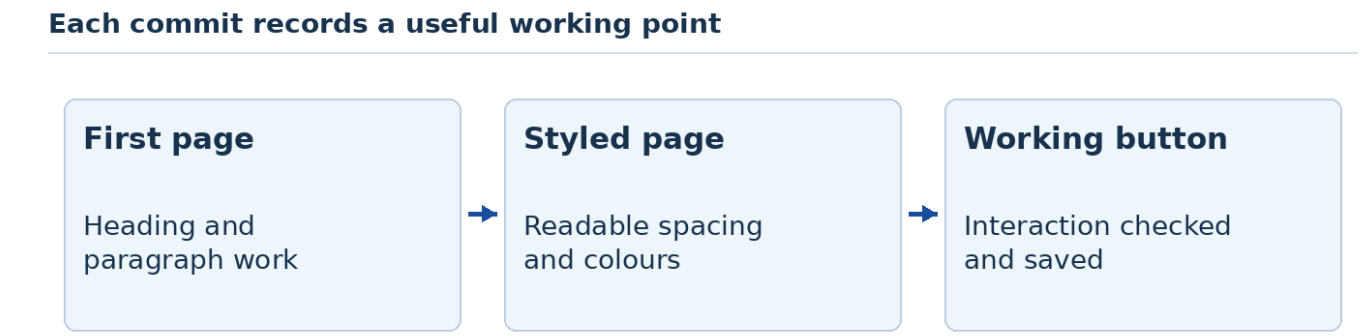
1. Download GitHub Desktop from https://desktop.github.com. Install it, open it and sign in to your GitHub account when prompted.

2. Choose File → New repository. Name it bakery-practice. Pick Documents as its location. Select the option to create a README, then create the repository.

3. In File Explorer, copy index.html, styles.css, app.js and images from ai-bakery into the new bakery-practice folder. Do not copy your backup folder.

4. In GitHub Desktop, look at Changes. Click a filename to see its changes.

5. In Summary, type Add my bakery website. Click Commit to the displayed branch name.



Teaching illustration • Simplified view

You have saved a local version. It is not a public website. Publishing a repository online is a separate action.

Open bakery-practice in VS Code. Change a heading and save. GitHub Desktop should show that change. Commit it with a clear summary.

In your own practice repository, choose History → right-click a practice commit → Revert Changes in Commit where available. This creates a new commit that reverses that change. Do not experiment with undoing shared work until you understand the effect.

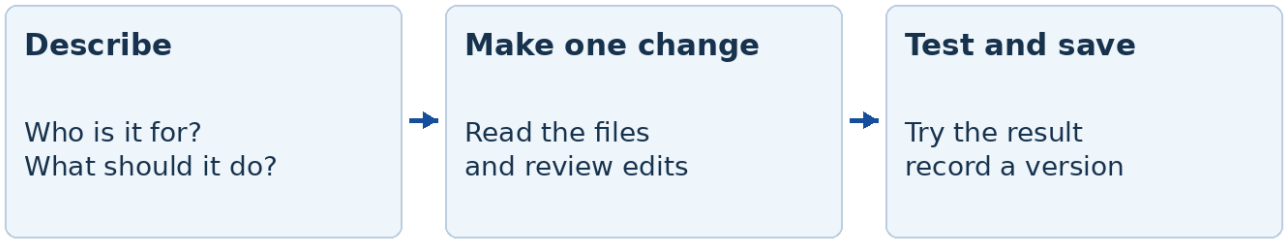
Source: S7.

Further lesson 5 Work like a careful website designer

Use: VS Code, your browser and either Codex or Qoder later.

A useful workflow makes the work clearer. You do not need ten AI tools open at once.

A small repeatable working routine



Teaching illustration • Simplified view

Make a brief before asking AI to build

Create a file called brief.txt in your project. Type:

```
Business: Ada Bakes, a fictional bakery.
Visitor: someone choosing a celebration cake.
Pages: Home, Cakes and Contact.
Style: white background, blue details, easy reading.
Main action: read the contact information.
No payments, accounts or real order submission yet.
```

Build in small rounds

1. Ask AI for a short plan based on brief.txt. Read it.

2. Build only the Home page. Open it and check it.

3. Save a version. Add Cakes next. Test its link.

4. Add Contact. Test all three pages.

5. Open the page in a fresh browser tab. Can you find a cake and the contact information without reading the code? If possible, ask someone else to try too.

A professional habit: write down what should happen before testing. For example, “Clicking Cakes opens the Cakes page.”

If two AI tools work on the same folder, use one at a time. Finish, review and save before switching. Otherwise, one tool may overwrite the other's work.

Your turn: change the brief for a fictional barber, school club or clothing shop.

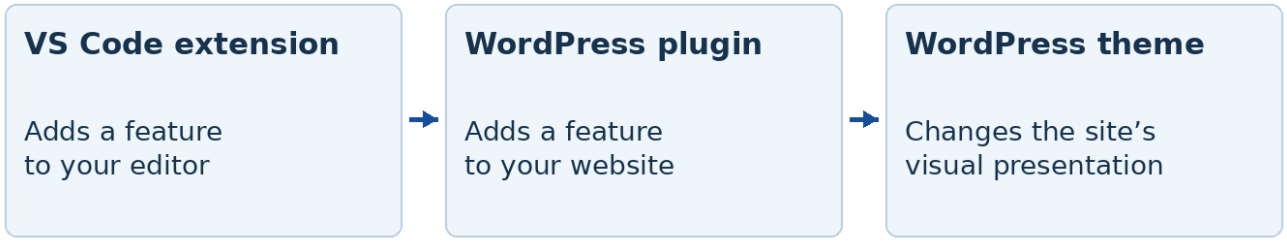
Further lesson 6 Choose the right kind of website tool

Use: this comparison to choose your next exercise.

You have built with files. Some tools let you arrange content with buttons and blocks. They are useful too.

Tool	What we will use it for
Notepad and VS Code	Understand and edit our website files
WordPress.com	Make a hosted website without arranging a server ourselves
WordPress.org software	Run WordPress with our own local setup or chosen hosting
Blogger	Publish short articles and updates
Google Sites	Make a simple information site using visual controls
Dreamweaver	Briefly compare code and visual editing
Codex and Qoder	Help plan, edit and check our code

Different tools change different things



Teaching illustration • Simplified view

WordPress.com and WordPress.org are not two spellings of the same service. WordPress.org provides the software. WordPress.com provides a hosted service built around WordPress.

A theme changes a WordPress site's design. A plugin adds a feature. A VS Code extension adds a feature to the editor. These are different kinds of add-on.

Your turn: choose a tool for a class news blog. Then choose a tool for a custom student-results application. Explain why they may need different tools.

Suggested answers: Blogger or WordPress can suit a news blog. A custom application may suit Laravel or Django, with proper access controls.

Source: S8.

Further lesson 7 Make a page on WordPress com

Use: your browser, internet and an approved WordPress.com account.

Create a classroom site

1. Go to https://wordpress.com. Choose the option to start a site and follow account setup.

2. Use a fictional site name such as Ada Bakes Practice. Choose a free site address and free option where offered. If the flow requires unexpected payment, go back and check the free option. Skip paid features for this exercise.

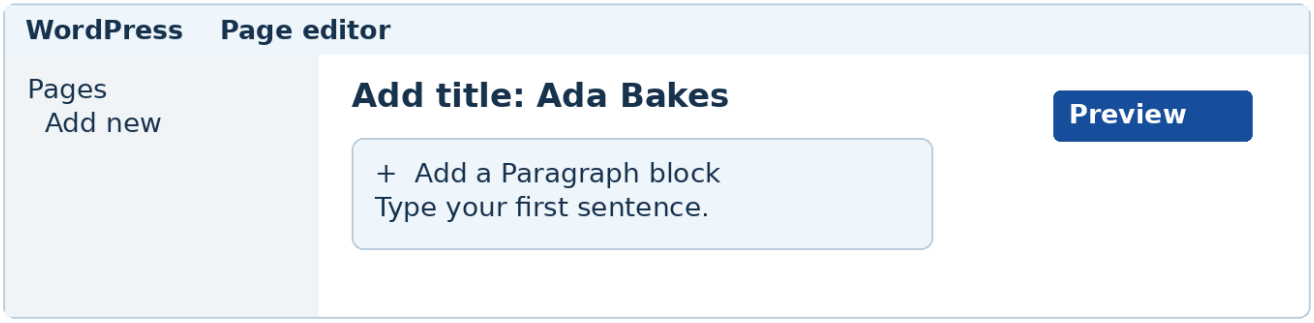
3. Open the site's dashboard. Find Pages → Add new page.

4. Enter About Ada Bakes as the title. In the writing area, type two short sentences.

5. Click + to add a block. Choose Heading and type Our cakes. Add a Paragraph block below it.

6. Use Preview to inspect the page. Save as a draft. Before publishing, check the words, images and any personal details.

Build a WordPress page from blocks



Teaching illustration • Simplified view

A block is one piece of content, such as a heading, paragraph or image. Click a block to change it.

To explore design: dashboard → Appearance → Themes. Preview a theme before activating it. With a compatible block theme, Appearance → Editor opens the site-wide design editor.

Your turn: add a Contact page containing fictional details. Check it on a narrow preview.

Publishing a page and launching the whole site can be separate steps. Confirm the site's visibility before sharing a link. Installing plugins on WordPress.com requires an eligible paid plan; this brief lab does not need them.

Sources: S9–S11.

Further lesson 8 Try WordPress software on your computer

Use: WordPress Studio for a local WordPress site.

This is our practical introduction to the WordPress.org route. WordPress Studio runs the WordPress software locally. You do not have to buy hosting to do this lesson.

Create a local site

1. Visit https://wordpress.com/studio or follow the download link in S12.

2. Download the Windows installer, open it and complete setup. Compare the stated requirements with Settings → System → About first.

3. Open Studio. Name the first site Ada Bakes Local and click Continue. If Studio already has sites, use Add site.

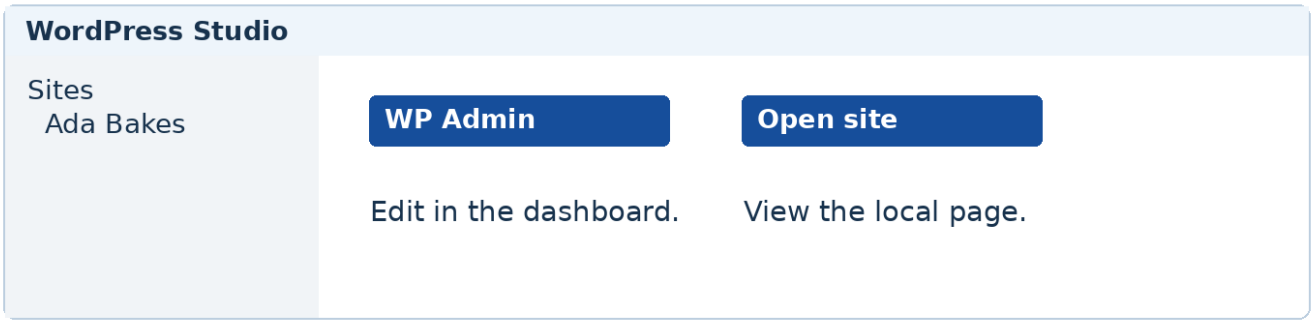
4. Select the site. Start it if it is stopped.

5. Choose WP Admin to open its WordPress dashboard.

6. Open Pages → Add New. Create About Us, add two sentences, then publish it locally.

7. Use Open site from Studio to see the result. A new page may need to be added to navigation separately.

Studio runs a WordPress site on your computer



Teaching illustration • Simplified view

Local means running on this computer. Publishing inside this local WordPress installation does not automatically publish a public internet site.

If installation is unavailable, open https://playground.wordpress.net in your browser. It is a useful WordPress sandbox. Use its save or export options before closing; do not assume a temporary session keeps your work forever.

Your turn: change the page heading and view the updated local site.

Sources: S12–S14.

Further lesson 9 Change a WordPress theme and try a plugin

Use: the local WordPress site from Further lesson 8. Do this locally, not on a client's live site.

Change the design

1. Open WP Admin from Studio.

2. Click Appearance → Themes. Look at the installed themes.

3. Preview a theme, then Activate it. Open the site and compare the result.

4. If you need another theme, use Add New Theme and the official directory. Choose a simple block theme from the official directory and preview it before activating it.

Try one small plugin

1. Open Plugins → Add New Plugin.

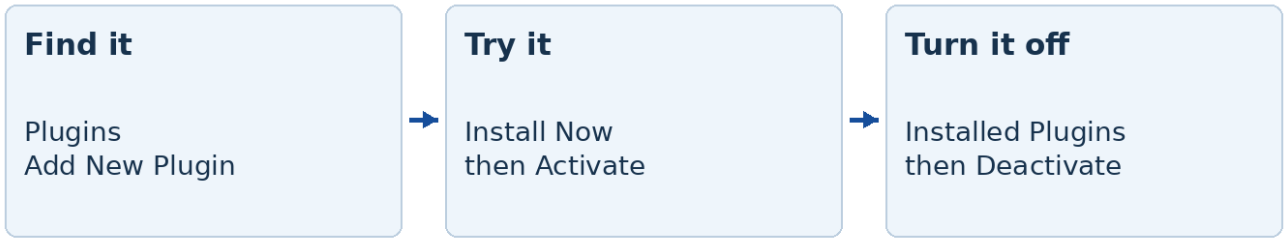
2. Search for Hello Dolly, the demonstration plugin by Matt Mullenweg.

3. If available, Install Now, then Activate. It displays a short lyric in the administration area; it does not create a website page.

4. Look for that change. Then Deactivate the plugin and compare.

5. Leave it deactivated or delete it after the demonstration.

Try a plugin locally and learn how to remove it



Teaching illustration • Simplified view

The lesson is how installation, activation and deactivation work. A real site's plugins should solve a real need. Check maintenance, compatibility and the author before adding them.

Before changing more, open Studio's Import / Export area and export your local site using the offered export option. Save the archive in Documents → Website Class → Backups. Create Backups if it does not exist. A complete site backup differs from WordPress Tools → Export, which mainly exports content.

Your turn: explain the difference between a theme and a plugin.

Sources: S13, S15, S16.

Further lesson 10 Publish a short article with Blogger

Use: your browser and an approved Google account.

A blog is useful for articles, news and regular updates.

Create a practice blog

1. Visit https://www.blogger.com and sign in.

2. For an existing dashboard, open the blog selector at the top left and choose New blog. First-time users may see a direct Create your blog option.

3. Enter a title such as My Website Class. Continue and choose an available blog address.

4. In the dashboard, click New Post.

5. Enter My first website lesson as the post title. Write three sentences about what you made.

6. Click Preview. Correct any mistakes. Save as a draft. Publish only after previewing and checking that the content is yours to share.

A blog is made from individual posts



Teaching illustration • Simplified view

To change the appearance, open Theme in the left menu. Preview a theme before applying it. Layout controls where supported blog items appear.

A post is an article. A page is better for information that stays useful, such as About Me. Use Pages → New Page for that kind of content.

Your turn: make an About page with only information you are comfortable sharing. Do not publish a child's private details or class contact list.

READY TO CONTINUE open the published link in a signed-out browser if public viewing is intended.

Source: S17.

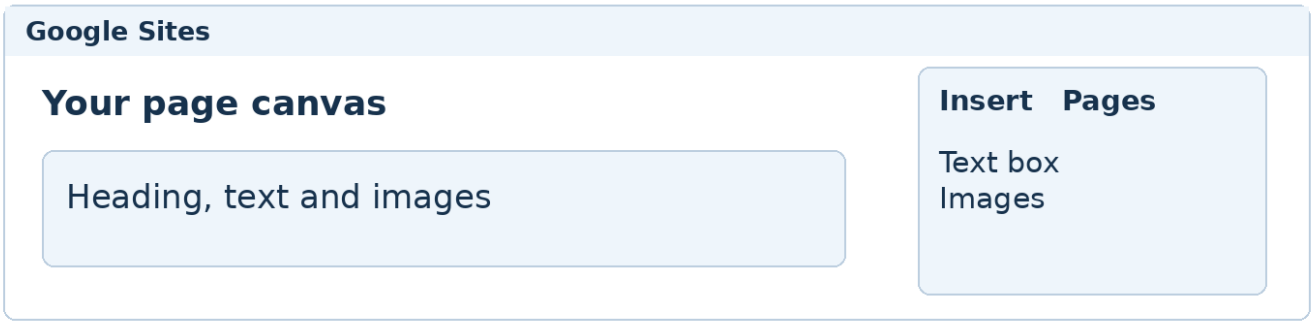
Further lesson 11 Build a simple site with Google Sites

Use: your browser and an approved Google account.

Make a class information site

1. Go to https://sites.google.com. Choose Blank under Start a new site.
2. Enter Website Class as the site document name. Click the page title and type Welcome to our class.
3. On the right, choose Insert → Text box. Type a welcome sentence.
4. Choose Insert → Images and add an image you created or have permission to use.
5. Open Pages on the right. Click + and create an About page.
6. Use the Preview button near the top. Try the phone and computer preview sizes.
7. When ready, click Publish. Choose an available address and check who can view the published site before confirming.

Google Sites combines page content visually



Teaching illustration • Simplified view

Google Sites saves editing work as you go, but later changes still need publishing before visitors see the updated public version.

If an account cannot create or publicly share a site, it may be restricted by a school, organisation or account type. Read the restriction message. Use an account you own only if the service permits it; otherwise continue with a local project.

Your turn: add a Learning Goals page with three goals. Open the published site and test its navigation.

This is Google Sites. The ChatGPT Sites demonstration later is a different product.

Source: S18.

Further lesson 12 Briefly try Dreamweaver and a design tool

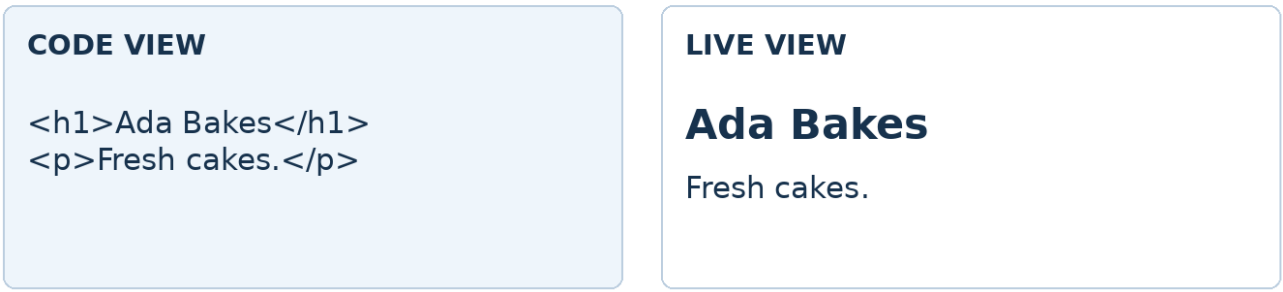
Use: Dreamweaver only if you already have licensed access. This optional comparison is not required to finish the course.

Compare another editor

1. In Dreamweaver, choose File → New.
2. Select an HTML document and click Create.
3. In Code view, place <h1>Hello from our class</h1> between the body tags.
4. Choose File → Save as. Save the document as dreamweaver-practice.html in a separate practice folder.
5. Switch to Split or Live view using the document view controls. Compare the code with its visible result.

You should recognise the HTML even though the editor looks different. An editor helps you work; the HTML file still matters.

Dreamweaver can show code and a visual page



Teaching illustration • Simplified view

Sketch before building

Paper is enough. Draw a rectangle for a phone screen. Add a business name, a heading, three products and a contact button.

For a digital alternative, you can open Figma or Penpot, create a design file and draw the same arrangement with rectangles and text. These tools help plan a design; a drawing is not automatically a working website.

Your turn: point to the first thing you want a visitor to notice. Make it clearer without making every item large.

Sources: S19, S20.

Further lesson 13 Try building with ChatGPT Sites

Use: your account if ChatGPT Sites is available. Skip this optional exercise if you cannot see it.

This is separate from Google Sites. It allows an AI-assisted site-building workflow inside ChatGPT.

Watch one small build

1. Open ChatGPT and start a new conversation with Sites available in that workspace.

2. Send this request:

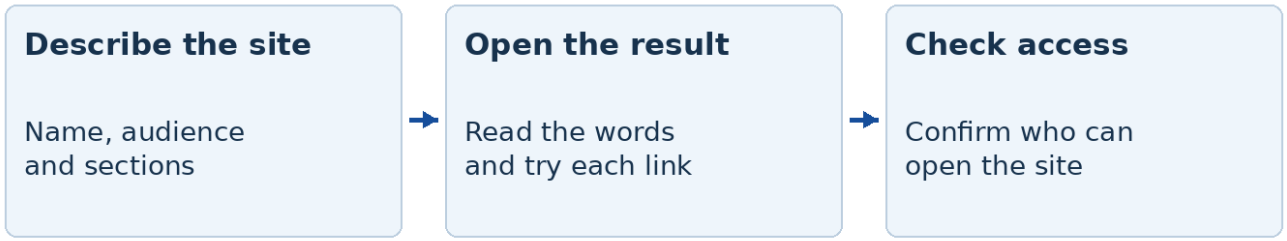
“Use Sites to build a simple fictional bakery website called Ada Bakes. Use a white background, blue buttons and three cake cards. Add Home, Cakes and Contact sections. Keep it private for review. Do not add payments or collect customer information.”

3. Open the resulting site preview or link. Read the actual text and try every link.

4. Ask for one change: “Make the cake descriptions easier to read on a phone. Keep everything else.”

5. View the revised result. Check the site's actual sharing setting before sending its link to anyone.

Look at the generated site before sharing it



Teaching illustration • Simplified view

Some generated sites may already have a hosted link while remaining private. A preview, a hosted site and a publicly accessible site are not automatically the same thing.

Discuss: what was quicker than hand-coding? What still needed a human decision? Look for wrong content, broken actions and small-screen problems.

Source: S21. Availability depends on the account and workspace.

Further lesson 14 What is a stack

Use: paper and a pencil. No new installation yet.

A stack is a group of tools used together to make a working website. You choose it for the job you need to do.

A website can have a visible side and a server side



Teaching illustration • Simplified view

The front end is what a visitor sees and uses. The back end handles work on a server, such as checking a sign-in. A database stores organised information, such as orders.

Tool or stack	A useful starting project
HTML, CSS and JavaScript	A small business information website
Laravel with PHP	A website that saves enquiries or bookings
Django with Python	A data-driven website with an administration area
React with JavaScript	An interactive interface with changing screens
Flutter with Dart	An app-style interface for web and other platforms

VS Code and Qoder are editors. They can help you work on different stacks. They are not replacements for PHP, Python or Dart.

Choose the job first

Circle one: a bakery brochure, a booking system or an interactive learning app. Write which tools you would start with and one reason for your choice.

For a simple brochure, our existing HTML website is enough. For ordinary text-heavy websites, HTML or a web framework is usually a better first choice than Flutter. Flutter is useful when you want an app experience.

Sources: S22–S27.

Further lesson 15 Meet the terminal slowly

Use: VS Code. Read a command first, type it once, then compare the result.

The terminal is a place to give the computer written instructions. Each instruction is called a command.

1. Open VS Code. Choose File → Open Folder and open Website Class.

2. Choose Terminal → New Terminal. A panel opens at the bottom.

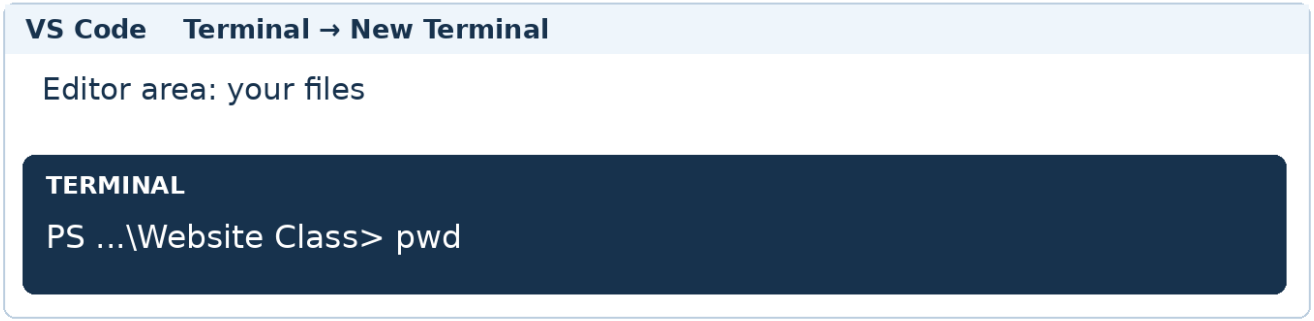
3. Click beside the blinking cursor in that panel.

4. Type the command below and press Enter once.

```
pwd
```

In Windows PowerShell, this shows your current folder. If your terminal is Command Prompt instead, use cd with no extra words.

Commands go in the bottom terminal panel



Teaching illustration • Simplified view

Three useful habits

Type one command at a time. Wait for it to finish before typing the next one. Commands go in the terminal, not inside index.html.

When a command starts a local website, it may keep running. Leave that terminal open. To stop the website, click inside that terminal and press Ctrl + C.

An address such as http://127.0.0.1:8000 points to a service on your own computer. It is not a public link you can send to a customer.

Your turn: close the terminal panel using its panel controls. Open a new terminal again. Find the folder name it shows.

Further lesson 16 Start a Laravel project

Use: VS Code and the working Laravel Herd installation from the PHP section.

PHP runs Laravel. Composer downloads the PHP packages it needs. Herd supplies these tools. Use supported versions of both tools; this exercise targets Laravel 13.

1. Visit herd.laravel.com. Choose the Windows download. Follow the same installation and verification steps used in the PHP section.

2. Restart VS Code after installation. Open Website Class, then Terminal → New Terminal.

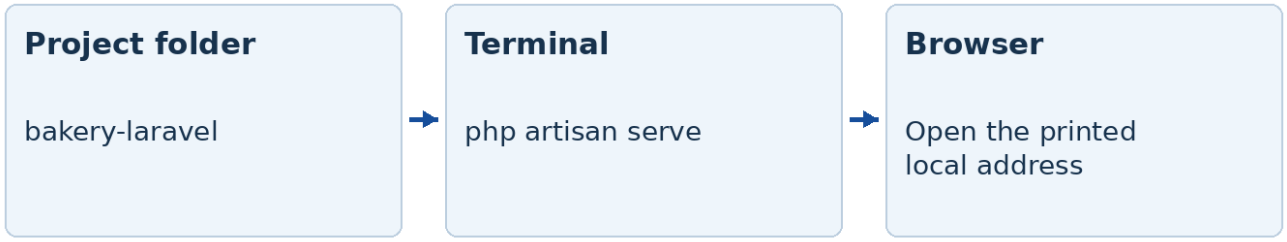
3. Run each command separately. The first two should show version numbers. If either fails, close and reopen VS Code, check that Herd opens, then repeat only the failed command.

```
php --version
composer --version
composer create-project laravel/laravel bakery-laravel "^13.0"
cd bakery-laravel
php artisan serve
```

4. Wait while packages download. Open the local address printed after the final command. You should see Laravel's starting page.

5. In VS Code, choose File → Open Folder and open the new bakery-laravel folder. If the running terminal closes, open a terminal here and run php artisan serve again.

Start the local server and use the address it prints



Teaching illustration • Simplified view

This first page does not need a paid Herd service. We will use plain HTML in a Laravel template, so we do not need a separate front-end build process for this exercise.

Success: the page opens while the local server is running. If the address fails, check the terminal first.

Sources: S22, S28. Check the current PHP requirement in Laravel's installation guide before creating the project.

Further lesson 17 Give Laravel your own page

Use: the bakery-laravel folder in VS Code.

A route tells Laravel what to show for an address. A view is the file containing the page you want to show.

1. In the Explorer, open resources, then views. Create a file named home.blade.php inside views.

2. Put this small page in the file. Save it.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Ada Bakes</title>
</head>
<body>
  <h1>Ada Bakes</h1>
  <p>Our Laravel website is working.</p>
</body>
</html>
```

3. Open routes/web.php. Replace its contents with this and save:

```
<?php
use Illuminate\Support\Facades\Route;
Route::get('/', function () {
    return view('home');
});
```

4. Refresh the local Laravel page. You should see Ada Bakes.

Now ask Codex: “Style resources/views/home.blade.php with simple CSS inside a style tag. Use a blue heading and a readable centred layout. Keep the route and packages unchanged.” Review the change, refresh and narrow the browser.

Source: S22. A blank page or error means stop and read the terminal before adding more features.

Further lesson 18 Let Laravel save an enquiry

Use: your working Laravel project and Codex. This is a supervised extension.

Until now, our pages have mainly displayed information. This task adds storage. Use invented details only.

Build one small feature

1. Save a Git checkpoint or copy the project folder before editing.

2. Open the Codex panel. Ask it to inspect the project and propose a short plan using this request:

“Add an enquiry form to this Laravel 13 project. Use SQLite. Fields: name, email and message. Validate them on the server. Use Laravel CSRF protection. Save valid enquiries in the database and show a clear success message. Do not add payments, email sending or a public list of enquiries. Explain the files and commands before changing them.”

3. Read the plan. Compare the proposed files with your requirements before allowing changes. Ask Codex to explain any database migration command before running it. A migration sets up or changes database tables.

4. Run the suggested tests. Then test the form yourself using the table below.

Try this	You should see
Submit an empty form	Helpful messages; nothing saved
Enter an invalid email	A useful email error
Submit fictional valid details	A success message and one saved record
Stop and restart the server	The record still exists in SQLite

Open the project database/database.sqlite in DB Browser, choose Browse Data, then choose the table named in your migration. Submit fictional data and confirm that its row exists. A success message alone does not prove that anything was saved.

Keep this exercise local. A real public enquiry service also needs spam controls, secure hosting, access controls, backups and a privacy review.

Sources: S22, S29.

Further lesson 19 Start a Django project

Use: VS Code and Python 3.12 in a separate project folder.

Django is a web framework written in Python. We will use the supported Django 5.2 series for this classroom example.

1. Open https://www.python.org/downloads/windows/ and locate Python 3.12. Download the Windows installer matching your system. Run it and include the Python launcher if offered. Reopen VS Code and run py -3.12 --version. Expect Python 3.12.x.

2. Inside Website Class, make a new empty folder named bakery-django. Open that folder in VS Code.

3. Choose Terminal → New Terminal. Run these commands one at a time:

```
py -3.12 -m venv .venv
.venv\Scripts\python -m pip install "Django>=5.2,<5.3"
.venv\Scripts\python -m django startproject bakery .
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py runserver
```

4. Open the local address printed in the terminal. You should see Django's welcome page.

Keep this Django project together in one folder

VS Code bakery-django

```
bakery-django
  .venv
  bakery
  manage.py
```

.venv holds Python tools.

bakery holds project settings.

manage.py runs project tasks.

Teaching illustration • Simplified view

The .venv folder keeps this project's Python tools together. Using its Python path means we do not need a separate activation step here. The dot at the end of startproject bakery . means “put the project in this folder.” Do not leave it out.

If py is not recognised, reopen the official installer, choose Modify and check that the launcher is included, then reopen VS Code. Do not keep pasting the remaining commands after an error.

Sources: S23, S30.

Further lesson 20 Give Django your own page

Use: bakery-django in VS Code. Keep the server running.

1. Inside the inner bakery folder, create views.py. This is beside settings.py and urls.py. Add:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse(
        "<h1>Ada Bakes</h1>"
        "<p>Our Django page is working.</p>"
    )
```

2. Open bakery/urls.py. Replace its contents with:

```
from django.contrib import admin
from django.urls import path
from .views import home

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", home),
]
```

3. Save both files and refresh the local page. You should see your heading.

Ask AI for the next small step

“Move my Django home page into a proper HTML template. Add a page title, viewport setting and simple readable CSS. Keep the home address working. Explain where each new file goes. Do not add packages.”

Then request a Cake model with name and price, registered in Django admin. Read the generated migration, then run .venv\Scripts\python manage.py makemigrations followed by .venv\Scripts\python manage.py migrate. Run .venv\Scripts\python manage.py createsuperuser and answer its prompts. Open /admin/ on the local server, sign in, add a cake and confirm it appears.

READY TO CONTINUE sign in at /admin/, add one fictional cake and confirm it remains after restarting. Never paste the administrator password into AI chat.

Sources: S23 and the Django admin reference on the last source page. The template and admin extensions are guided AI exercises, not required before your first Django success.

Further lesson 21 Start a Flutter web project

Use: VS Code, Git, Chrome and the Flutter extension. Allow setup time before class.

Flutter uses the Dart language. Today we are making a small web app, not a phone installation.

1. Install Git from https://git-scm.com/downloads/win and Chrome from https://www.google.com/chrome/. Use the VS Code installation you already prepared.

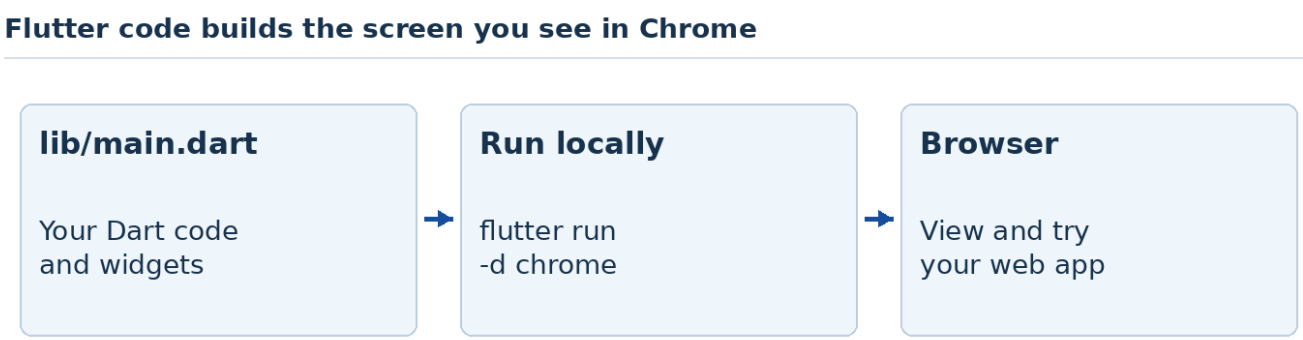
2. In VS Code, open Extensions. Search Flutter. Install the official extension published by Dart Code.

3. Press Ctrl + Shift + P. Type Flutter: New Project and select it. If asked for the Flutter SDK, choose Download SDK. Choose a simple folder, such as C:\dev, and follow Clone Flutter and Add SDK to PATH when offered.

4. Restart VS Code. Run flutter doctor -v. Read each message and follow the linked instructions for Flutter and Chrome. Missing Android tools can be ignored for this web-only exercise.

5. Open Website Class in VS Code. Run these commands separately:

```
flutter create bakery_flutter
cd bakery_flutter
flutter run -d chrome
```



Teaching illustration • Simplified view

You should see Flutter's sample app in Chrome. Keep the terminal open. If Chrome is unavailable on Windows, check Edge support and use flutter run -d edge.

Source: S24. If the SDK download is interrupted, resume setup when the connection is stable. Keep practising HTML and CSS while it downloads.

Further lesson 22 Change the Flutter screen

Use: the bakery_flutter folder in VS Code.

A widget is a piece of a Flutter screen. Text is a widget. A button is a widget. We put widgets together to make a page.

1. Open lib/main.dart. Replace its contents with the code below.

2. Save. In the running Flutter terminal, press r to reload. If no server is running, use flutter run -d chrome again.

```
import 'package:flutter/material.dart';

void main() => runApp(const BakeryApp());

class BakeryApp extends StatelessWidget {
  const BakeryApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Ada Bakes')),
        body: const Center(
          child: Text('Our Flutter web app works!'),
        ),
      ),
    );
  }
}
```

3. You should see Ada Bakes and your message. Change the message and reload once more.

Ask Codex: “Add a simple cake quantity selector with plus and minus buttons. Do not allow a negative quantity. Keep all changes in lib/main.dart and add no packages. Explain the state that remembers the number.”

READY TO CONTINUE start at zero, press plus three times, then press minus four times. The number should finish at zero. This is local screen state; it is not a saved customer order.

Sources: S24, S25.

Further lesson 23 A short visit to React

Use: VS Code and a current Node.js LTS release supported by Vite.

React helps build interactive interfaces from reusable pieces called components. Vite provides a development setup. Neither one automatically gives us a database or payment system.

1. Open https://nodejs.org, download the Windows LTS installer and follow setup. Restart VS Code. Run node --version and npm --version; both should print a number. Compare the Node version with https://vite.dev/guide/.

2. Open Website Class. In a new terminal, run each command:

```
npm create vite@latest bakery-react -- --template react
cd bakery-react
npm install
npm run dev
```

3. Open the address printed by Vite. Open bakery-react as the VS Code folder.

4. Replace src/App.jsx with this and save:

```
import { useState } from 'react';

export default function App() {
  const [count, setCount] = useState(0);
  return <main>
    <h1>Ada Bakes</h1>
    <button onClick={() => setCount(count + 1)}>
      Cakes selected: {count}
    </button>
  </main>;
}
```

Click the button three times. You should see 3. Refresh: it returns to 0 because we have not added storage.

Ask AI to add a Reset button. Review its changes and test again. After these demonstrations, choose one stack for your next project rather than trying to master all four at once.

Sources: S26, S27.

Further lesson 24 Meet Qoder IDE

Use: Qoder IDE in Editor mode. Sign in and check the plan shown in your account.

You already know the important habit: ask for a small change, inspect it and test it. We will use that same habit in Qoder.

1. Visit qoder.com. Download Qoder IDE for Windows from its official download page. Run the installer and sign in with an account you are allowed to use.

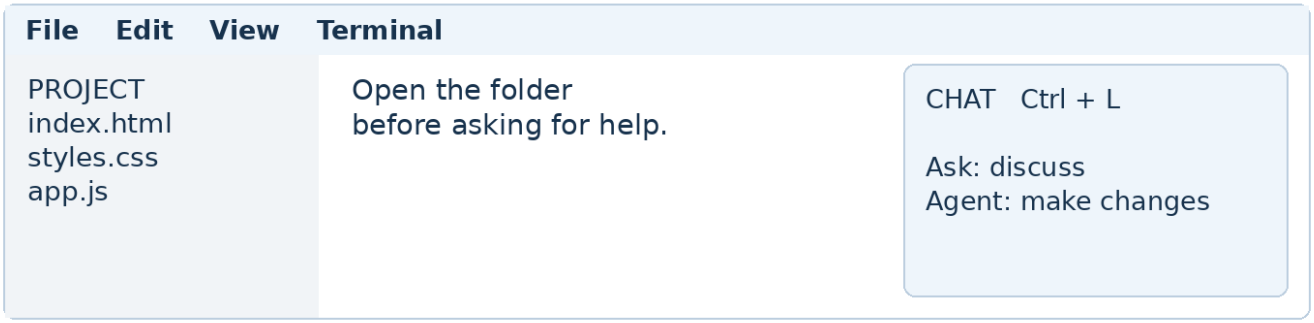
2. Open your existing bakery project folder using the welcome screen's open-project control or File → Open Folder. Confirm that index.html appears in the file list.

3. Stay in the Editor. Press Ctrl + L to open Chat. Select Ask mode in the chat controls.

4. Ask: “Explain the files in this small website in everyday English. Do not change anything.”

5. Read the answer and point to the actual files it describes.

Qoder Editor chat supports asking and making changes



Teaching illustration • Simplified view

Ask mode is useful for discussion. Agent mode can make changes across files. Before allowing changes, make a backup or Git checkpoint. Review requested commands and the changed files.

Qoder has a free plan, but it is not an unlimited promise. Agent access, credits and trial offers depend on the current plan. Check the account's plan and remaining allowance in your account settings. If access is limited, continue the same exercise with your available Codex access or manual edits.

Sources: S31, S32. Teach Editor rather than the retiring Quest window; see the tool notes.

Further lesson 25 Your final Qoder website

Use: Qoder Editor and a fresh folder named ada-final inside Website Class.

We will make a complete small website in three rounds. Do not ask AI to build every possible feature in one message.

Round one Plan

Create brief.txt in the folder. Write the business name, visitors, three sections and one main action. Example: “Ada Bakes. Local customers. Home, Cakes, Contact. Main action: read how to enquire. This is a fictional classroom demo.”

In Ask mode, say: “Read brief.txt. Suggest a simple page layout using HTML, CSS and JavaScript. Use index.html, styles.css and app.js. Explain your plan before editing.”

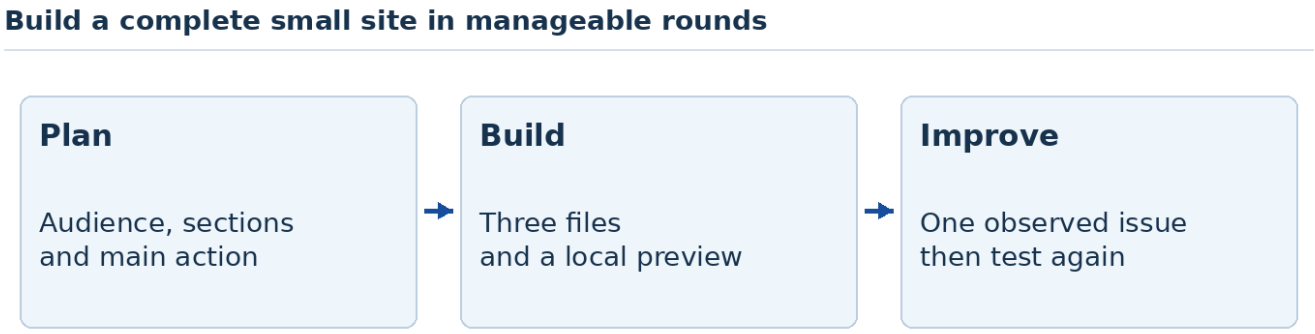
Round two Build

Switch to Agent mode after reviewing the plan. Say: “Build the agreed page. Use three fictional cakes, readable text and a blue-and-white design. Make it work on a narrow screen. Use no external packages. Do not add a working order form or payments. Explain how to open it.”

Open index.html in your browser. If Qoder provides a local preview, use its printed address. You can also open this folder in VS Code and use Live Server.

Round three Improve

Take a screenshot using Windows + Shift + S. Select the page area. If image attachments are available in your Qoder chat, attach it and ask for one specific improvement. Otherwise describe the problem and the screen width.



Teaching illustration • Simplified view

Success: all three files are present, the contact section is reachable and the page reads clearly on a narrow screen. Save a checkpoint before the next lesson.

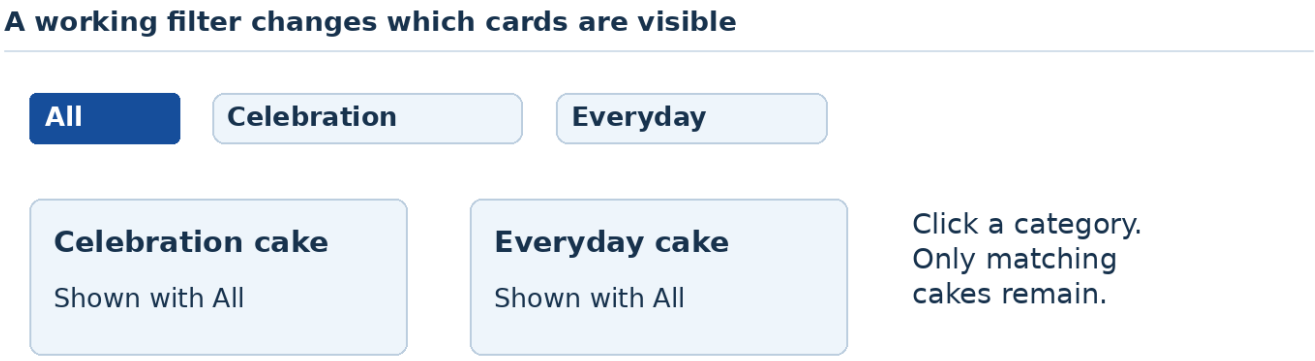
Further lesson 26 Give AI a clear finished test

Use: the same final project. Choose the task that matches your progress.

Everyone Add a useful interaction

Ask Qoder Agent:

“Add buttons labelled All, Celebration and Everyday above the cake cards. Clicking a button should show only the matching cakes. Keep All selected on first load. Use real buttons, show which filter is selected, and keep keyboard use working. Use only our three existing files.”



Teaching illustration • Simplified view

Test before accepting the work:

1. Refresh. All cakes should be visible.
2. Click Celebration. Only cakes in that category should remain.
3. Click Everyday, then All. Check both results.
4. Press Tab to reach a filter button. Press Enter. It should work without the mouse.
5. Narrow the browser and repeat the test.

If it fails, tell Qoder: “I clicked Celebration. Everyday cakes still showed. I expected only Celebration cakes. Find the cause and fix only this behaviour.”

Advanced option Use your Laravel project

Open bakery-laravel in Qoder. Ask it to inspect the enquiry feature from Further lesson 18 and propose tests for valid, empty and invalid submissions. Read each test, run it and check the database yourself. Keep the project local until you have completed the publishing checklist and obtained experienced review for real customer data.

Your achievement is not the size of the AI reply. It is a feature that you can explain, use and test.

Further lesson 27 Put the simple website online

Use: a GitHub account and the browser. This route is for the HTML project only.

GitHub Pages can host our static files. It does not run Laravel, Django or WordPress's PHP server. Use fictional public content for this exercise.

1. Sign in at github.com. Click + near the top, then New repository. Name it ada-bakes-demo. Choose Public and create the repository. Public means other people can see these files.

2. On the empty repository page, choose uploading an existing file. If files already exist, choose Add file → Upload files.

3. Drag in index.html, styles.css, app.js and your images folder. Make sure index.html is at the top level, not inside another folder. Write “First website” as the commit message and commit the files.

4. Open Settings → Pages. Under Build and deployment, choose Deploy from a branch. Select main and / (root), then Save.

5. Wait for GitHub's deployment to finish. Return to Pages and open the site address shown there.

GitHub Pages publishes the static files you upload



Teaching illustration • Simplified view

Use paths such as images/cake.jpg, not C:\Users\... paths. Filenames and capital letters must match exactly.

Open the public link on another device or in a private browser window. Check pictures, filters and contact links. If an image is missing online but works locally, compare its exact spelling and folder path.

Do not upload passwords, private files or .env files. For changes, upload the corrected files and wait for the new deployment. Keep a local copy too.

Source: S33. A GitHub account's eligible plan and repository visibility affect Pages availability.

Further lesson 28 Show what you can do

Use: your finished website, the browser and your project folder.

Open your website and say: “This is who it is for. This is what a visitor should do first.” Then demonstrate the work instead of reading a long report.

Your final check

- I can find the project folder and open the right file.
- I can change a heading, save it and see the change.
- I can explain what HTML, CSS and JavaScript do here.
- The page is readable on a narrow screen and at 200% zoom.
- I can reach buttons and links with Tab and activate them.
- Pictures have useful alternative text, or empty alt text if decorative.
- My links and filters do what their labels promise.
- I can show a backup or saved Git version.
- I can explain one AI mistake that I found and corrected.
- The public website uses appropriate, accurate content.

Hand over the project

Create README.txt in your project folder. Write its name, how to open it, what works, what is only a demonstration, and how to change the main text. Include the live address if you published it.

If possible, invite someone to try the website without your help. When working alone, close your notes and repeat the main task as a visitor. Write down one confusing moment and improve it.

You began by learning where to click. You can now create files, build a page, improve its design and use AI with judgment. Your next goal is one small real project, completed carefully.

Your completion check: do each practical task without copying the answer. Revisit any task you can describe but cannot yet perform.

Your course completion record

Write the date beside each result you can produce without copying the answer.

I can do this	Date and project
Create folders and save a real HTML file	_____
Explain browser, server, DNS and hosting	_____
Finish all five HTML practicals	_____
Finish all five CSS practicals	_____
Finish all five JavaScript practicals	_____
Run all five PHP practicals	_____
Run all five SQL practicals	_____
Save and inspect a record in each complete website	_____
Review an AI change and test the result	_____
Publish a static site and check its public link	_____

READY TO CONTINUE Choose the project you understand best. Change its content for a new purpose and explain your choices. That is stronger evidence of learning than a folder full of untouched answer files.

You can now use the contents as a reference. Return to a practical when you need to remember how something works.

Optional tutor notes Teach the person in front of you

The sequence is deliberately slow at the beginning. A student who cannot find a saved file is not ready for a framework installer.

A simple classroom rhythm

Demonstrate one small action. Let everyone repeat it. Walk around and look at their screens. Ask them to change one detail by themselves. End with a visible result.

Use short blocks: about 5 minutes to demonstrate, 15 minutes to practise and 5 minutes to check. These are suggested blocks, not deadlines. Give extra time for typing and mouse practice.

Stage	Move on when the learner can show this
Computer basics	Open Notepad, select text and find a saved file
First webpage	Save index.html correctly and refresh the browser
VS Code	Open the correct folder and edit two linked files
Design practice	Improve spacing and demonstrate a narrow layout
AI	Request one change and check the actual result
Platforms	Build a page and explain draft versus published
Stack sampler	Run one local project and make a visible change
Final build	Demonstrate a tested site and explain its limits

Pair students for discussion, but let each person use the mouse and keyboard. Avoid having the confident partner do all the clicking.

Give this feedback: “You saved the file correctly. Now let us check which folder it is in.” A broken page is a useful problem to investigate, not evidence that someone cannot learn.

Advanced does not mean rushing through more tool names. Revisit one chosen stack until students can understand the generated structure, check data handling and maintain a small working application.

Optional tutor notes Prepare the room and accounts

Before the first class

Check each computer's keyboard, mouse, display, free space and Windows compatibility. Provide a teacher-approved cake.jpg image and a place to save work. Test Notepad's Save As behaviour on those exact computers.

Do not require an AI account for the first success. The opening computer and Notepad exercises can work offline. Download installers in advance where classroom internet is unreliable.

Before later stages

Stage	Prepare and try once on a classroom computer
VS Code	Current supported release and Live Server extension
Codex	Official OpenAI extension, working sign-in and usage access
Platforms	Suitable accounts; WordPress Studio local installation
Laravel	Supported PHP, Composer, Herd and a clean Laravel 13 build
Django	Python 3.12, the py launcher and a Django 5.2 environment
Flutter	Git, SDK, Dart Code extension, Chrome and a working web run
React	Compatible Node.js LTS and the current Vite setup
Qoder	Editor login, selected plan and available Agent allowance
Publishing	GitHub access and a fictional static practice project

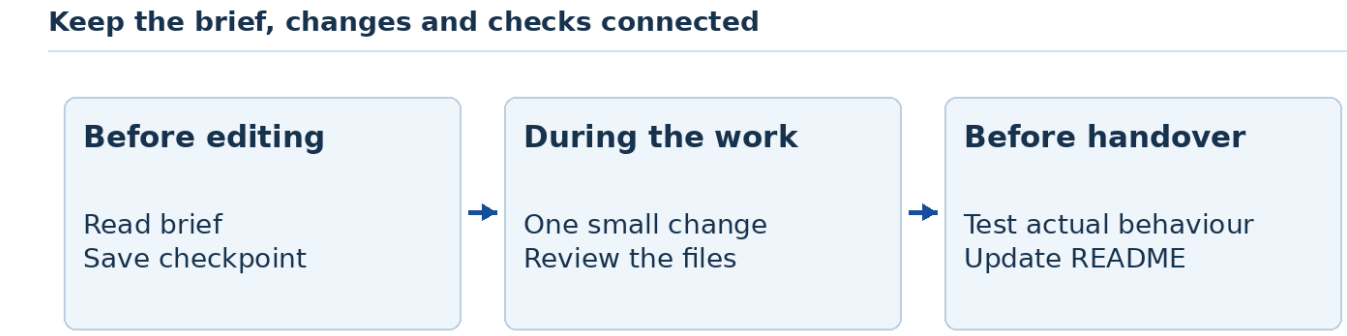
The framework walkthroughs were checked against official documentation; they still need a trial run on your classroom computers. Run every practical once on the classroom setup before teaching it. Downloads and account screens change. Record the versions that worked in a classroom setup sheet. Do not update the whole class during a lesson unless needed.

If an account is unavailable, demonstrate on the teacher's screen and let students continue their local project. Never share one password around the room. Check age and institutional account requirements before arranging student sign-ups.

A tool asking for a subscription is a teaching decision for the teacher, not a problem a beginner should solve alone.

Optional tutor notes Use AI like a professional colleague

Use clear requirements, small changes, saved versions and observed results. This routine works across AI tools.



Teaching illustration • Simplified view

Keep three short files

brief.txt says who the site serves and what it should do. README.txt says how to run it. A test checklist says how you know it works. Keep them with the project so another person can continue the work.

Add tools only when they solve a problem

Tool	Useful moment
Figma or Penpot	Agree on a layout before building it
GitHub Desktop	Save and compare working versions
Browser developer tools	Investigate layout, errors and loading
Prettier for VS Code	Keep formatting consistent after students understand files
Playwright	Later, automate a repeated browser check with teacher support
Documentation connections through MCP	Later, let an AI consult approved technical documentation

MCP is a way for an AI tool to connect to additional tools or information. It is optional here. An MCP connection is different from a WordPress plugin or a VS Code extension. Review its permissions before use.

Use Codex for explanation and code work, and Qoder for the final build as planned. Avoid running two editing agents on the same files at the same time. Give the next tool the project, brief and tests instead of expecting it to remember the previous chat.

For advanced work, ask for evidence from tests. Try the feature yourself.

Sources: S39–S41.

Optional tutor notes Important current tool details

Checked against official documentation on 8 October 2026. Recheck before a new class, especially prices, account access and installation screens.

Qoder

The current documentation describes a Free plan and paid plans. Free usage is limited; an eligible trial may temporarily add paid features and credits. Do not promise students free unlimited Agent work. A personal API key can involve separate provider charges.

Qoder's official notice says the Quest window inside Qoder IDE is due to retire on 15 October 2026 at 23:59 UTC+8. The Editor remains available. This handout therefore teaches Editor chat and Agent mode, not a Quest-dependent course. Recheck the current notice if teaching after that date.

Codex

Use the official OpenAI extension. Available usage depends on the account and plan. Check sign-in and permissions before the lesson. Do not require students to purchase API credits just to follow the manual HTML exercises.

WordPress

WordPress.com is a hosted service. WordPress.org distributes the WordPress software. The software can be free while hosting and a domain still cost money. WordPress.com's current documentation allows plugin installation on paid plans; the free classroom visit does not depend on that feature.

Dreamweaver and Sites

Dreamweaver is an optional demonstration using an appropriately licensed installation. Google Sites and ChatGPT Sites are different products. ChatGPT Sites availability and sharing controls depend on the account and workspace.

Sources: S6, S8, S11, S19, S21, S31, S32, S42. These optional notes explain tool details without interrupting the first lessons.

Optional tutor notes Publish the right kind of project

A domain is a name people type, such as example.com. Hosting is the service that makes a website available online. A local development server is not production hosting.

Project	What the host must support
HTML, CSS and JavaScript	Static files; GitHub Pages suits the class exercise
React built with Vite	Run npm run build; deploy the dist output with the correct base path
Flutter web	Run flutter build web; deploy build/web through a web host
Laravel	Supported PHP, web server, application settings and database
Django	Supported Python, production application server, settings and database
WordPress.org software	Supported WordPress hosting, database and managed updates

A supervised production workflow

Choose a host that explicitly supports your stack. Read its current deployment guide. First deploy a test copy, often called staging. Set secret configuration through the host's settings, not public source files. Use a supported production server rather than artisan serve or Django runserver.

Run the database setup safely, configure HTTPS, turn off debug output and test the real hosted address. Check file uploads and database storage survive restarts. Make a backup and practise restoring it before handling real customer information.

For a public form, check validation, spam controls, restricted access and retention. For sign-in or payment, use well-maintained framework features and established providers with experienced review. A generated demo is not a security assessment.

Advanced assignment: ask AI to draft a deployment checklist for the chosen host and exact project. Verify every step against that host's official guide before acting.

Sources: S25, S27, S33, S43, S44. Hosting choices, free tiers and billing change; confirm them at the time of use.

Quick help When something goes wrong

Start with what you can see. Change one thing, then test again.

What you see	First thing to check
Code appears instead of a page	Is it index.html rather than index.html.txt?
The old words remain	Did you save, refresh and open the same file?
CSS has no effect	Does the link say styles.css and match the file?
A picture is missing	Check its folder, filename, extension and letter case
Live Server does not start	Open the folder and right-click index.html
A local address will not load	Is its server still running? Use the printed address
A terminal command fails	Read the first error; check tool installation and folder
An AI change breaks something	Return to the saved version and request a smaller fix
A form says success	Check the record or delivery; do not trust text alone
A published page looks old	Wait for deployment, refresh and check the published files

Ask for help clearly

Write this when asking a person or AI for help:

“I am using [tool]. I opened [file]. I did [action]. I expected [result]. Instead I saw [message or behaviour].”

For example: “In VS Code I changed styles.css and saved. I expected a blue heading. It stayed black. My HTML link says style.css.”

Include a screenshot or the exact error when useful. Remove passwords, private names and account details first. Do not ask AI to rebuild the whole project for a small spelling mistake.

Small glossary Words you have already used

Word	Everyday meaning in this course
File	One saved item, such as index.html
Folder	A container that keeps files together
Browser	The program that opens webpages, such as Edge
Editor	The program where you change code, such as VS Code
HTML	The page's content and structure
CSS	Rules for its appearance and layout
JavaScript	Code that can respond to actions on the page
Extension	An extra feature installed in an editor
Plugin	An add-on for a product such as WordPress
Responsive	A layout that adapts to available screen space
Terminal	A panel where you type commands
Local	Running or stored on your own computer
Framework	A prepared structure and tools for building applications
Database	Organised saved information
Repository	A project tracked with version history
Commit	A recorded set of changes with a description
Deploy	Put a working version on its hosting service
Debug	Find and fix the cause of a problem

You do not need to memorise this table. Return to a word when you meet it during a task. Understanding grows through use.

Official sources 1

These links support the tool instructions. Checked 8 October 2026. Open a reference when you need more detail or a menu has changed.

S1 Windows file extensions

https://support.microsoft.com/en-us/windows/experience/storage-filemanagement/common-file-name-extensions-in-windows

S2 VS Code Windows setup

https://code.visualstudio.com/docs/setup/windows

S3 VS Code requirements

https://code.visualstudio.com/docs/supporting/requirements

S4 VS Code interface

https://code.visualstudio.com/docs/editing/getting-started/userinterface

S5 Live Server by Ritwick Dey

https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer

S6 Codex IDE extension

https://learn.chatgpt.com/docs/codex/ide

S7 GitHub Desktop first repository

https://docs.github.com/en/desktop/overview/creating-your-first-repository-using-github-desktop

S8 WordPress com and org compared

https://wordpress.com/support/com-vs-org/

S9 WordPress com getting started

https://wordpress.com/support/getting-started-with-wordpress-com/

S10 WordPress com pages

https://wordpress.com/support/pages/

S11 WordPress com plugins and plans

https://wordpress.com/support/plugins/

Official sources 2

These links support the tool instructions. Checked 8 October 2026. Open a reference when you need more detail or a menu has changed.

S12 WordPress Studio setup

https://developer.wordpress.com/docs/developer-tools/studio/

S13 WordPress Studio sites

https://developer.wordpress.com/docs/developer-tools/studio/sites/

S14 WordPress Playground quick start

https://developer.wordpress.org/playground/handbook/quick-start-guide/

S15 Hello Dolly plugin publisher and behaviour

https://wordpress.org/plugins/hello-dolly/

S16 WordPress Studio import and export

https://developer.wordpress.com/docs/developer-tools/studio/import-export/

S17 Blogger create a blog

https://support.google.com/blogger/answer/1623800

S18 Google Sites create and publish

https://support.google.com/sites/answer/6372878

S19 Dreamweaver new documents

https://helpx.adobe.com/dreamweaver/desktop/manage-files/creating-opening-documents.html

S20 Penpot design platform

https://penpot.app/

S21 ChatGPT Sites and sharing

https://learn.chatgpt.com/docs/sites

S22 Laravel 13 installation

https://laravel.com/framework/docs/13.x/installation

Official sources 3

These links support the tool instructions. Checked 8 October 2026. Open a reference when you need more detail or a menu has changed.

S23 Django 5.2 first application

https://docs.djangoproject.com/en/5.2/intro/tutorial01/

S24 Flutter installation in VS Code

https://docs.flutter.dev/install/with-vs-code

S25 Flutter web build and deployment

https://docs.flutter.dev/platform-integration/web/building

S26 React learning guide

https://react.dev/learn

S27 Vite getting started

https://vite.dev/guide/

S28 Laravel Herd Windows installation

https://herd.laravel.com/docs/windows/getting-started/installation

S29 Laravel validation

https://laravel.com/docs/13.x/validation

S30 Django on Windows

https://docs.djangoproject.com/en/5.2/howto/windows/

S31 Qoder quick start

https://docs.qoder.com/quick-start

S32 Qoder plans and usage

https://docs.qoder.com/account/pricing

S33 GitHub Pages create a site

https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site

Official sources 4

These links support the tool instructions. Checked 8 October 2026. Open a reference when you need more detail or a menu has changed.

S34 MDN CSS box model

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Box_model

S35 MDN CSS Grid

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Grids

S36 W3C accessible page structure

https://www.w3.org/WAI/tutorials/page-structure/

S37 Google SEO starter guide

https://developers.google.com/search/docs/fundamentals/seo-starter-guide

S38 Chrome Lighthouse

https://developer.chrome.com/docs/lighthouse/overview/

S39 Prettier VS Code extension

https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode

S40 Playwright browser testing

https://playwright.dev/docs/intro

S41 OpenAI documentation MCP

https://developers.openai.com/learn/docs-mcp

S42 Qoder Quest retirement notice

https://docs.qoder.com/release-notes/quest-retirement-notice

S43 Laravel deployment

https://laravel.com/framework/docs/13.x/deployment

S44 Django deployment checklist

https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

Additional reference for choosing Flutter: https://docs.flutter.dev/platform-integration/web/faq

Django admin extension reference

https://docs.djangoproject.com/en/5.2/intro/tutorial02/

Optional tutor notes Assess the new foundation practicals

Use a demonstration, not only a written answer. For each of the 25 separate practicals, check four things: the student created the files, obtained the expected output, explained the relevant code and completed the change task.

For each of the five websites, check navigation, the catalogue filter, server validation, a saved record, readable mobile layout and clear limits on what the site does.

A student who has copied a page but cannot find the file needs more file practice. A student whose form says success but has no stored row needs help tracing the request. Use the error as the next lesson.

The separate SQL exercises run in DB Browser, not inside a webpage. Do not skip them because a generated application already has a database.

The companion files are answer copies. Keep clean copies so learners can compare changes. A sensible class pace is one practical over as many sessions as needed; do not force all five into one lesson.

Before public use of a complete project, review production hosting, session settings, secrets, backups, rate limits, privacy and accessibility. The guesthouse and event examples accept requests; they do not allocate stock, dates or seats. Add those features only with explicit requirements and appropriate tests.

Foundation sources 1

Official references for the added lessons. Checked during this revision on 12–13 October 2026.

N1 Internet infrastructure

https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/How_does_the_Internet_work

N2 How the web works

https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works

N3 URL components

https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL

N4 HTTP overview

https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview

N5 Browser network panel

https://developer.chrome.com/docs/devtools/network/

N6 HTML elements

https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements

N7 JavaScript guide

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

N8 PHP tutorial

https://www.php.net/manual/en/tutorial.php

Foundation sources 2

Official references for the added lessons. Checked during this revision on 12–13 October 2026.

N9 DB Browser installation

https://sqlitebrowser.org/dl/

N10 SQLite SQL reference

https://www.sqlite.org/lang.html

N11 CSS reference

https://developer.mozilla.org/en-US/docs/Web/CSS/Reference

N12 Prepared SQL with PDO

https://www.php.net/manual/en/pdo.prepare.php

N13 PHP sessions

https://www.php.net/manual/en/book.session.php

N14 PHP development server

https://www.php.net/manual/en/features.commandline.webserver.php

N15 PHP validation filters

https://www.php.net/manual/en/filter.constants.php

N16 PHP output escaping

https://www.php.net/manual/en/function.htmlspecialchars.php